

路与连通

School of Computer
Wuhan University

1 路径

- 路径与基本路径
- 连通性
- 回路 with 基本回路
- 距离与直径

2 路径的应用

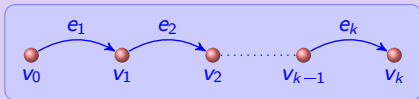
- 赋权图与最短路径
- Euler路径与回路
- Hamilton路径与回路

路径

- 路径与基本路径
- 连通性
- 回路与基本回路
- 距离与直径

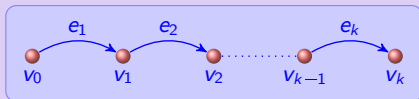
- 赋权图与最短路径
- Euler路径与回路
- Hamilton路径与回路

路径、简单路径和基本路径



设 $D = \langle V, E \rangle$ (或 G) 是一图, $P = (v_1 e_1 v_2 e_2 v_3 \cdots v_k e_k v_{k+1})$ 是一结点与边交错序列, 且序列中相邻的点与边关联, 称这样的序列为**路径(链, 通路, walk)**;
更进一步:

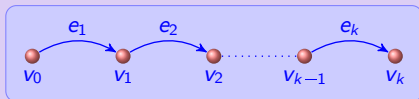
路径、简单路径和基本路径



设 $D = \langle V, E \rangle$ (或 G) 是一图, $P = (v_1 e_1 v_2 e_2 v_3 \cdots v_k e_k v_{k+1})$ 是一结点与边交错序列, 且序列中相邻的点与边关联, 称这样的序列为**路径(链, 通路, walk)**;
更进一步:

- e_1 的起点(端点)是路径的**起点(端点)**; e_k 的终点(端点)是路径的**终点(端点)**, 其余的点称为**内部点**.
- 序列中边的个数 **k** 称为路径的**长度**.
- 同一条边在 **P** 中仅出现一次, 称为简单路径(trail, 迹);
- 同一结点在 **P** 中仅出现一次, 称为基本路径(path, 路);
- **P** 的起点点和终点相同称为回路(closed, 闭链);
- 简单回路(circuit, 闭迹), 基本回路(圈)(cycle);
- 对简单图 **P** 可简记为 $(v_1, v_2, \dots, v_n)$ 或 (v_1, v_n) .

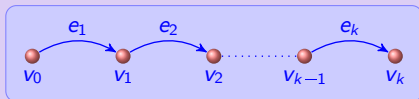
路径、简单路径和基本路径



设 $D = \langle V, E \rangle$ (或 G) 是一图, $P = (v_1 e_1 v_2 e_2 v_3 \cdots v_k e_k v_{k+1})$ 是一结点与边交错序列, 且序列中相邻的点与边关联, 称这样的序列为**路径(链, 通路, walk)**;
更进一步:

- e_1 的起点(端点)是路径的**起点(端点)**; e_k 的终点(端点)是路径的**终点(端点)**, 其余的点称为**内部点**.
- 序列中边的个数 **k** 称为路径的**长度**.
- 同一条边在 **P** 中仅出现一次, 称为简单路径(trail, 迹);
- 同一结点在 **P** 中仅出现一次, 称为基本路径(path, 路);
- **P** 的起点点和终点相同称为回路(closed, 闭链);
- 简单回路(circuit, 闭迹), 基本回路(圈)(cycle);
- 对简单图 **P** 可简记为 $(v_1, v_2, \dots, v_n)$ 或 (v_1, v_n) .

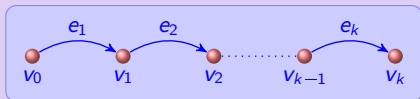
路径、简单路径和基本路径



设 $D = \langle V, E \rangle$ (或 G) 是一图, $P = (v_1 e_1 v_2 e_2 v_3 \cdots v_k e_k v_{k+1})$ 是一结点与边交错序列, 且序列中相邻的点与边关联, 称这样的序列为**路径(链, 通路, walk)**;
更进一步:

- e_1 的起点(端点)是路径的**起点(端点)**; e_k 的终点(端点)是路径的**终点(端点)**, 其余的点称为**内部点**.
- 序列中边的个数 **k** 称为路径的**长度**.
- 同一条边在 **P** 中仅出现一次, 称为**简单路径(trail, 迹)**;
- 同一结点在 **P** 中仅出现一次, 称为**基本路径(path, 路)**;
- **P** 的起点点和终点相同称为**回路(closed, 闭链)**;
- **简单回路(circuit, 闭迹)**, **基本回路(圈)(cycle)**;
- 对简单图 **P** 可简记为 **$(v_1, v_2, \dots, v_n)$** 或 **(v_1, v_n)** .

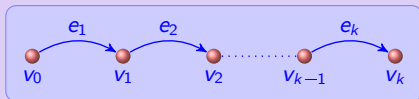
路径、简单路径和基本路径



设 $D = \langle V, E \rangle$ (或 G) 是一图, $P = (v_1 e_1 v_2 e_2 v_3 \cdots v_k e_k v_{k+1})$ 是一结点与边交错序列, 且序列中相邻的点与边关联, 称这样的序列为**路径(链, 通路, walk)**;
更进一步:

- e_1 的起点(端点)是路径的**起点(端点)**; e_k 的终点(端点)是路径的**终点(端点)**, 其余的点称为**内部点**.
- 序列中边的个数 **k** 称为路径的**长度**.
- 同一条边在 **P** 中仅出现一次, 称为**简单路径(trail, 迹)**;
- 同一结点在 **P** 中仅出现一次, 称为**基本路径(path, 路)**;
- P 的起点点和终点相同称为**回路(closed, 闭链)**;
- **简单回路(circuit, 闭迹)**, **基本回路(圈)(cycle)**;
- 对简单图 **P** 可简记为 **$(v_1, v_2, \dots, v_n)$** 或 **(v_1, v_n)** .

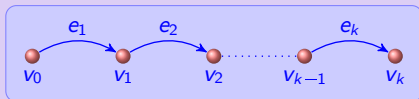
路径、简单路径和基本路径



设 $D = \langle V, E \rangle$ (或 G) 是一图, $P = (v_1 e_1 v_2 e_2 v_3 \cdots v_k e_k v_{k+1})$ 是一结点与边交错序列, 且序列中相邻的点与边关联, 称这样的序列为**路径(链, 通路, walk)**;
更进一步:

- e_1 的起点(端点)是路径的**起点(端点)**; e_k 的终点(端点)是路径的**终点(端点)**, 其余的点称为**内部点**.
- 序列中边的个数 **k** 称为路径的**长度**.
- 同一条边在 **P** 中仅出现一次, 称为**简单路径(trail, 迹)**;
- 同一结点在 **P** 中仅出现一次, 称为**基本路径(path, 路)**;
- **P** 的起点点和终点相同称为**回路(closed, 闭链)**;
- **简单回路(circuit, 闭迹)**, **基本回路(圈)(cycle)**;
- 对简单图 **P** 可简记为 **$(v_1, v_2, \dots, v_n)$** 或 **(v_1, v_n)** .

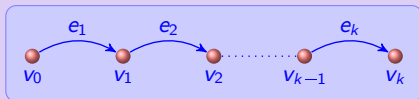
路径、简单路径和基本路径



设 $D = \langle V, E \rangle$ (或 G) 是一图, $P = (v_1 e_1 v_2 e_2 v_3 \cdots v_k e_k v_{k+1})$ 是一结点与边交错序列, 且序列中相邻的点与边关联, 称这样的序列为**路径(链, 通路, walk)**;
更进一步:

- e_1 的起点(端点)是路径的**起点(端点)**; e_k 的终点(端点)是路径的**终点(端点)**, 其余的点称为**内部点**.
- 序列中边的个数 **k** 称为路径的**长度**.
- 同一条边在 **P** 中仅出现一次, 称为**简单路径(trail, 迹)**;
- 同一结点在 **P** 中仅出现一次, 称为**基本路径(path, 路)**;
- **P** 的起点点和终点相同称为**回路(closed, 闭链)**;
- **简单回路(circuit, 闭迹)**, **基本回路(圈)(cycle)**;
- 对简单图 **P** 可简记为 $(v_1, v_2, \dots, v_n)$ 或 (v_1, v_n) .

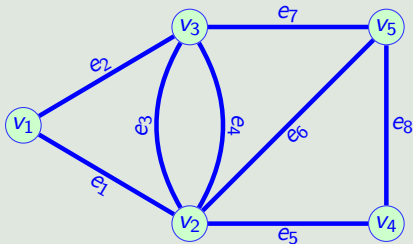
路径、简单路径和基本路径



设 $D = \langle V, E \rangle$ (或 G) 是一图, $P = (v_1 e_1 v_2 e_2 v_3 \cdots v_k e_k v_{k+1})$ 是一结点与边交错序列, 且序列中相邻的点与边关联, 称这样的序列为**路径(链, 通路, walk)**;
更进一步:

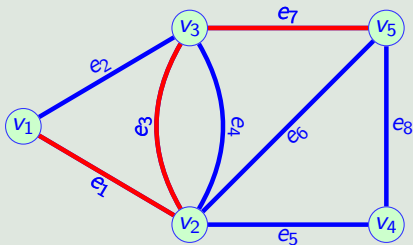
- e_1 的起点(端点)是路径的**起点(端点)**; e_k 的终点(端点)是路径的**终点(端点)**, 其余的点称为**内部点**.
- 序列中边的个数 **k** 称为路径的**长度**.
- 同一条边在 **P** 中仅出现一次, 称为**简单路径(trail, 迹)**;
- 同一结点在 **P** 中仅出现一次, 称为**基本路径(path, 路)**;
- **P** 的起点点和终点相同称为**回路(closed, 闭链)**;
- **简单回路(circuit, 闭迹)**, **基本回路(圈)(cycle)**;
- 对简单图 **P** 可简记为 **$(v_1 v_2 \cdots v_n)$** 或 **(v_1, v_n)** .

Example



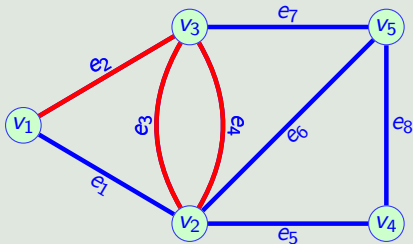
- $(v_1 e_1 v_2 e_3 v_3 e_7 v_5 e_7 v_3 e_7 v_5 e_7 v_3)$ 是路径(有重复的边);
- $(v_1 e_2 v_3 e_3 v_2 e_4 v_3)$ 是简单路径(无复边, 有重复点);
- $(v_1 e_1 v_2 e_5 v_4 e_8 v_5 e_7 v_3)$ 是基本路径(无重复点), 也是简单路径(无重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_3 v_3 e_4 v_2)$ 是回路(端点重合), 但不是简单(有重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_6 v_5 e_8 v_4 e_5 v_2)$ 是简单回路(无重复边), 不是基本回路;
- $(v_2 e_3 v_3 e_7 v_5 e_6 v_2)$ 是基本回路(无重复点), 也是简单回路.

Example



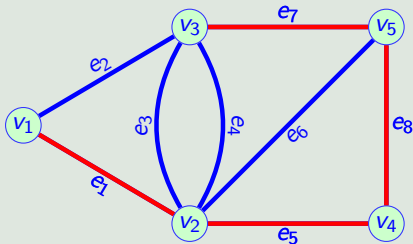
- $(v_1 e_1 v_2 e_3 v_3 e_7 v_5 e_7 v_3 e_7 v_5 e_7 v_3)$ 是路径(有重复的边);
- $(v_1 e_2 v_3 e_3 v_2 e_4 v_3)$ 是简单路径(无复边, 有重复点);
- $(v_1 e_1 v_2 e_5 v_4 e_8 v_5 e_7 v_3)$ 是基本路径(无重复点), 也是简单路径(无重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_3 v_3 e_4 v_2)$ 是回路(端点重合), 但不是简单(有重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_6 v_5 e_8 v_4 e_5 v_2)$ 是简单回路(无重复边), 不是基本回路;
- $(v_2 e_3 v_3 e_7 v_5 e_6 v_2)$ 是基本回路(无重复点), 也是简单回路.

Example



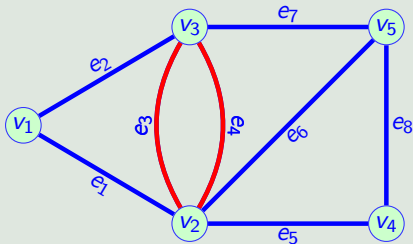
- $(v_1 e_1 v_2 e_3 v_3 e_7 v_5 e_7 v_3 e_7 v_5 e_7 v_3)$ 是路径(有重复的边);
- $(v_1 e_2 v_3 e_3 v_2 e_4 v_3)$ 是简单路径(无复边, 有重复点);
- $(v_1 e_1 v_2 e_5 v_4 e_8 v_5 e_7 v_3)$ 是基本路径(无重复点), 也是简单路径(无重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_3 v_3 e_4 v_2)$ 是回路(端点重合), 但不是简单(有重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_6 v_5 e_8 v_4 e_5 v_2)$ 是简单回路(无重复边), 不是基本回路;
- $(v_2 e_3 v_3 e_7 v_5 e_6 v_2)$ 是基本回路(无重复点), 也是简单回路.

Example



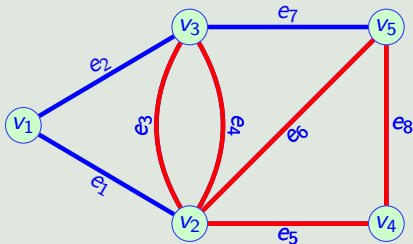
- $(v_1 e_1 v_2 e_3 v_3 e_7 v_5 e_7 v_3 e_7 v_5 e_7 v_3)$ 是路径(有重复的边);
- $(v_1 e_2 v_3 e_3 v_2 e_4 v_3)$ 是简单路径(无复边, 有重复点);
- $(v_1 e_1 v_2 e_5 v_4 e_8 v_5 e_7 v_3)$ 是基本路径(无重复点), 也是简单路径(无重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_3 v_3 e_4 v_2)$ 是回路(端点重合), 但不是简单(有重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_6 v_5 e_8 v_4 e_5 v_2)$ 是简单回路(无重复边), 不是基本回路;
- $(v_2 e_3 v_3 e_7 v_5 e_6 v_2)$ 是基本回路(无重复点), 也是简单回路.

Example



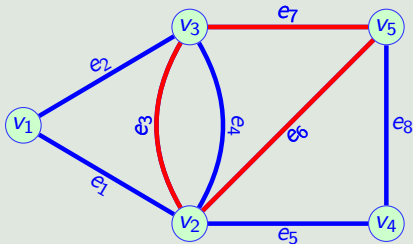
- $(v_1 e_1 v_2 e_3 v_3 e_7 v_5 e_7 v_3 e_7 v_5 e_7 v_3)$ 是路径(有重复的边);
- $(v_1 e_2 v_3 e_3 v_2 e_4 v_3)$ 是简单路径(无复边, 有重复点);
- $(v_1 e_1 v_2 e_5 v_4 e_8 v_5 e_7 v_3)$ 是基本路径(无重复点), 也是简单路径(无重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_3 v_3 e_4 v_2)$ 是回路(端点重合), 但不是简单(有重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_6 v_5 e_8 v_4 e_5 v_2)$ 是简单回路(无重复边), 不是基本回路;
- $(v_2 e_3 v_3 e_7 v_5 e_6 v_2)$ 是基本回路(无重复点), 也是简单回路.

Example



- $(v_1 e_1 v_2 e_3 v_3 e_7 v_5 e_7 v_3 e_7 v_5 e_7 v_3)$ 是路径(有重复的边);
- $(v_1 e_2 v_3 e_3 v_2 e_4 v_3)$ 是简单路径(无复边, 有重复点);
- $(v_1 e_1 v_2 e_5 v_4 e_8 v_5 e_7 v_3)$ 是基本路径(无重复点), 也是简单路径(无重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_3 v_3 e_4 v_2)$ 是回路(端点重合), 但不是简单(有重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_6 v_5 e_8 v_4 e_5 v_2)$ 是简单回路(无重复边), 不是基本回路;
- $(v_2 e_3 v_3 e_7 v_5 e_6 v_2)$ 是基本回路(无重复点), 也是简单回路。

Example



- $(v_1 e_1 v_2 e_3 v_3 e_7 v_5 e_7 v_3 e_7 v_5 e_7 v_3)$ 是路径(有重复的边);
- $(v_1 e_2 v_3 e_3 v_2 e_4 v_3)$ 是简单路径(无复边, 有重复点);
- $(v_1 e_1 v_2 e_5 v_4 e_8 v_5 e_7 v_3)$ 是基本路径(无重复点), 也是简单路径(无重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_3 v_3 e_4 v_2)$ 是回路(端点重合), 但不是简单(有重复边);
- $(v_2 e_3 v_3 e_4 v_2 e_6 v_5 e_8 v_4 e_5 v_2)$ 是简单回路(无重复边), 不是基本回路;
- $(v_2 e_3 v_3 e_7 v_5 e_6 v_2)$ 是基本回路(无重复点), 也是简单回路。

相关性质

Propositions

相关性质

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n-1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

相关性质

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n - 1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

相关性质

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n - 1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

相关性质

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n - 1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

Proof.



相关性质

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n-1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

Proof.

- 长度为 k 的路径经过 $k + 1$ 个结点;
- 抽屉原则.



相关性质

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n-1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

Proof.

- 长度为 k 的路径经过 $k + 1$ 个结点;
- 抽屉原则.



相关性质

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n-1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

Proof.

- 长度为 k 的路径经过 $k + 1$ 个结点;
- 抽屉原则.



Definition

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n - 1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

Proof.

- 长度为 k 的路径经过 $k + 1$ 个结点;
- 抽屉原则.



Definition

- 长度最长的基本路径称为**最长路径**;
- 包含每个点的基本路径称为**Hamilton路径**. Hamilton路径的长度等于 $n - 1$.

相关性质

Propositions

- 基本路径也是简单路径, 基本回路也是简单回路;
- 基本路径的长度 $\leq n-1$, 简单路径的长度 $\leq m$;
- 基本回路的长度 $\leq n$, 简单回路的长度 $\leq m$.

Proof.

- 长度为 k 的路径经过 $k + 1$ 个结点;
- 抽屉原则.



Definition

- 长度最长的基本路径称为**最长路径**;
- 包含每个点的基本路径称为**Hamilton路径**. Hamilton路径的长度等于 $n - 1$.

Proposition

若 u 到 v 有路径, 则 u 到 v 也有基本路径.

Proposition

若 u 到 v 有路径, 则 u 到 v 也有基本路径.

Proof(归纳法)

对 u, v 路径的长度 l 用归纳法:

Proposition

若 u 到 v 有路径, 则 u 到 v 也有基本路径.

Proof(归纳法)

对 u, v 路径的长度 l 用归纳法:

- $l = 0$, 即路径 W 无边, 因此 $W = (u)$ 是条基本路径.
- 设 W 的长度 $< l$ 时结论成立, 即若有长度 $< l$ 的 (u, v) 路径, 则有 (u, v) 基本路径.
- 设有长度为 l 的 (u, v) 路径, 若该路径中没有重复结点, W 即是一条 (u, v) 基本路径. 否则:
- 设 l 有重复结点 w , 则从 w 点与边的序列中删除两次 w 出现中间的点与边后还是一条 (u, v) 路径, 且该路径的长度比 l 要小. 根据归纳假设有 (u, v) 基本路径.

- 7/61 -

Proposition

若 u 到 v 有路径, 则 u 到 v 也有基本路径.

对 u, v 路径的长度 l 用归纳法:

- $l = 0$, 即路径 W 无边, 因此 $W = (u)$ 是条基本路径.
- 设 W 的长度 $< l$ 时结论成立, 即若有长度 $< l$ 的 (u, v) 路径, 则有 (u, v) 基本路径.
- 设有长度为 l 的 (u, v) 路径, 若该路径中没有重复结点, W 即是一条 (u, v) 基本路径. 否则:
- 设 l 有重复结点 w , 则从 w 点与边的序列中删除两次 w 出现中间的点与边后还是一条 (u, v) 路径, 且该路径的长度比 l 要小. 根据归纳假设有 (u, v) 基本路径.

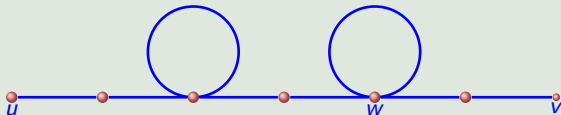
Proposition

若 u 到 v 有路径, 则 u 到 v 也有基本路径.

Proof(归纳法)

对 u, v 路径的长度 l 用归纳法:

- $l = 0$, 即路径 W 无边, 因此 $W = (u)$ 是条基本路径.
- 设 W 的长度 $< l$ 时结论成立, 即若有长度 $< l$ 的 (u, v) 路径, 则有 (u, v) 基本路径.
- 设有长度为 l 的 (u, v) 路径, 若该路径中没有重复结点, W 即是一条 (u, v) 基本路径. 否则:
- 设 l 有重复结点 w , 则从 w 点与边的序列中删除两次 w 出现中间的点与边后还是一条 (u, v) 路径, 且该路径的长度比 l 要小. 根据归纳假设有 (u, v) 基本路径.

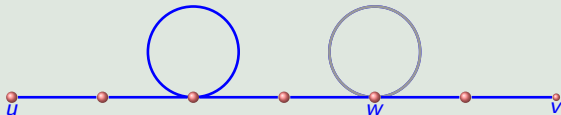


Proposition

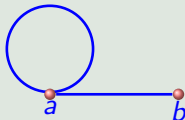
若 u 到 v 有路径, 则 u 到 v 也有基本路径.

Proof(归纳法)

对 u, v 路径的长度 l 用归纳法:

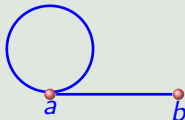


Example 1/2



求上图中从a到b长度为n的路径总数.

Example 1/2



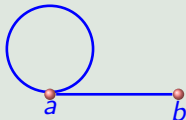
求上图中从 a 到 b 长度为 n 的路径总数.

Solution[利用路径递归增长的属性].

设长度为 n 的路径总数为 s_n , 则:



Example 1/2



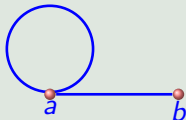
求上图中从a到b长度为n的路径总数.

Solution[利用路径递归增长的属性].

设长度为 n 的路径总数为 s_n , 则:



Example 1/2



求上图中从 a 到 b 长度为 n 的路径总数.

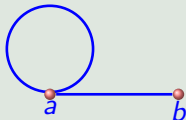
Solution[利用路径递归增长的属性].

设长度为 n 的路径总数为 s_n , 则:

- $s_0 = 0$;
- $s_1 = 1$;
- $s_n = s_{n-1} + s_{n-2}$;
- $s_n = f_{n-1}$ (Fibonacci序列).



Example 1/2



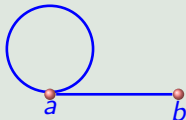
求上图中从 a 到 b 长度为 n 的路径总数.

Solution[利用路径递归增长的属性].

设长度为 n 的路径总数为 s_n , 则:



Example 1/2



求上图中从 a 到 b 长度为 n 的路径总数.

Solution[利用路径递归增长的属性].

设长度为 n 的路径总数为 s_n , 则:



每个竞赛图都含有一个Hamilton有向路径.

100



100

- $n = 2$ 时结论成立;
- 设阶数为 n 的竞赛图 T_n 有一个Hamilton路径 $v_1 v_2 \cdots v_n$;
- 设 T_{n+1} 是增加点 v_{n+1} 的竞赛图, 设 $i \in \{1, 2, \dots, n\}$ 且 i 是最大的使得 $v_i v_{n+1}$ 是图 T_{n+1} 的边的下标.
- 如果 $i = n$, 则 $(v_1 v_2 \cdots v_n v_{n+1})$ 是图 T_{n+1} 的一条Hamilton路径.
- 如果 $i < n$, 则 $v_{i+1} v_{n+1} \notin E_{n+1}$, 这样 $v_{n+1} v_{i+1} \in E_{n+1}$.
即 $(v_1 v_2 \cdots v_i v_{n+1} v_{i+1} \cdots v_n)$ 是 T_{n+1} 的一条Hamilton路径.



Abstract



每个竞赛图都含有一个Hamilton有向路径.

设竞赛图的阶数为 n , 则

- $n = 2$ 时结论成立;
- 设阶数为 n 的竞赛图 T_n 有一个Hamilton路径 $v_1 v_2 \cdots v_n$;
- 设 T_{n+1} 是增加点 v_{n+1} 的竞赛图, 设 $i \in \{1, 2, \dots, n\}$ 且 i 是最大的使得 $v_i v_{n+1}$ 是图 T_{n+1} 的边的下标.
- 如果 $i = n$, 则 $(v_1 v_2 \cdots v_n v_{n+1})$ 是图 T_{n+1} 的一条Hamilton路径.
- 如果 $i < n$, 则 $v_{i+1} v_{n+1} \notin E_{n+1}$, 这样 $v_{n+1} v_{i+1} \in E_{n+1}$.
即 $(v_1 v_2 \cdots v_i v_{n+1} v_{i+1} \cdots v_n)$ 是 T_{n+1} 的一条Hamilton路径.

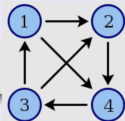


每个竞赛图都含有一个Hamilton有向路径.

设竞赛图的阶数为 n , 则

- $n = 2$ 时结论成立;
- 设阶数为 n 的竞赛图 T_n 有一个Hamilton路径 $v_1 v_2 \cdots v_n$;
- 设 T_{n+1} 是增加点 v_{n+1} 的竞赛图, 设 $i \in \{1, 2, \dots, n\}$ 且 i 是最大的使得 $v_i v_{n+1}$ 是图 T_{n+1} 的边的下标.
- 如果 $i = n$, 则 $(v_1 v_2 \cdots v_n v_{n+1})$ 是图 T_{n+1} 的一条Hamilton路径.
- 如果 $i < n$, 则 $v_{i+1} v_{n+1} \notin E_{n+1}$, 这样 $v_{n+1} v_{i+1} \in E_{n+1}$.
即 $(v_1 v_2 \cdots v_i v_{n+1} v_{i+1} \cdots v_n)$ 是 T_{n+1} 的一条Hamilton路径.





1. **Introduction**

1

连通性

Definition

连通性

Definition

连通性

Definition

连通性

Definition

Proposition

连通性

Definition

Proposition

连通性

Definition

- 设 $G(D)$ 是无向图(有向图), $u, v \in V$, 称 u 到 v 是可达的(accessible), iff, 从 u 到 v 有条有向路径(有向图); u 和 v 之间有条路径(无向图);
- 无向图中如果任意两个结点都是可达的, 称为连通图(connected).

Proposition

连通性

Definition

Proposition

连通性

Definition

Proposition

连通性

Definition

Proposition

-

有向图的连通性

Definition

- 称有向图是**强连通的(Strong connected)**, iff, 任意两个结点 u 和 v , u 到 v 是可达的, 并且 v 到 u 是可达的;
- 称有向图是**单向连通的**, iff, 任意两个结点 u 和 v , u 到 v 是可达的, 或者 v 到 u 是可达的;
- 称有向图是**弱连通的(Weakly connected)**, iff, G 的底图是连通的.

有向图的连通性

Definition

- 称有向图是**强连通的**(Strong connected), iff, 任意两个结点 u 和 v , u 到 v 是可达的, 并且 v 到 u 是可达的;
- 称有向图是**单向连通的**, iff, 任意两个结点 u 和 v , u 到 v 是可达的, 或者 v 到 u 是可达的;
- 称有向图是**弱连通的**(Weakly connected), iff, G 的底图是连通的.

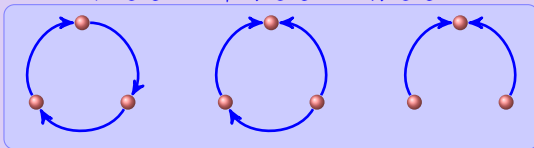
有向图的连通性

Definition

- 称有向图是**强连通的**(Strong connected), iff, 任意两个结点 u 和 v , u 到 v 是可达的, 并且 v 到 u 是可达的;
- 称有向图是**单向连通的**, iff, 任意两个结点 u 和 v , u 到 v 是可达的, 或者 v 到 u 是可达的;
- 称有向图是**弱连通的**(Weakly connected), iff, G 的底图是连通的.

Example

强连通 $\implies$ 单向连通 $\implies$ 弱连通



-

1. *Journal of Management Studies*, 1990, 27, 1, 1-14.

- 定义关系 $\mathcal{R} \subseteq V^2$, $u\mathcal{R}v$ iff u 到 v 可达且 v 到 u 可达. 则 $\mathcal{R}$ 也是强连通图上的等价关系.
- 设 $V/\mathcal{R} = \{V_1, V_2, \dots, V_p\}$, D_i 是 V_i 导出的子图, D_i 称为强连通分支. 一般 $D \neq \bigcup_{i=1}^p D_i$;
- $|V/\mathcal{R}|$ 称为强连通分支数.

强连通分支

Proposition

- 定义关系 $\mathcal{R} \subseteq V^2$, $u\mathcal{R}v$ iff u 到 v 可达且 v 到 u 可达. 则 $\mathcal{R}$ 也是强连通图上的等价关系.
- 设 $V/\mathcal{R} = \{V_1, V_2, \dots, V_p\}$, D_i 是 V_i 导出的子图, D_i 称为强连通分支. 一般 $D \neq \bigcup_{i=1}^p D_i$;
- $|V/\mathcal{R}|$ 称为强连通分支数.

强连通分支

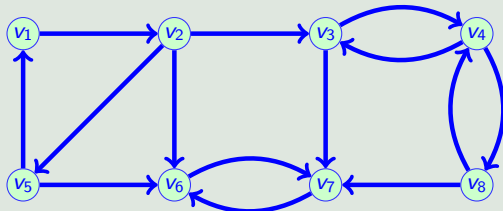
Proposition

- 定义关系 $\mathcal{R} \subseteq V^2$, $u\mathcal{R}v$ iff u 到 v 可达且 v 到 u 可达. 则 $\mathcal{R}$ 也是强连通图上的等价关系.
- 设 $V/\mathcal{R} = \{V_1, V_2, \dots, V_p\}$, D_i 是 V_i 导出的子图, D_i 称为强连通分支. 一般 $D \neq \bigcup_{i=1}^p D_i$;
- $|V/\mathcal{R}|$ 称为强连通分支数.

强连通分支

Proposition

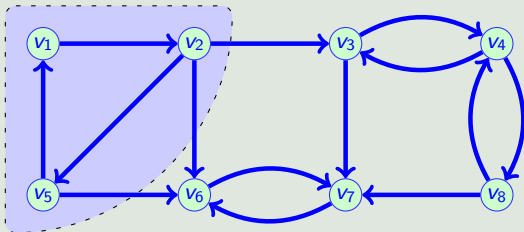
- 定义关系 $\mathcal{R} \subseteq V^2$, $u\mathcal{R}v$ iff u 到 v 可达且 v 到 u 可达. 则 $\mathcal{R}$ 也是强连通图上的等价关系.
- 设 $V/\mathcal{R} = \{V_1, V_2, \dots, V_p\}$, D_i 是 V_i 导出的子图, D_i 称为强连通分支. 一般 $D \neq \bigcup_{i=1}^p D_i$;
- $|V/\mathcal{R}|$ 称为强连通分支数.



强连通分支

Proposition

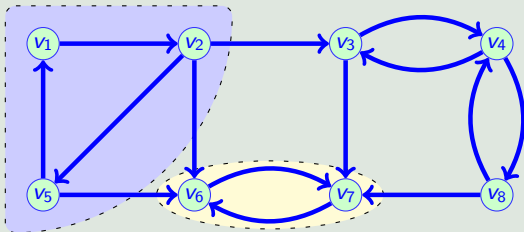
- 定义关系 $\mathcal{R} \subseteq V^2$, $u\mathcal{R}v$ iff u 到 v 可达且 v 到 u 可达. 则 $\mathcal{R}$ 也是强连通图上的等价关系.
- 设 $V/\mathcal{R} = \{V_1, V_2, \dots, V_p\}$, D_i 是 V_i 导出的子图, D_i 称为强连通分支. 一般 $D \neq \bigcup_{i=1}^p D_i$;
- $|V/\mathcal{R}|$ 称为强连通分支数.



强连通分支

Proposition

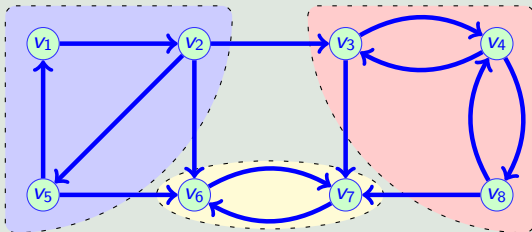
- 定义关系 $\mathcal{R} \subseteq V^2$, $u\mathcal{R}v$ iff u 到 v 可达且 v 到 u 可达. 则 $\mathcal{R}$ 也是强连通图上的等价关系.
- 设 $V/\mathcal{R} = \{V_1, V_2, \dots, V_p\}$, D_i 是 V_i 导出的子图, D_i 称为强连通分支. 一般 $D \neq \bigcup_{i=1}^p D_i$;
- $|V/\mathcal{R}|$ 称为强连通分支数.



强连通分支

Proposition

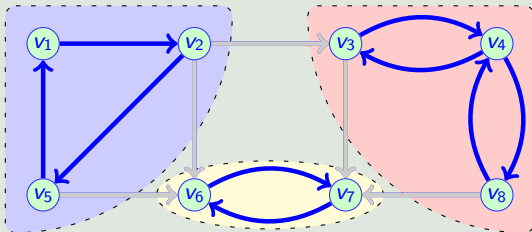
- 定义关系 $\mathcal{R} \subseteq V^2$, $u\mathcal{R}v$ iff u 到 v 可达且 v 到 u 可达. 则 $\mathcal{R}$ 也是强连通图上的等价关系.
- 设 $V/\mathcal{R} = \{V_1, V_2, \dots, V_p\}$, D_i 是 V_i 导出的子图, D_i 称为强连通分支. 一般 $D \neq \bigcup_{i=1}^p D_i$;
- $|V/\mathcal{R}|$ 称为强连通分支数.



强连通分支

Proposition

- 定义关系 $\mathcal{R} \subseteq V^2$, $u\mathcal{R}v$ iff u 到 v 可达且 v 到 u 可达. 则 $\mathcal{R}$ 也是强连通图上的等价关系.
- 设 $V/\mathcal{R} = \{V_1, V_2, \dots, V_p\}$, D_i 是 V_i 导出的子图, D_i 称为强连通分支. 一般 $D \neq \bigcup_{i=1}^p D_i$;
- $|V/\mathcal{R}|$ 称为强连通分支数.



-

-

边集合

Definition

- D 是有向图, S 和 T 是 V 的两个不相交的非空子集合, 定义 $(S, T) = \{e \mid e \in E \wedge \psi(e) = \langle s, t \rangle \wedge s \in S \wedge t \in T\}$;
- $[S, T] = (S, T) \cup (T, S)$;
- G 是无向图时, $[S, T]$ 定义为两个端点分别落在 S 和 T 的边的集合

边集合

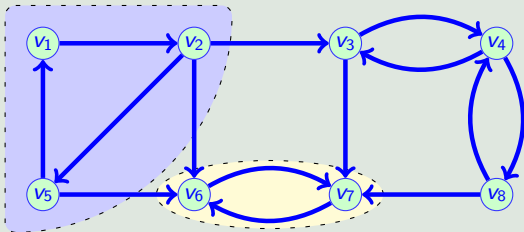
Definition

- D 是有向图, S 和 T 是 V 的两个不相交的非空子集合, 定义
 $(S, T) = \{e \mid e \in E \wedge \psi(e) = \langle s, t \rangle \wedge s \in S \wedge t \in T\}$;
- $[S, T] = (S, T) \cup (T, S)$;
- G 是无向图时, $[S, T]$ 定义为两个端点分别落在 S 和 T 的边的集合.

边集合

Definition

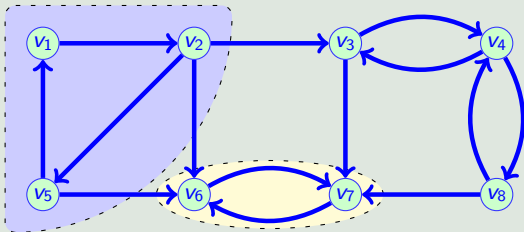
- D 是有向图, S 和 T 是 V 的两个不相交的非空子集合, 定义
 $(S, T) = \{e \mid e \in E \wedge \psi(e) = \langle s, t \rangle \wedge s \in S \wedge t \in T\}$;
- $[S, T] = (S, T) \cup (T, S)$;
- G 是无向图时, $[S, T]$ 定义为两个端点分别落在 S 和 T 的边的集合.



边集合

Definition

- D 是有向图, S 和 T 是 V 的两个不相交的非空子集合, 定义
 $(S, T) = \{e \mid e \in E \wedge \psi(e) = \langle s, t \rangle \wedge s \in S \wedge t \in T\}$;
- $[S, T] = (S, T) \cup (T, S)$;
- G 是无向图时, $[S, T]$ 定义为两个端点分别落在 S 和 T 的边的集合.

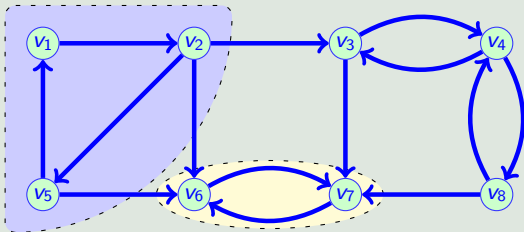


- 设 $S = \{v_1, v_2, v_5\}$, $T = \{v_6, v_7\}$.
- $(S, T) = \{v_2v_6, v_5v_6\}$. $(T, S) = \emptyset$, $[S, T] = (S, T)$.

边集合

Definition

- D 是有向图, S 和 T 是 V 的两个不相交的非空子集合, 定义
 $(S, T) = \{e \mid e \in E \wedge \psi(e) = \langle s, t \rangle \wedge s \in S \wedge t \in T\}$;
- $[S, T] = (S, T) \cup (T, S)$;
- G 是无向图时, $[S, T]$ 定义为两个端点分别落在 S 和 T 的边的集合.



- 设 $S = \{v_1, v_2, v_5\}$, $T = \{v_6, v_7\}$.
- $(S, T) = \{v_2v_6, v_5v_6\}$. $(T, S) = \emptyset$, $[S, T] = (S, T)$.

-

连通图的一个充要条件

Theorem

- ① G 是连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset$
- ② D 是强连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset \wedge [\bar{S}, S] \neq \emptyset$

Proof

$\Rightarrow$ (反证法)

← (反证法)

连通图的一个充要条件

Theorem

- ① G 是连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset$
- ② D 是强连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset \wedge [\bar{S}, S] \neq \emptyset$

Proof

⇒ (反证法)

- 设存在 V 的真子集 S , 使得 $[S, \bar{S}] = \emptyset$.
- 设 $u \in S \wedge v \in \bar{S}$. 设 $ue_1x_1e_2x_2 \cdots x_n e_nv$ 是一条连接 u 和 v 的基本路径.
- 设 $x_i = \max\{j \mid x_j \in S\}$. 则 $x_{i+1} \in \bar{S} \wedge \psi(e_{i+1}) = x_i x_{i+1}$.
- 即 $e_{i+1} \in [S, \bar{S}] \neq \emptyset$. 矛盾.

← (反证法)

连通图的一个充要条件

Theorem

- ① G 是连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset$
- ② D 是强连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset \wedge [\bar{S}, S] \neq \emptyset$

Proof

⇒ (反证法)

- 设存在 V 的真子集 S , 使得 $[S, \bar{S}] = \emptyset$.
- 设 $u \in S \wedge v \in \bar{S}$. 设 $ue_1x_1e_2x_2 \cdots x_n e_nv$ 是一条连接 u 和 v 的基本路径.
- 设 $x_i = \max\{j \mid x_j \in S\}$. 则 $x_{i+1} \in \bar{S} \wedge \psi(e_{i+1}) = x_i x_{i+1}$.
- 即 $e_{i+1} \in [S, \bar{S}] \neq \emptyset$. 矛盾.

← (反证法)

连通图的一个充要条件

Theorem

- ① G 是连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset$
- ② D 是强连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset \wedge [\bar{S}, S] \neq \emptyset$

Proof

⇒ (反证法)

- 设存在 V 的真子集 S , 使得 $[S, \bar{S}] = \emptyset$.
- 设 $u \in S \wedge v \in \bar{S}$. 设 $ue_1x_1e_2x_2 \cdots x_n e_nv$ 是一条连接 u 和 v 的基本路径.
- 设 $x_i = \max\{j \mid x_j \in S\}$. 则 $x_{i+1} \in \bar{S} \wedge \psi(e_{i+1}) = x_i x_{i+1}$.
- 即 $e_{i+1} \in [S, \bar{S}] \neq \emptyset$. 矛盾.

← (反证法)

连通图的一个充要条件

Theorem

- ① G 是连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset$
- ② D 是强连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset \wedge [\bar{S}, S] \neq \emptyset$

Proof

⇒ (反证法)

- 设存在 V 的真子集 S , 使得 $[S, \bar{S}] = \emptyset$.
- 设 $u \in S \wedge v \in \bar{S}$. 设 $ue_1x_1e_2x_2 \cdots x_n e_nv$ 是一条连接 u 和 v 的基本路径.
- 设 $x_i = \max\{j \mid x_j \in S\}$. 则 $x_{i+1} \in \bar{S} \wedge \psi(e_{i+1}) = x_i x_{i+1}$.
- 即 $e_{i+1} \in [S, \bar{S}] \neq \emptyset$. 矛盾.

← (反证法)

连通图的一个充要条件

Theorem

- ① G 是连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset$
- ② D 是强连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset \wedge [\bar{S}, S] \neq \emptyset$

Proof

⇒ (反证法)

- 设存在 V 的真子集 S , 使得 $[S, \bar{S}] = \emptyset$.
- 设 $u \in S \wedge v \in \bar{S}$. 设 $ue_1x_1e_2x_2 \cdots x_n e_nv$ 是一条连接 u 和 v 的基本路径.
- 设 $x_i = \max\{j \mid x_j \in S\}$. 则 $x_{i+1} \in \bar{S} \wedge \psi(e_{i+1}) = x_i x_{i+1}$.
- 即 $e_{i+1} \in [S, \bar{S}] \neq \emptyset$. 矛盾.

← (反证法)

- 设 G 不连通, 则 G 至少有两个连通分支 G_1 和 G_2 ,
 $G = G_1(V_1, E_1) \cup G_2(V_2, E_2)$.
- 则 $V_2 = \overline{V_1}$. $[V_1, \overline{V_1}] = \emptyset$. 矛盾.

连通图的一个充要条件

Theorem

- ① G 是连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset$
- ② D 是强连通图 iff 对 V 的任意非空真子集 S 都有 $[S, \bar{S}] \neq \emptyset \wedge [\bar{S}, S] \neq \emptyset$

Proof

⇒ (反证法)

- 设存在 V 的真子集 S , 使得 $[S, \bar{S}] = \emptyset$.
- 设 $u \in S \wedge v \in \bar{S}$. 设 $ue_1x_1e_2x_2 \cdots x_n e_nv$ 是一条连接 u 和 v 的基本路径.
- 设 $x_i = \max\{j \mid x_j \in S\}$. 则 $x_{i+1} \in \bar{S} \wedge \psi(e_{i+1}) = x_i x_{i+1}$.
- 即 $e_{i+1} \in [S, \bar{S}] \neq \emptyset$. 矛盾.

← (反证法)

- 设 G 不连通, 则 G 至少有两个连通分支 G_1 和 G_2 ,
 $G = G_1(V_1, E_1) \cup G_2(V_2, E_2)$.
- 则 $V_2 = \overline{V_1}$. $[V_1, \overline{V_1}] = \emptyset$. 矛盾.

Example 1

有向图 D 是强连通的, iff, 存在一条经过每个结点的回路.

Example 1

有向图 D 是强连通的, iff, 存在一条经过每个结点的回路.

Proof.



Example 1

有向图 D 是强连通的, iff, 存在一条经过每个结点的回路.

Proof.

$\Rightarrow$ 设 $V = \{v_1, v_2, \dots, v_n\}$ 则:

$$\exists (v_1 \dots v_2) = P_1, (v_2 \dots v_3) = P_3, \dots, (v_n \dots v_1) = P_n;$$

$\therefore P_1 P_2 \cdots P_n$ 是条经过每个结点的回路;



Example 1

有向图 D 是强连通的, iff, 存在一条经过每个结点的回路.

Proof.

$\Rightarrow$ 设 $V = \{v_1, v_2, \dots, v_n\}$ 则:

$$\exists (v_1 \dots v_2) = P_1, (v_2 \dots v_3) = P_3, \dots, (v_n \dots v_1) = P_n;$$

$\therefore P_1 P_2 \cdots P_n$ 是条经过每个结点的回路;

⇐ 设 $P = (u_1 e_1 u_2 \dots u_n e_n u_1)$ 是一条经过每个结点的回路,

$\forall v_i \neq v_j$, 则在 P 中存在 $u_p = v_i \wedge u_q = v_j$, 设 $p \leq q$, 则:

$P' = (u_p \dots u_q)$ 是 v_i 到 v_j 的路径;

$P'' = (u_q \dots u_n e_n u_1 \dots u_p)$ 是 v_j 到 v_i 的路径;

$\therefore D$ 是强连通的.



Example 2 (归纳法)

设 G 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Example 2 (归纳法)

设 G 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Proof(归纳法)

Example 2 (归纳法)

设 G 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Proof(归纳法)

- 设 $n = n \wedge m = m$.
- G 没有孤立点, 否则, 设 v 是孤立点, 则 $G - \{v\}$ 是 $(n-1, m)$ 图, 而 $n-1$ 个结点的简单无向图的最多边数是 $\binom{n-1}{2} < m$, 矛盾.
- 用归纳法证明:

Example 2 (归纳法)

设 G 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Proof(归纳法)

- 设 $n = n \wedge m = m$.
- G 没有孤立点, 否则, 设 v 是孤立点, 则 $G - \{v\}$ 是 $(n-1, m)$ 图, 而 $n-1$ 个结点的简单无向图的最多边数是 $\binom{n-1}{2} < m$, 矛盾.
- 用归纳法证明:

Example 2 (归纳法)

设 G 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Proof(归纳法)

- 设 $n = n \wedge m = m$.
- G 没有孤立点, 否则, 设 v 是孤立点, 则 $G - \{v\}$ 是 $(n-1, m)$ 图, 而 $n-1$ 个结点的简单无向图的最多边数是 $\binom{n-1}{2} < m$, 矛盾.
- 用归纳法证明:
 - ① $n = 3$ 时, 则 $m > 1$, 一定连通;
 - ② 设 $n = k-1$ 时, G 是连通图;
 - ③ 则 $n = k$ 时, 设 $v \in V$, 则 $\deg(v) \geq 1$, 否则是孤立点.
 - if $\deg(v) = n-1$; 则 v 和其他 $n-1$ 个结点相邻, 所以 G 连通.
 - if $1 \leq \deg(v) \leq n-2$, 则 $G' = G - \{v\}$,
 - 其中: $n(G') = n-1$, $m(G') = m - \deg(v)$.
 - $\therefore m(G') = m - \deg(v) > \binom{n-1}{2} - (n-2) = \binom{n-2}{2}$,
 - 由归纳假设 G' 是连通, $\therefore G$ 是连通的.

Example 2 (归纳法)

设 G 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Proof(归纳法)

- 设 $n = n \wedge m = m$.
- G 没有孤立点, 否则, 设 v 是孤立点, 则 $G - \{v\}$ 是 $(n-1, m)$ 图, 而 $n-1$ 个结点的简单无向图的最多边数是 $\binom{n-1}{2} < m$, 矛盾.
- 用归纳法证明:
 - ① $n = 3$ 时, 则 $m > 1$, 一定连通;
 - ② 设 $n = k-1$ 时, G 是连通图;
 - ③ 则 $n = k$ 时, 设 $v \in V$, 则 $\deg(v) \geq 1$, 否则是孤立点.
 - if $\deg(v) = n-1$; 则 v 和其他 $n-1$ 个结点相邻, 所以 G 连通.
 - if $1 \leq \deg(v) \leq n-2$, 则 $G' = G - \{v\}$,
 - 其中: $n(G') = n-1$, $m(G') = m - \deg(v)$.
 - $\therefore m(G') = m - \deg(v) > \binom{n-1}{2} - (n-2) = \binom{n-2}{2}$,
 - 由归纳假设 G' 是连通, $\therefore G$ 是连通的.

Example 2 (归纳法)

设 G 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Proof(归纳法)

- 设 $n = n \wedge m = m$.
- G 没有孤立点, 否则, 设 v 是孤立点, 则 $G - \{v\}$ 是 $(n-1, m)$ 图, 而 $n-1$ 个结点的简单无向图的最多边数是 $\binom{n-1}{2} < m$, 矛盾.
- 用归纳法证明:
 - ① $n = 3$ 时, 则 $m > 1$, 一定连通;
 - ② 设 $n = k-1$ 时, G 是连通图;
 - ③ 则 $n = k$ 时, 设 $v \in V$, 则 $\deg(v) \geq 1$, 否则是孤立点.
 - if $\deg(v) = n-1$; 则 v 和其他 $n-1$ 个结点相邻, 所以 G 连通.
 - if $1 \leq \deg(v) \leq n-2$, 则 $G' = G - \{v\}$,
 - 其中: $n(G') = n-1$, $m(G') = m - \deg(v)$.
 - $\therefore m(G') = m - \deg(v) > \binom{n-1}{2} - (n-2) = \binom{n-2}{2}$,
 - 由归纳假设 G' 是连通, $\therefore G$ 是连通的.

Example 2 (归纳法)

设 G 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Proof(归纳法)

- 设 $n = n \wedge m = m$.
- G 没有孤立点, 否则, 设 v 是孤立点, 则 $G - \{v\}$ 是 $(n-1, m)$ 图, 而 $n-1$ 个结点的简单无向图的最多边数是 $\binom{n-1}{2} < m$, 矛盾.
- 用归纳法证明:
 - ① $n = 3$ 时, 则 $m > 1$, 一定连通;
 - ② 设 $n = k-1$ 时, G 是连通图;
 - ③ 则 $n = k$ 时, 设 $v \in V$, 则 $\deg(v) \geq 1$, 否则是孤立点.
 - if $\deg(v) = n-1$; 则 v 和其他 $n-1$ 个结点相邻, 所以 G 连通.
 - if $1 \leq \deg(v) \leq n-2$, 则 $G' = G - \{v\}$,
 - 其中: $n(G') = n-1$, $m(G') = m - \deg(v)$.
 - $\therefore m(G') = m - \deg(v) > \binom{n-1}{2} - (n-2) = \binom{n-2}{2}$,
 - 由归纳假设 G' 是连通, $\therefore G$ 是连通的.

Example 2 (2)

设 $G(V, E)$ 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Example 2 (2)

设 $G(V, E)$ 是简单无向图, 其中 $n \geq 3$, 如果 m 满足下述条件, 则 G 是连通图:

$$m > \frac{(n-1)(n-2)}{2} = \binom{n-1}{2}$$

Proof(反证法)

- 设 G 不连通, 则 $G = G_1(V_1, E_1) \cup G_2(V_2, E_2)$.
- $n = n_1 + n_2 \wedge n_1 \leq n-1 \wedge n_2 \leq n-1$.

$$\begin{aligned} m &= m_1 + m_2 \\ &\leq \frac{n_1(n_1 - 1)}{2} + \frac{n_2(n_2 - 1)}{2} \\ &\leq \frac{(n - 1)(n_1 - 1)}{2} + \frac{(n - 1)(n_2 - 1)}{2} \\ &= \frac{(n - 1)(n_1 + n_2 - 2)}{2} \\ &= \frac{(n - 1)(n - 2)}{2} \end{aligned}$$

矛盾.

回路和基本回路

Definition

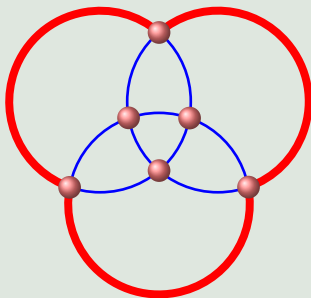
Definition

- C_n 表示长度为 n 的基本回路.
- 长度为3的基本回路称为三角形.
- 长度最小基本回路的长度称为围长(girth), 记为 $g(D)$.

回路和基本回路

Definition

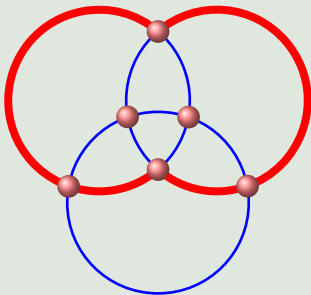
- C_n 表示长度为 n 的基本回路.
- 长度为3的基本回路称为三角形.
- 长度最小基本回路的长度称为围长(girth), 记为 $g(D)$.



回路和基本回路

Definition

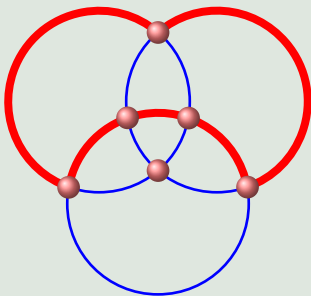
- C_n 表示长度为 n 的基本回路.
- 长度为3的基本回路称为三角形.
- 长度最小基本回路的长度称为围长(girth), 记为 $g(D)$.



回路和基本回路

Definition

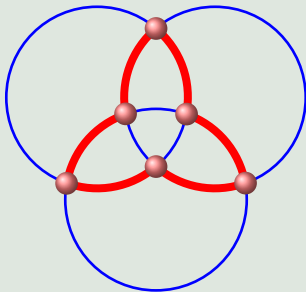
- C_n 表示长度为 n 的基本回路.
- 长度为3的基本回路称为三角形.
- 长度最小基本回路的长度称为围长(girth), 记为 $g(D)$.



回路和基本回路

Definition

- C_n 表示长度为 n 的基本回路.
- 长度为3的基本回路称为三角形.
- 长度最小基本回路的长度称为围长(girth), 记为 $g(D)$.



Example

设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.

Example

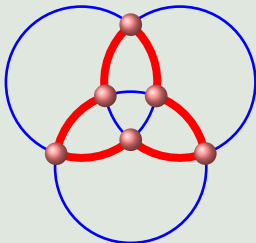
设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.

Example

设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.

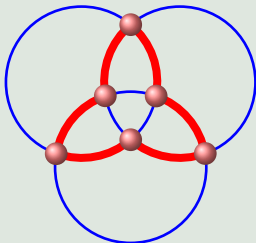
Example

设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.



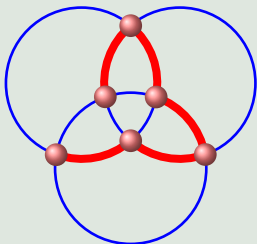
Example

设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.



Example

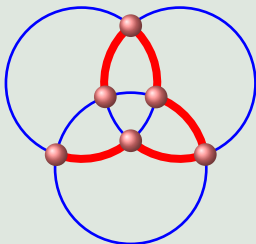
设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.



Proof(extremality)

Example

设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.

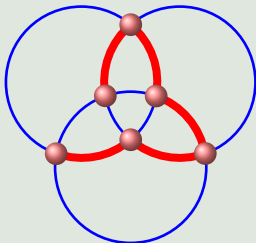


Proof(extremality)

- 设 $P = (v_0 v_1 \cdots v_k)$ 是 G 中长度最长的基本路径。
- 则对任意的与 v_0 相邻的点 v 都在路径 P 上, 否则 $(v v_0 \cdots v_k)$ 是一个长度更长的基本路径。
- $|P| \geq \deg(v) \geq \delta$.
- 设 v_l 是 P 中最后一个与 v_0 相邻的点。
 则 $C = (v_0 v_1 \cdots v_l v_0)$ 是一个长度不小于 $\delta + 1$ 的基本回路。

Example

设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.

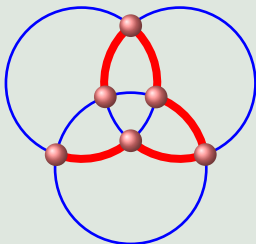


Proof(extremality)

- 设 $P = (v_0 v_1 \cdots v_k)$ 是 G 中长度最长的基本路径。
- 则对任意的与 v_0 相邻的点 v 都在路径 P 上, 否则 $(v v_0 \cdots v_k)$ 是一个长度更长的基本路径。
- $|P| \geq \deg(v) \geq \delta$.
- 设 v_l 是 P 中最后一个与 v_0 相邻的点。
 则 $C = (v_0 v_1 \cdots v_l v_0)$ 是一个长度不小于 $\delta + 1$ 的基本回路。

Example

设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.

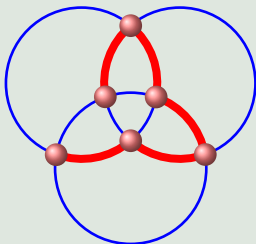


Proof(extremality)

- 设 $P = (v_0 v_1 \cdots v_k)$ 是 G 中长度最长的基本路径.
- 则对任意的与 v_0 相邻的点 v 都在路径 P 上, 否则 $(vv_0 \cdots v_k)$ 是一个长度更长的基本路径.
- $|P| \geq \deg(v) \geq \delta$.
- 设 v_l 是 P 中最后一个与 v_0 相邻的点.
 则 $C = (v_0 v_1 \cdots v_l v_0)$ 是一个长度不小于 $\delta + 1$ 的基本回路.

Example

设 $G(V, E)$ 是简单无向图, 若 $\delta \geq 2$. 则 G 含有一个长度至少为 $\delta + 1$ 的基本回路.



Proof(extremality)

- 设 $P = (v_0 v_1 \cdots v_k)$ 是 G 中长度最长的基本路径。
- 则对任意的与 v_0 相邻的点 v 都在路径 P 上, 否则 $(v v_0 \cdots v_k)$ 是一个长度更长的基本路径。
- $|P| \geq \deg(v) \geq \delta$.
- 设 v_l 是 P 中最后一个与 v_0 相邻的点。
 则 $C = (v_0 v_1 \cdots v_l v_0)$ 是一个长度不小于 $\delta + 1$ 的基本回路。

距离与直径

Definition

Definition

- 设 $D(G)$ 是有向图 (无向图), $u, v \in V$, $D(G)$ 中所有 (u, v) 基本路径中的最短长度称为从 u 到 v 的 **距离**, 记为 $d(u, v)$. 如果 u 到 v 不可达, 则约定 $d(u, v) = \infty$
- 直径: $d(D) = \max\{d(u, v) \mid u, v \in V\}$.
- $d(D) = 1 \iff K_n^* \subseteq D, d(G) = 1 \iff K_n \subseteq D$.

距离与直径

Definition

- 设 $D(G)$ 是有向图 (无向图), $u, v \in V$, $D(G)$ 中所有 (u, v) 基本路径中的最短长度称为从 u 到 v 的 **距离**, 记为 $d(u, v)$. 如果 u 到 v 不可达, 则约定 $d(u, v) = \infty$
- 直径: $d(D) = \max\{d(u, v) \mid u, v \in V\}$.
- $d(D) = 1 \iff K_n^* \subseteq D, d(G) = 1 \iff K_n \subseteq D$.

距离与直径

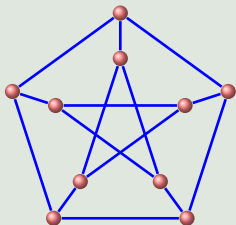
Definition

- 设 $D(G)$ 是有向图 (无向图), $u, v \in V$, $D(G)$ 中所有 (u, v) 基本路径中的最短长度称为从 u 到 v 的 **距离**, 记为 $d(u, v)$. 如果 u 到 v 不可达, 则约定 $d(u, v) = \infty$
- 直径: $d(D) = \max\{d(u, v) \mid u, v \in V\}$.
- $d(D) = 1 \iff K_n^* \subseteq D, d(G) = 1 \iff K_n \subseteq D$.

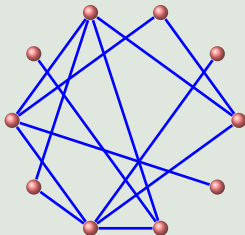
距离与直径

Definition

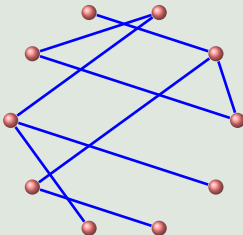
- 设 $D(G)$ 是有向图 (无向图), $u, v \in V$, $D(G)$ 中所有 (u, v) 基本路径中的最短长度称为从 u 到 v 的 **距离**, 记为 $d(u, v)$. 如果 u 到 v 不可达, 则约定 $d(u, v) = \infty$
- 直径: $d(D) = \max\{d(u, v) \mid u, v \in V\}$.
- $d(D) = 1 \iff K_n^* \subseteq D$, $d(G) = 1 \iff K_n \subseteq D$.



(a) 直径为2



(b) 直径为4



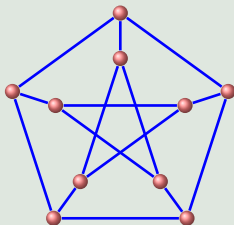
(c) 直径为7

距离与直径

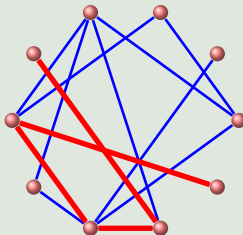
Definition

- 设 $D(G)$ 是有向图(无向图), $u, v \in V$, $D(G)$ 中所有 (u, v) 基本路径中的最短长度称为从 u 到 v 的 **距离**, 记为 $d(u, v)$. 如果 u 到 v 不可达, 则约定 $d(u, v) = \infty$
- 直径: $d(D) = \max\{d(u, v) \mid u, v \in V\}$.
- $d(D) = 1 \iff K_n^* \subseteq D$, $d(G) = 1 \iff K_n \subseteq D$.

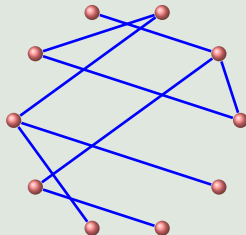
Example



(a) 直径为2



(b) 直径为4

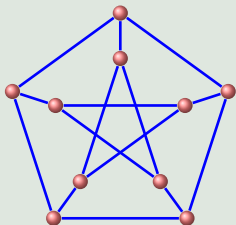


(c) 直径为7

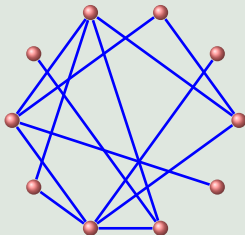
距离与直径

Definition

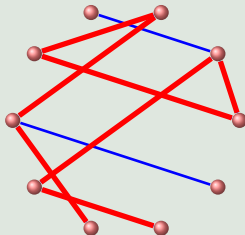
- 设 $D(G)$ 是有向图 (无向图), $u, v \in V$, $D(G)$ 中所有 (u, v) 基本路径中的最短长度称为从 u 到 v 的 **距离**, 记为 $d(u, v)$. 如果 u 到 v 不可达, 则约定 $d(u, v) = \infty$
- 直径: $d(D) = \max\{d(u, v) \mid u, v \in V\}$.
- $d(D) = 1 \iff K_n^* \subseteq D, d(G) = 1 \iff K_n \subseteq D$.



(a) 直径为2



(b) 直径为4



(c) 直径为7

Example

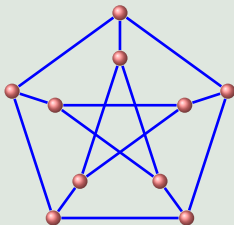
围长为5的 k 正则无向图至少有 $k^2 + 1$ 阶.

Example

围长为5的 k 正则无向图至少有 $k^2 + 1$ 阶.

Example

围长为5的 k 正则无向图至少有 $k^2 + 1$ 阶.



围长为5的 k 正则无向图至少有 $k^2 + 1$ 阶.

围长为5的 k 正则无向图至少有 $k^2 + 1$ 阶.

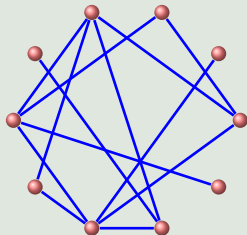
- 设 $v \in V$, 定义 $S_{v,i}$ 为与 v 距离为 i 的结点集合 ($i = 0, 1, 2$).
- 则 $|S_{v,1}| = k$, 且 $S_{v,1}$ 中的点互不相邻. 否则有三角形, 与围长 5 矛盾.
- 设 $v', v'' \in S_{v,1}$. 则 $(S_{v',1} - \{v\}) \cap (S_{v'',1} - \{v\}) = \emptyset$. 否则有长度为 4 的基本回路.
- 而 $S_{v',1} - \{v\} \in S_{v,2}$ 且 $|S_{v',1} - \{v\}| = k - 1$.
- 因此 $|S_{v,0} \cup S_{v,1} \cup S_{v,2}| = 1 + k + k(k - 1) = k^2 + 1$.

Example

设 G 是简单图. 如果 $d(G) > 3$, 则 $d(\overline{G}) < 3$.

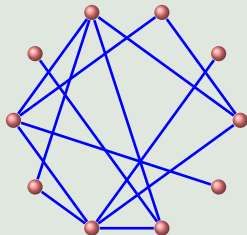
Example

设 G 是简单图. 如果 $d(G) > 3$, 则 $d(\overline{G}) < 3$.



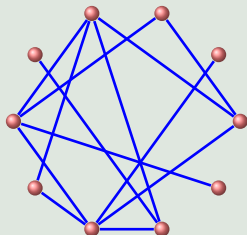
Example

设 G 是简单图. 如果 $d(G) > 3$, 则 $d(\overline{G}) < 3$.



Example

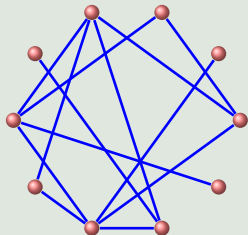
设 G 是简单图. 如果 $d(G) > 3$, 则 $d(\overline{G}) < 3$.



Proof

Example

设 G 是简单图. 如果 $d(G) > 3$, 则 $d(\overline{G}) < 3$.

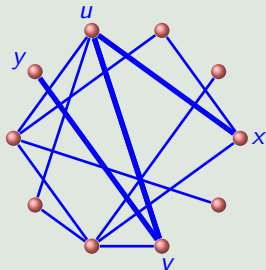


Proof

- 设 $u, v \in V$, 若 $uv \notin E(G)$, 则 $uv \in E(\overline{G})$. $d_{\overline{G}}(u, v) = 1$.
- 若 $uv \in E(G)$, 若在 G 中任意的一个结点至少与 u 和 v 其中之一相邻, 则 $\forall x, y \in V, d_G(x, y) \leq 3$, 矛盾.
- 设 $w \in V, uw, vw \notin E(G)$, 则 $uw, vw \in E(\overline{G})$, 即 $d_{\overline{G}}(u, v) \leq 2$.

Example

设 G 是简单图. 如果 $d(G) > 3$, 则 $d(\overline{G}) < 3$.

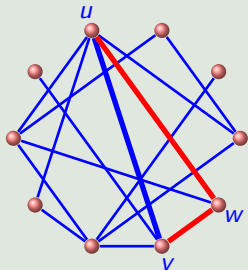


Proof

- 设 $u, v \in V$, 若 $uv \notin E(G)$, 则 $uv \in E(\overline{G})$. $d_{\overline{G}}(u, v) = 1$.
- 若 $uv \in E(G)$, 若在 G 中任意的一个结点至少与 u 和 v 其中之一相邻, 则 $\forall x, y \in V, d_G(x, y) \leq 3$, 矛盾.
- 设 $w \in V, uw, vw \notin E(G)$, 则 $uw, vw \in E(\overline{G})$, 即 $d_{\overline{G}}(u, v) \leq 2$.

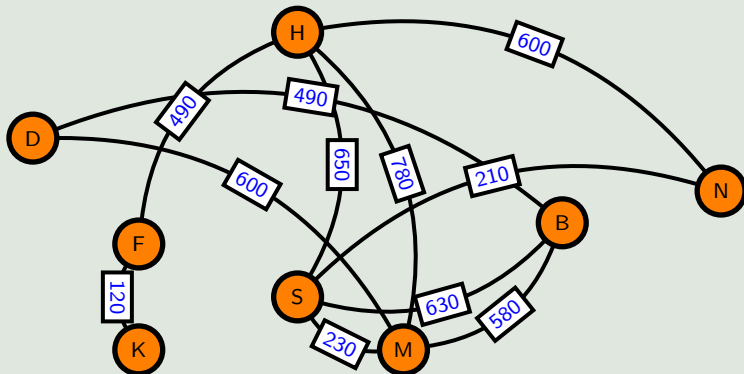
Example

设 G 是简单图. 如果 $d(G) > 3$, 则 $d(\overline{G}) < 3$.

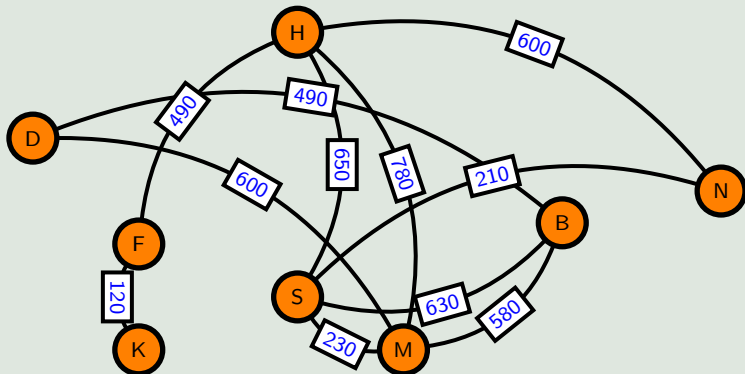


Proof

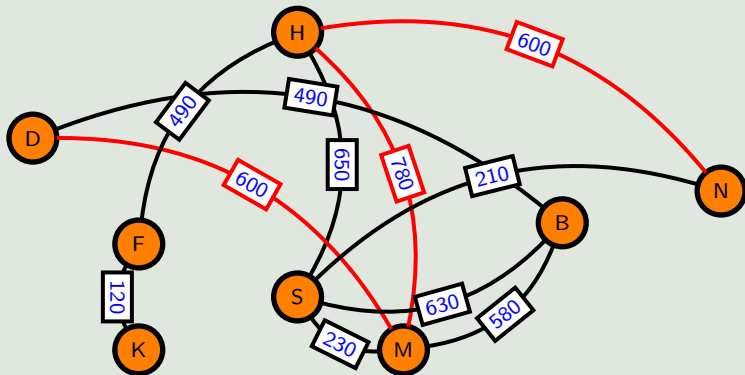
- 设 $u, v \in V$, 若 $uv \notin E(G)$, 则 $uv \in E(\overline{G})$. $d_{\overline{G}}(u, v) = 1$.
- 若 $uv \in E(G)$, 若在 G 中任意的一个结点至少与 u 和 v 其中之一相邻, 则 $\forall x, y \in V, d_G(x, y) \leq 3$, 矛盾.
- 设 $w \in V, uw, vw \notin E(G)$, 则 $uw, vw \in E(\overline{G})$, 即 $d_{\overline{G}}(u, v) \leq 2$.



Example

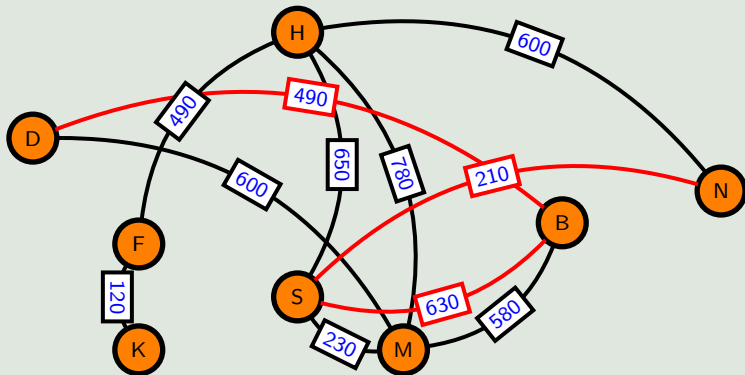


Example



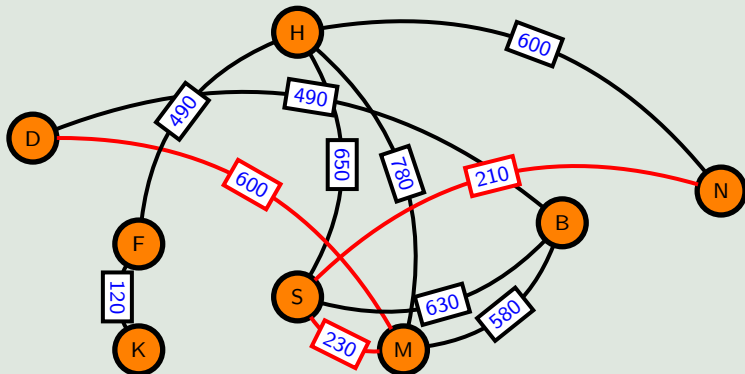
- ① $P_1 = (\text{DMHN})$, 航程 $600 + 780 + 600 = 1980$;
- ② $P_2 = (\text{DBSN})$, 航程 $490 + 630 + 210 = 1330$;
- ③ $P_3 = (\text{DMSN})$, 航程 $600 + 230 + 210 = 1040$.

Example



- ① $P_1 = (\text{DMHN})$, 航程 $600 + 780 + 600 = 1980$;
- ② $P_2 = (\text{DBSN})$, 航程 $490 + 630 + 210 = 1330$;
- ③ $P_3 = (\text{DMSN})$, 航程 $600 + 230 + 210 = 1040$.

Example



- ① $P_1 = (\text{DMHN})$, 航程 $600 + 780 + 600 = 1980$;
- ② $P_2 = (\text{DBSN})$, 航程 $490 + 630 + 210 = 1330$;
- ③ $P_3 = (\text{DMSN})$, 航程 $600 + 230 + 210 = 1040$.

Definition

$$W(P) \triangleq \sum_{i=1}^k \psi(e_i)$$
$$d(u, v) \triangleq \begin{cases} \min\{W(P) \mid P \text{ 是 } u \text{ 到 } v \text{ 的路径}\}; \\ \infty, \text{ if } u \text{ 到 } v \text{ 是不可达的.} \end{cases}$$

赋权图与最短路径

Definition

赋权图 $G = \langle V, E, \varphi, \psi \rangle$, 其中 $\psi: E \rightarrow \mathbb{R}^+$, 设 $P = (e_1, e_2, \dots, e_k)$; 则路径 P 的权值 $W(P)$ 定义为:

$$W(P) \triangleq \sum_{i=1}^k \psi(e_i)$$

u 到 v 的最短路径 $d(u, v)$ 定义为:

$$d(u, v) \triangleq \begin{cases} \min\{W(P) \mid P \text{ 是 } u \text{ 到 } v \text{ 的路径}\}; \\ \infty, \text{ if } u \text{ 到 } v \text{ 是不可达的.} \end{cases}$$

Remark

ソ为.

$$(\min\{W(P) \mid P \text{ 是 } u \text{ 到 } v \text{ 的路径}\},$$

-

赋权图与最短路径

Definition

赋权图 $G = \langle V, E, \varphi, \psi \rangle$, 其中 $\psi: E \rightarrow \mathbb{R}^+$, 设 $P = (e_1, e_2, \dots, e_k)$; 则路径 P 的权值 $W(P)$ 定义为:

$$W(P) \triangleq \sum_{i=1}^k \psi(e_i)$$

u 到 v 的最短路径 $d(u, v)$ 定义为:

$$d(u, v) \triangleq \begin{cases} \min\{W(P) \mid P \text{ 是 } u \text{ 到 } v \text{ 的路径}\}; \\ \infty, \text{ if } u \text{ 到 } v \text{ 是不可达的.} \end{cases}$$

Remark

- 要求每个边的权值非负;
- Dijkstra's algorithm — 求解某个结点到其他结点的最短距离;
- Floyd-Warshall algorithm — 求解每对结点的最短距离.

赋权图与最短路径

Definition

赋权图 $G = \langle V, E, \varphi, \psi \rangle$, 其中 $\psi: E \rightarrow \mathbb{R}^+$, 设 $P = (e_1, e_2, \dots, e_k)$; 则路径 P 的权值 $W(P)$ 定义为:

$$W(P) \triangleq \sum_{i=1}^k \psi(e_i)$$

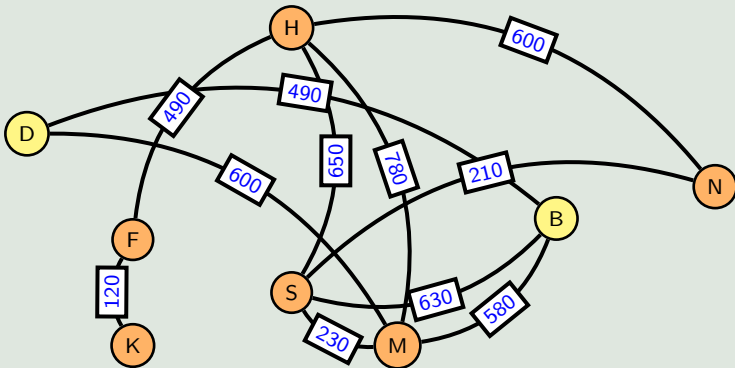
u 到 v 的最短路径 $d(u, v)$ 定义为:

$$d(u, v) \triangleq \begin{cases} \min\{W(P) \mid P \text{ 是 } u \text{ 到 } v \text{ 的路径}\}; \\ \infty, \text{ if } u \text{ 到 } v \text{ 是不可达的.} \end{cases}$$

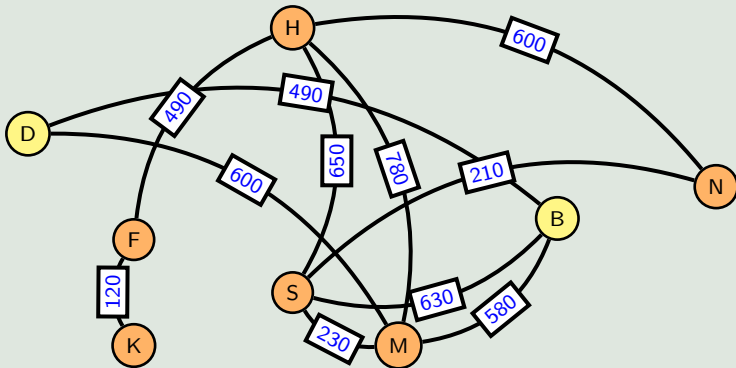
Remark

- 要求每个边的权值非负；
- Dijkstra's algorithm — 求解某个结点到其他结点的最短距离；
- Floyd-Warshall algorithm — 求解每对结点的最短距离。

Example

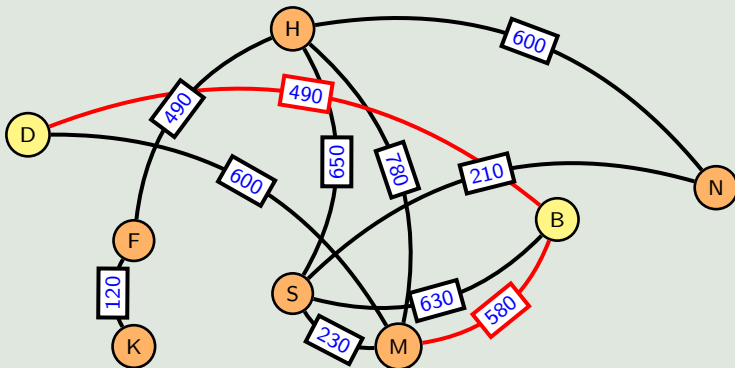


Example



Example

$S = \{D, B\}$

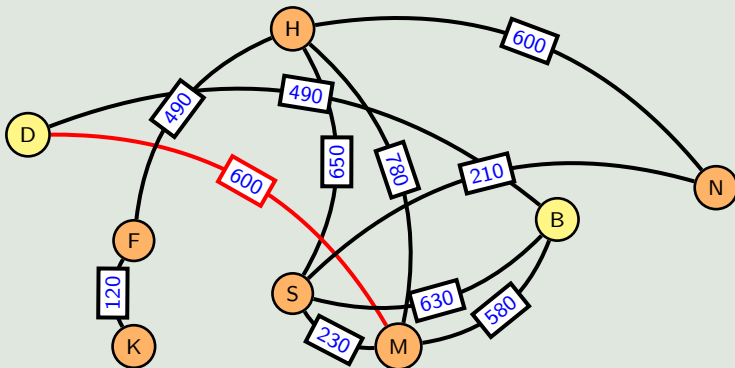


① $D_S(M) = \min\{D(DBM), D(DM)\} = 600;$

② $D_S(S) = D(DBS) = 490 + 630 = 1120;$

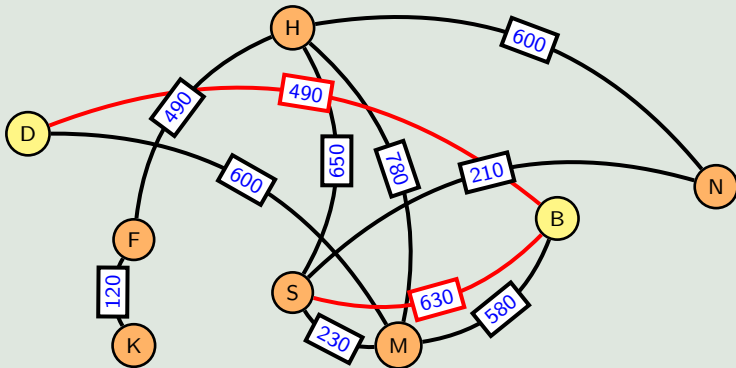
③ $D_S(H) = D_S(F) = D_S(K) = D_S(N) = \infty.$

Example



$$\textcircled{1} \quad D_S(M) = \min\{D(\text{DBM}), D(\text{DM})\} = 600;$$

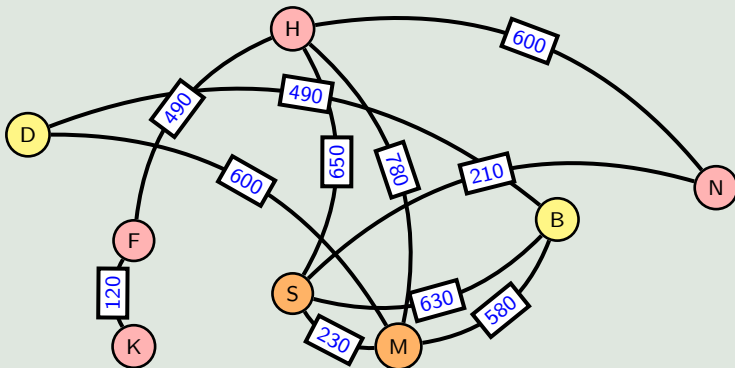
Example



- ① $D_S(M) = \min\{D(DBM), D(DM)\} = 600$;
- ② $D_S(S) = D(DBS) = 490 + 630 = 1120$;
- ③ $D_S(H) = D_S(F) = D_S(K) = D_S(N) = \infty$.

Example

$S = \{D, B\}$



- ① $D_S(M) = \min\{D(DBM), D(DM)\} = 600;$
- ② $D_S(S) = D(DBS) = 490 + 630 = 1120;$
- ③ $D_S(H) = D_S(F) = D_S(K) = D_S(N) = \infty.$

相关引理

Proposition

设 $D^T(t_0) \triangleq \min\{D^T(t) \mid t \in T\}$, 则:

$$d(a, t_0) = D^T(t_0)$$

if 结点 a 到 结点 $v \in V - T$ 的最短路径不经过 T 中的结点.

相关引理

Proposition

设 $D^T(t_0) \triangleq \min\{D^T(t) \mid t \in T\}$, 则:

$$d(a, t_0) = D^T(t_0)$$

if 结点 a 到结点 $v \in V - T$ 的最短路径不经过 T 中的结点.

Proof(反证法).

- 设 a 到 t_0 最短路径为 $P = (a, s_1, \dots, s_p, t, \dots, t_0)$, 其中 t 为第一次出现的 T 中的结点, 即 P 还要经过 T 中的其他的结点;
- 设 $P_1 = (a, s_1, \dots, s_p, t)$, $P_2 = (t, \dots, t_0)$, 则 $P = P_1 \cup P_2$; 并且 P_1 是 a 到 t 不经过 T 中除 t 以外其他结点的路径;
 $\therefore W(P_1) \geq D^T(t) \geq D^T(t_0)$;
- $\therefore W(P) = W(P_1) + W(P_2) > D^T(t_0)$, 与 $W(P)$ 是最短路径矛盾.



相关引理

Proposition

设 $D^T(t_0) \triangleq \min\{D^T(t) \mid t \in T\}$, 则:

$$d(a, t_0) = D^T(t_0)$$

if 结点 a 到结点 $v \in V - T$ 的最短路径不经过 T 中的结点.

Proof(反证法).

- 设 a 到 t_0 最短路径为 $P = (a, s_1, \dots, s_p, t, \dots, t_0)$, 其中 t 为第一次出现的 T 中的结点, 即 P 还要经过 T 中的其他的结点;
- 设 $P_1 = (a, s_1, \dots, s_p, t)$, $P_2 = (t, \dots, t_0)$, 则 $P = P_1 \cup P_2$; 并且 P_1 是 a 到 t 不经过 T 中除 t 以外其他结点的路径;
 $\therefore W(P_1) \geq D^T(t) \geq D^T(t_0)$;
- $\therefore W(P) = W(P_1) + W(P_2) > D^T(t_0)$, 与 $W(P)$ 是最短路径矛盾.



相关引理

Proposition

设 $D^T(t_0) \triangleq \min\{D^T(t) \mid t \in T\}$, 则:

$$d(a, t_0) = D^T(t_0)$$

if 结点 a 到结点 $v \in V - T$ 的最短路径不经过 T 中的结点.

Proof(反证法).

- 设 a 到 t_0 最短路径为 $P = (a, s_1, \dots, s_p, t, \dots, t_0)$, 其中 t 为第一次出现的 T 中的结点, 即 P 还要经过 T 中的其他的结点;
- 设 $P_1 = (a, s_1, \dots, s_p, t)$, $P_2 = (t, \dots, t_0)$, 则 $P = P_1 \cup P_2$; 并且 P_1 是 a 到 t 不经过 T 中除 t 以外其他结点的路径;
 $\therefore W(P_1) \geq D^T(t) \geq D^T(t_0)$;
- $\therefore W(P) = W(P_1) + W(P_2) > D^T(t_0)$, 与 $W(P)$ 是最短路径矛盾.



相关引理

Proposition

设 $D^T(t_0) \triangleq \min\{D^T(t) \mid t \in T\}$, 则:

$$d(a, t_0) = D^T(t_0)$$

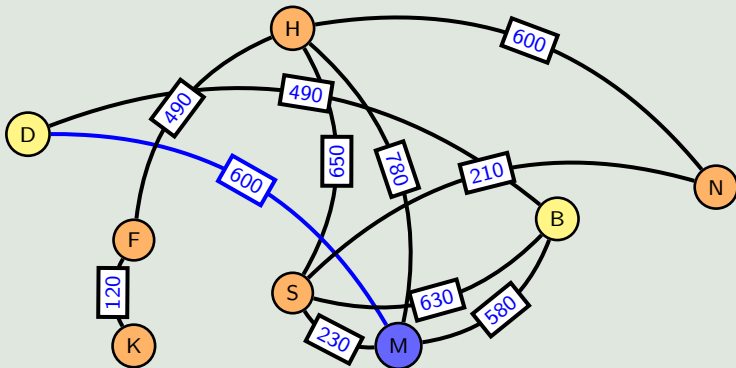
if 结点 a 到结点 $v \in V - T$ 的最短路径不经过 T 中的结点.

Proof(反证法).

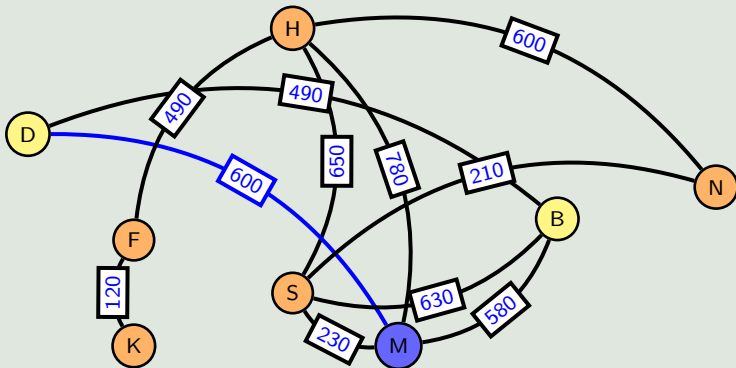
- 设 a 到 t_0 最短路径为 $P = (a, s_1, \dots, s_p, t, \dots, t_0)$, 其中 t 为第一次出现的 T 中的结点, 即 P 还要经过 T 中的其他的结点;
- 设 $P_1 = (a, s_1, \dots, s_p, t)$, $P_2 = (t, \dots, t_0)$, 则 $P = P_1 \cup P_2$; 并且 P_1 是 a 到 t 不经过 T 中除 t 以外其他结点的路径;
 $\therefore W(P_1) \geq D^T(t) \geq D^T(t_0)$;
- $\therefore W(P) = W(P_1) + W(P_2) > D^T(t_0)$, 与 $W(P)$ 是最短路径矛盾.



Example: $D(M)$ 一定是D到M的最短路径

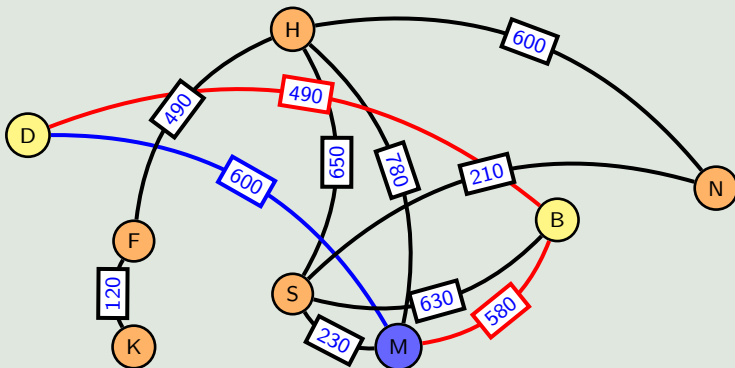


Example: $D(M)$ 一定是D到M的最短路径



Example: $D(M)$ 一定是D到M的最短路径

$S = \{D, B\}$

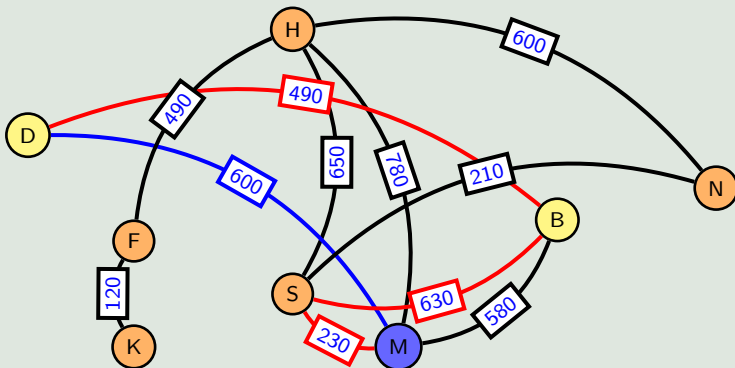


① $D(DBM) > D_S(M) = \min\{D(DBM), D(DM)\} = 600;$

② $D(DBSM) = D_S(S) + 230 > D_S(M).$

Example: $D(M)$ 一定是D到M的最短路径

$S = \{D, B\}$



- ① $D(DBM) > D_S(M) = \min\{D(DBM), D(DM)\} = 600$;
- ② $D(DBSM) = D_S(S) + 230 > D_S(M)$.

Dijkstra's Algorithm

Dijkstra's Algorithm

- ① 初始化: $S := \{a\}$, $T = V - \{a\}$, $D^T(t) = \psi(a, t)$, $d(a, t) = \infty$;
- ② 求 $D^T(t_0) = \min\{D^T(t) \mid t \in T\}$;
 $S := S \cup \{t_0\}$, $T := T - \{t_0\}$, $d(a, t_0) := D^T(t_0)$;
- ③ if ($T == \emptyset$) then stop; else goto ②.

Dijkstra's Algorithm

Dijkstra's Algorithm

- ① 初始化: $S := \{a\}$, $T = V - \{a\}$, $D^T(t) = \psi(a, t)$, $d(a, t) = \infty$;
- ② 求 $D^T(t_0) = \min\{D^T(t) \mid t \in T\}$;
 $S := S \cup \{t_0\}$, $T := T - \{t_0\}$, $d(a, t_0) := D^T(t_0)$;
- ③ if ($T == \emptyset$) then stop; else goto ②.

Dijkstra's Algorithm

Dijkstra's Algorithm

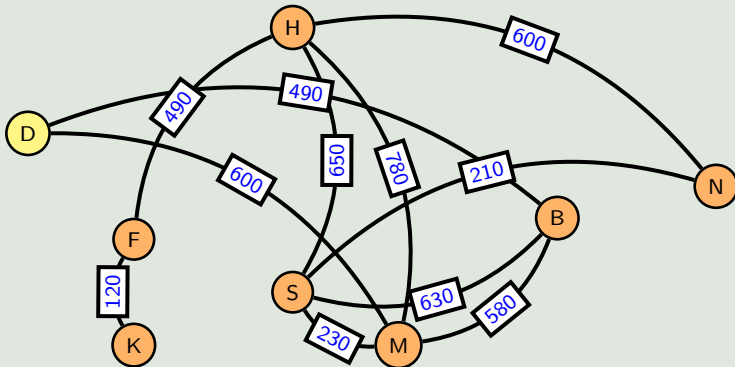
- ① 初始化: $S := \{a\}$, $T = V - \{a\}$, $D^T(t) = \psi(a, t)$, $d(a, t) = \infty$;
- ② 求 $D^T(t_0) = \min\{D^T(t) \mid t \in T\}$;
 $S := S \cup \{t_0\}$, $T := T - \{t_0\}$, $d(a, t_0) := D^T(t_0)$;
- ③ if ($T == \emptyset$) then stop; else goto ②.

Dijkstra's Algorithm

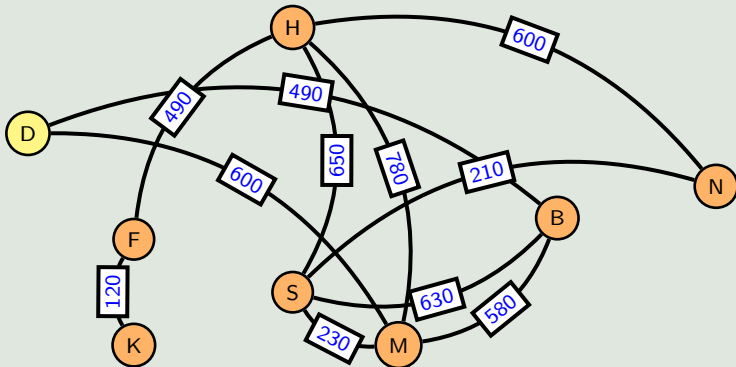
Dijkstra's Algorithm

- ① 初始化: $S := \{a\}$, $T = V - \{a\}$, $D^T(t) = \psi(a, t)$, $d(a, t) = \infty$;
- ② 求 $D^T(t_0) = \min\{D^T(t) \mid t \in T\}$;
 $S := S \cup \{t_0\}$, $T := T - \{t_0\}$, $d(a, t_0) := D^T(t_0)$;
- ③ if ($T == \emptyset$) then stop; else goto ②.

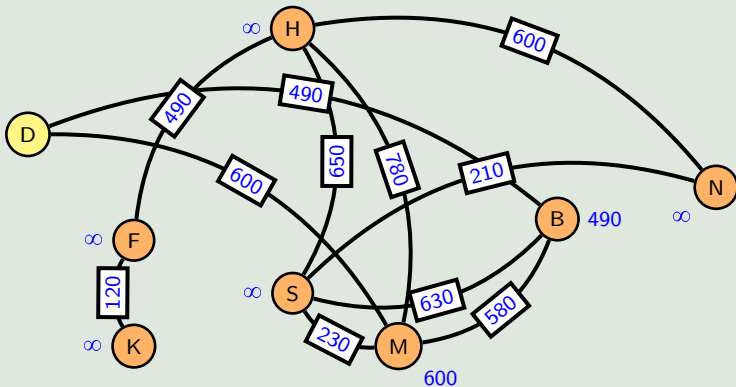
Example



Example

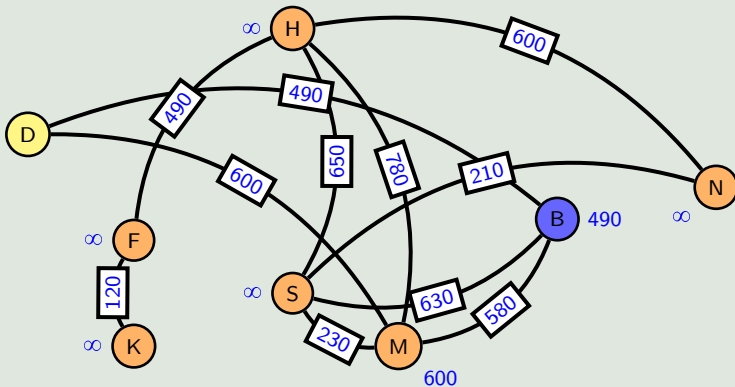


Example



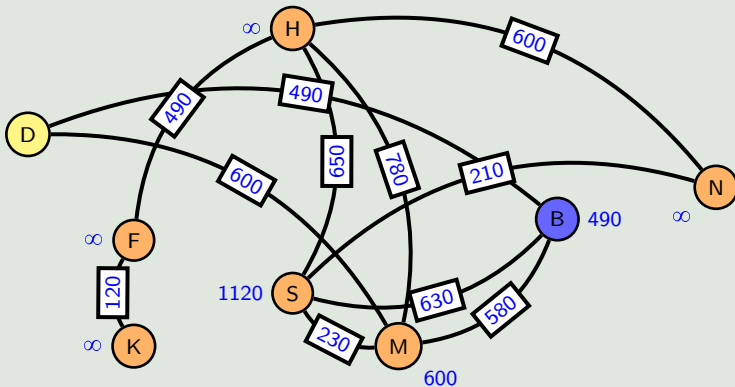
- 1 $S = \{ D \}, D_S(B) = \min\{ D_S(v) \};$
- 2 $S = \{ D, B \}, D_S(M) = \min\{ D_S(v) \};$
- 3 $S = \{ D, B, M \}, D_S(S) = \min\{ D_S(v) \};$

Example



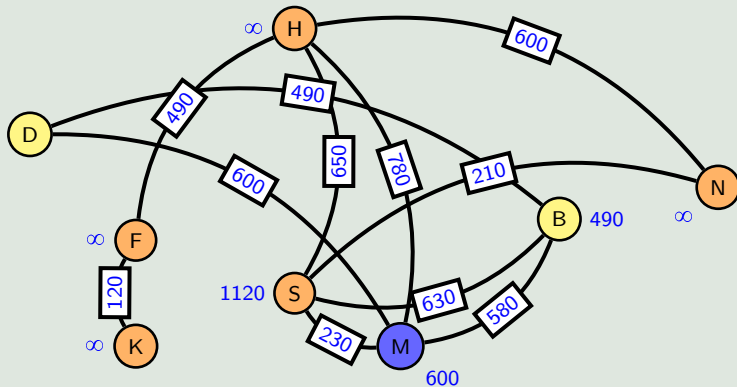
- ① $S = \{ D \}, D_S(B) = \min\{ D_S(v) \};$
- ② $S = \{ D, B \}, D_S(M) = \min\{ D_S(v) \};$
- ③ $S = \{ D, B, M \}, D_S(S) = \min\{ D_S(v) \};$

Example



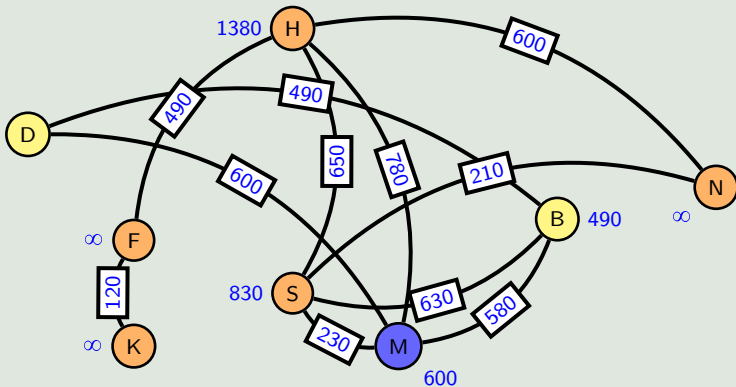
- ① $S = \{ D \}, D_S(B) = \min\{ D_S(v) \};$
- ② $S = \{ D, B \}, D_S(M) = \min\{ D_S(v) \};$
- ③ $S = \{ D, B, M \}, D_S(S) = \min\{ D_S(v) \};$

Example



- ① $S = \{ D \}, D_S(B) = \min\{ D_S(v) \};$
- ② $S = \{ D, B \}, D_S(M) = \min\{ D_S(v) \};$
- ③ $S = \{ D, B, M \}, D_S(S) = \min\{ D_S(v) \};$

Example



- ① $S = \{ D \}, D_S(B) = \min\{ D_S(v) \};$
- ② $S = \{ D, B \}, D_S(M) = \min\{ D_S(v) \};$
- ③ $S = \{ D, B, M \}, D_S(S) = \min\{ D_S(v) \};$

Proposition

$$D^{T'}(t) = \min\{ D^T(t), D(t_0) + W(t_0, t) \}$$

注意: 从 a 到 S 中结点的最短路径经过的结点一定在 S 中.

799

Proposition

$$D^{T'}(t) = \min\{ D^T(t), D(t_0) + W(t_0, t) \}$$

注意: 从a到S中结点的最短路径经过的结点一定在S中.

Proof.

设 P 是 a 到 t 不经过 T' 中除 t 以外其他结点的最短路径, 则:



相对距离的计算

Proposition

设 $D^T(t_0) \triangleq \min\{D^T(t) \mid t \in T\}$, 设 $S' = S \cup \{t_0\}$, $T' = T - \{t_0\}$, 则:

$$D^{T'}(t) = \min\{D^T(t), D(t_0) + W(t_0, t)\}$$

注意: 从 a 到 S 中结点的最短路径经过的结点一定在 S 中.

Proof.

设 P 是 a 到 t 不经过 T' 中除 t 以外其他结点的最短路径, 则:

- P 中不含有 t_0 , 则 P 也是 a 到 t 不经过 T 中除 t 以外其他结点的最短路径; $\therefore D^{T'}(t) = D^T(t)$;

- P 中含有 t_0 :

设 $P = (a, s_1, s_2, \dots, s_k, t_0, s'_1, s'_2, \dots, s'_j, t)$, 注意到 a 到 s'_j 的最短路径经过的结点仅在 S 中, $\therefore W(P) > d(a, s'_j) + W(s'_j, t)$, 这

与 P 是 a 到 t 仅经过 S' 中的结点的最短路径矛盾;

$\therefore P = (a, s_1, s_2, \dots, s_k, t_0, t)$, 此时 $W(P) = D(t_0) + W(t_0, t)$.



相对距离的计算

Proposition

设 $D^T(t_0) \triangleq \min\{D^T(t) \mid t \in T\}$, 设 $S' = S \cup \{t_0\}$, $T' = T - \{t_0\}$, 则:

$$D^{T'}(t) = \min\{D^T(t), D(t_0) + W(t_0, t)\}$$

注意: 从 a 到 S 中结点的最短路径经过的结点一定在 S 中.

Proof.

设 P 是 a 到 t 不经过 T' 中除 t 以外其他结点的最短路径, 则:

- P 中不含有 t_0 , 则 P 也是 a 到 t 不经过 T 中除 t 以外其他结点的最短路径; $\therefore D^{T'}(t) = D^T(t)$;

- P 中含有 t_0 :

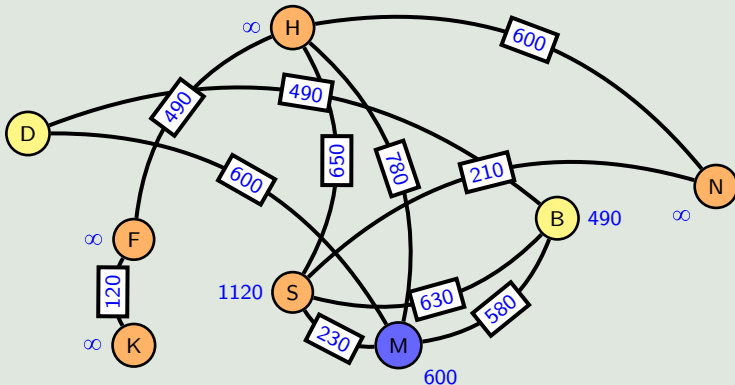
设 $P = (a, s_1, s_2, \dots, s_k, t_0, s'_1, s'_2, \dots, s'_j, t)$, 注意到 a 到 s'_j 的最短路径经过的结点仅在 S 中, $\therefore W(P) > d(a, s'_j) + W(s'_j, t)$, 这

与 P 是 a 到 t 仅经过 S' 中的结点的最短路径矛盾;

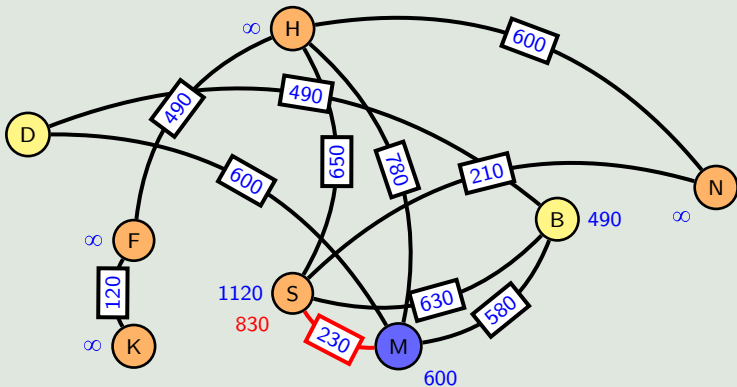
$\therefore P = (a, s_1, s_2, \dots, s_k, t_0, t)$, 此时 $W(P) = D(t_0) + W(t_0, t)$.



Example: 已知 $D_{\{D,B\}}$, 求 $D_{\{D,B,M\}}$

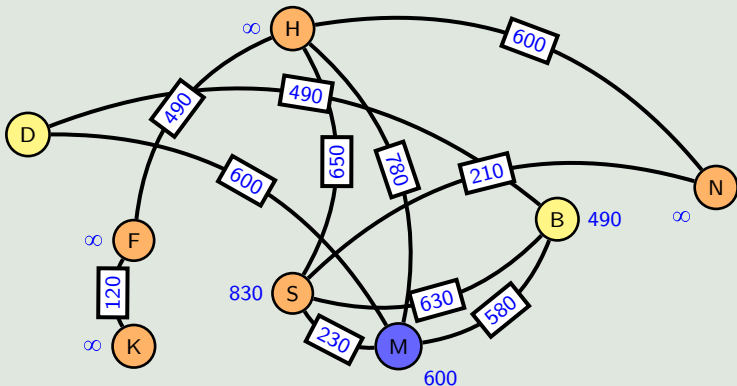


Example: 已知 $D_{\{D,B\}}$, 求 $D_{\{D,B,M\}}$



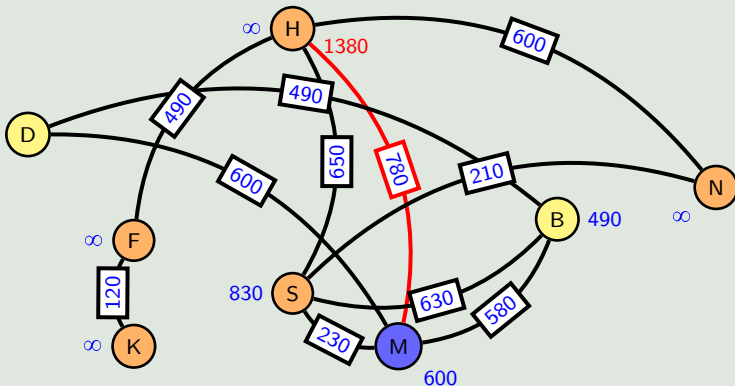
- ① $D_{\{D,B,M\}}(S) = \min\{ D_{\{D,B\}}(S), D_{\{D,B\}}(M) + d(M,D) \};$
- ② $D_{\{D,B,M\}}(H) = \min\{ D_{\{D,B\}}(H), D_{\{D,B\}}(M) + d(M,H) \};$
- ③ 其他结点与M不相邻, 相对距离保持不变.

Example: 已知 $D_{\{D,B\}}$, 求 $D_{\{D,B,M\}}$



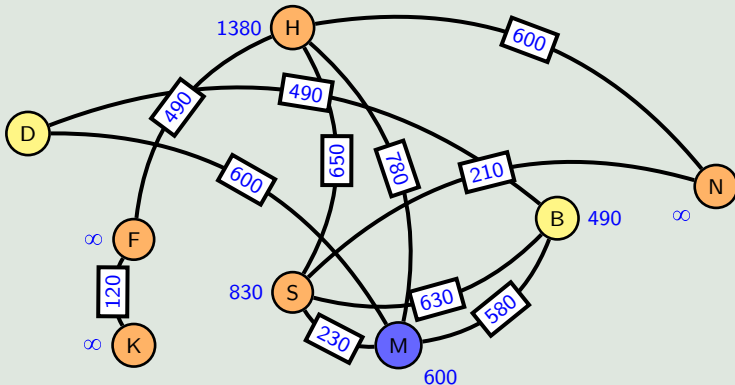
- 1 $D_{\{D,B,M\}}(S) = \min\{D_{\{D,B\}}(S), D_{\{D,B\}}(M) + d(M,D)\};$
- 2 $D_{\{D,B,M\}}(H) = \min\{D_{\{D,B\}}(H), D_{\{D,B\}}(M) + d(M,H)\};$
- 3 其他结点与M不相邻, 相对距离保持不变

Example: 已知 $D_{\{D,B\}}$, 求 $D_{\{D,B,M\}}$



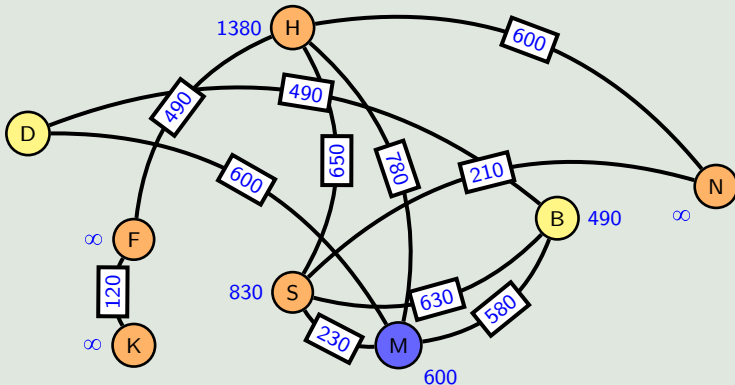
- ① $D_{\{D,B,M\}}(S) = \min\{ D_{\{D,B\}}(S), D_{\{D,B\}}(M) + d(M,D) \};$
- ② $D_{\{D,B,M\}}(H) = \min\{ D_{\{D,B\}}(H), D_{\{D,B\}}(M) + d(M,H) \};$
- ③ 其他结点与M不相邻, 相对距离保持不变.

Example: 已知 $D_{\{D,B\}}$, 求 $D_{\{D,B,M\}}$



- ① $D_{\{D,B,M\}}(S) = \min\{ D_{\{D,B\}}(S), D_{\{D,B\}}(M) + d(M,S) \};$
- ② $D_{\{D,B,M\}}(H) = \min\{ D_{\{D,B\}}(H), D_{\{D,B\}}(M) + d(M,H) \};$
- ③ 其他结点与M不相邻, 相对距离保持不变.

Example: 已知 $D_{\{D,B\}}$, 求 $D_{\{D,B,M\}}$

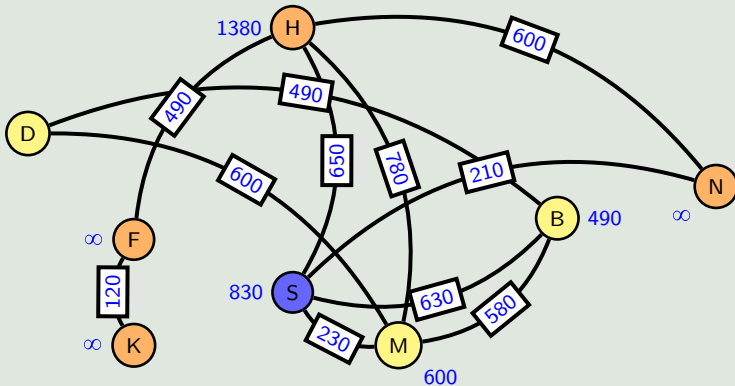


- ① $D_{\{D,B,M\}}(S) = \min\{ D_{\{D,B\}}(S), D_{\{D,B\}}(M) + d(M,D) \};$
- ② $D_{\{D,B,M\}}(H) = \min\{ D_{\{D,B\}}(H), D_{\{D,B\}}(M) + d(M,H) \};$
- ③ 其他结点与M不相邻, 相对距离保持不变.

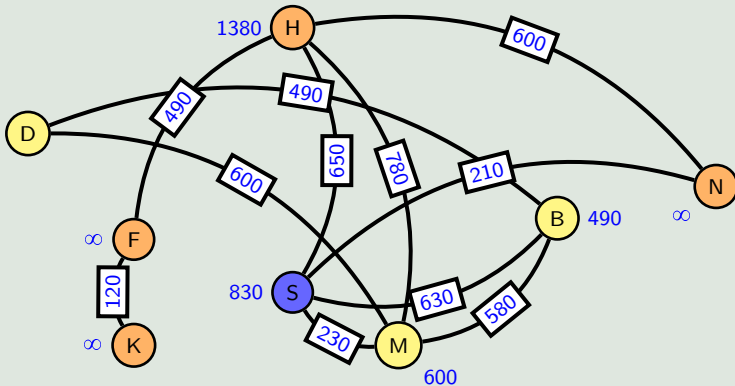
Dijkstra's Algorithm

```
1  function Dijkstra(G, W, a)
2    for each vertex v in V[G] // Initialization
3      do d[v] := w(a,v)
4    t0 := undefined
5    d[s] := 0
6    T := set of all vertices
7    while T is not an empty set
8      do t0 := Extract-Min(T)
9         T := T - { t0 }
10       for each edge (t0, t) where t in T
11         do if d[t] > d[t0] + w(t0,t) //Relax (u,v)
12           then d[t] := d[t0] + w(t0,t)
```


Example

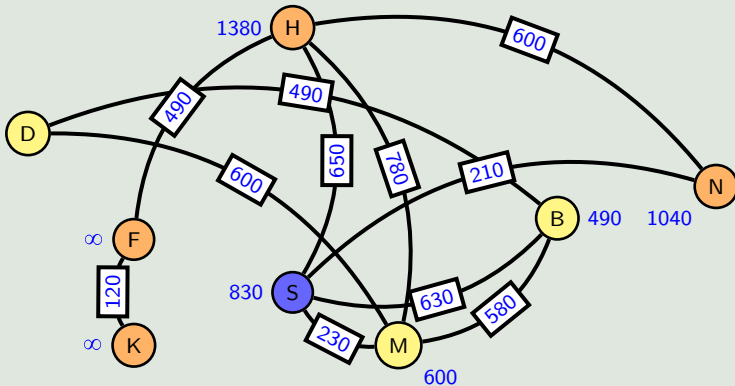


Example



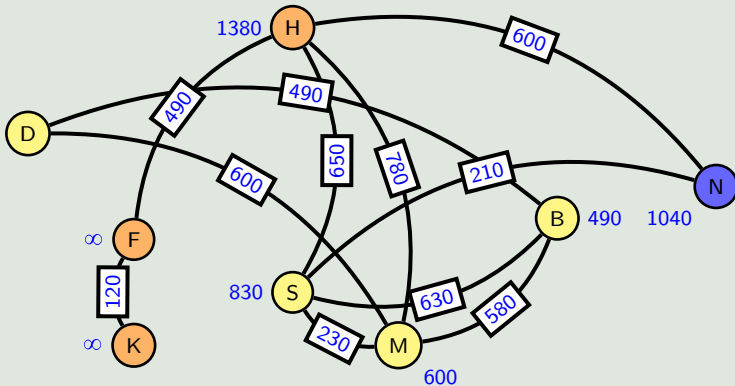
- ④ $S = \{ D, B, M, S \}, D_S(N) = \min \{ D_S(v) \};$
- ⑤ $S = \{ D, B, M, S, N \}, D_S(H) = \min \{ D_S(v) \};$
- ⑥ $S = \{ D, B, M, S, N, H \}, D_S(F) = \min \{ D_S(v) \};$
- ⑦ $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min \{ D_S(v) \}.$

Example



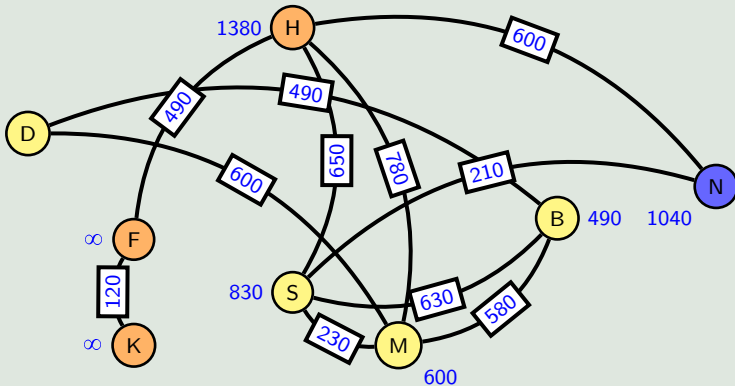
- ④ $S = \{ D, B, M, S \}, D_S(N) = \min\{ D_S(v) \};$
- ⑤ $S = \{ D, B, M, S, N \}, D_S(H) = \min\{ D_S(v) \};$
- ⑥ $S = \{ D, B, M, S, N, H \}, D_S(F) = \min\{ D_S(v) \};$
- ⑦ $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min\{ D_S(v) \}.$

Example



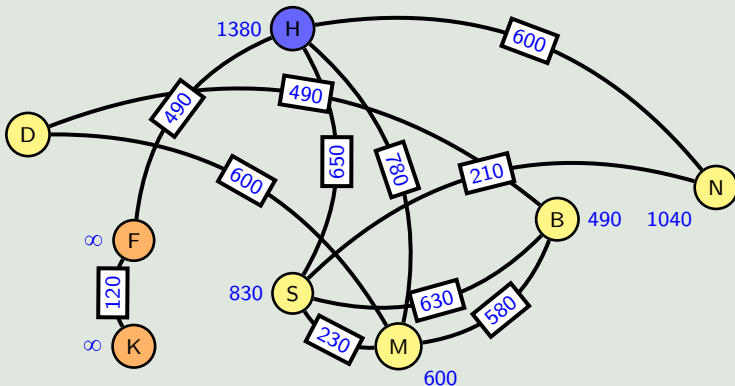
- ④ $S = \{ D, B, M, S \}, D_S(N) = \min\{ D_S(v) \};$
- ⑤ $S = \{ D, B, M, S, N \}, D_S(H) = \min\{ D_S(v) \};$
- ⑥ $S = \{ D, B, M, S, N, H \}, D_S(F) = \min\{ D_S(v) \};$
- ⑦ $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min\{ D_S(v) \}.$

Example



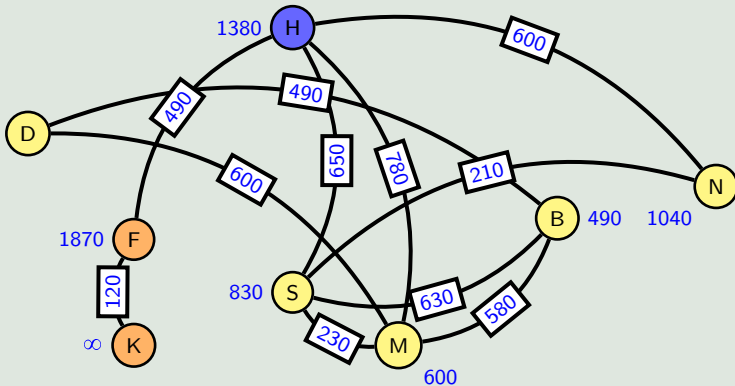
- ④ $S = \{ D, B, M, S \}, D_S(N) = \min\{ D_S(v) \};$
- ⑤ $S = \{ D, B, M, S, N \}, D_S(H) = \min\{ D_S(v) \};$
- ⑥ $S = \{ D, B, M, S, N, H \}, D_S(F) = \min\{ D_S(v) \};$
- ⑦ $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min\{ D_S(v) \}.$

Example



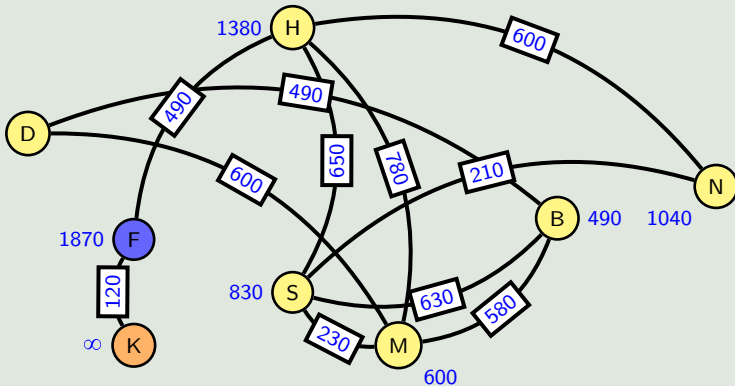
- 4 $S = \{ D, B, M, S \}, D_S(N) = \min\{ D_S(v) \};$
- 5 $S = \{ D, B, M, S, N \}, D_S(H) = \min\{ D_S(v) \};$
- 6 $S = \{ D, B, M, S, N, H \}, D_S(F) = \min\{ D_S(v) \};$
- 7 $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min\{ D_S(v) \}.$

Example



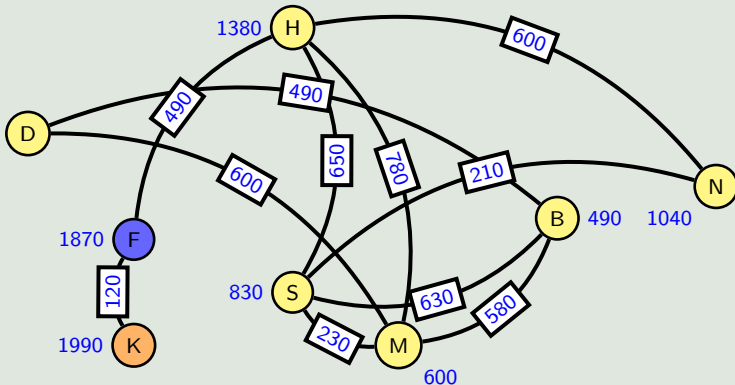
- ④ $S = \{ D, B, M, S \}, D_S(N) = \min\{ D_S(v) \};$
- ⑤ $S = \{ D, B, M, S, N \}, D_S(H) = \min\{ D_S(v) \};$
- ⑥ $S = \{ D, B, M, S, N, H \}, D_S(F) = \min\{ D_S(v) \};$
- ⑦ $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min\{ D_S(v) \}.$

Example



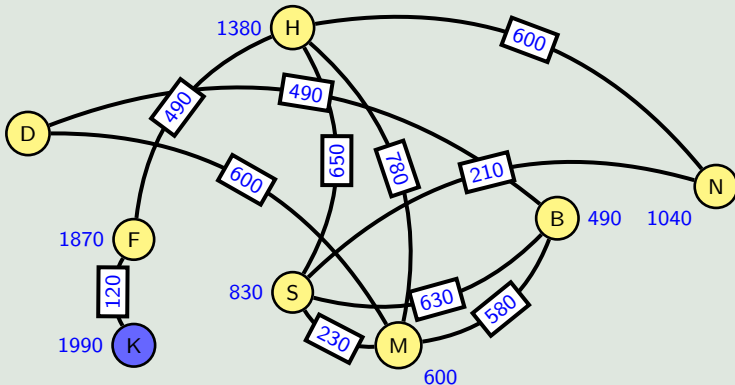
- ④ $S = \{ D, B, M, S \}, D_S(N) = \min\{ D_S(v) \};$
- ⑤ $S = \{ D, B, M, S, N \}, D_S(H) = \min\{ D_S(v) \};$
- ⑥ $S = \{ D, B, M, S, N, H \}, D_S(F) = \min\{ D_S(v) \};$
- ⑦ $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min\{ D_S(v) \}.$

Example



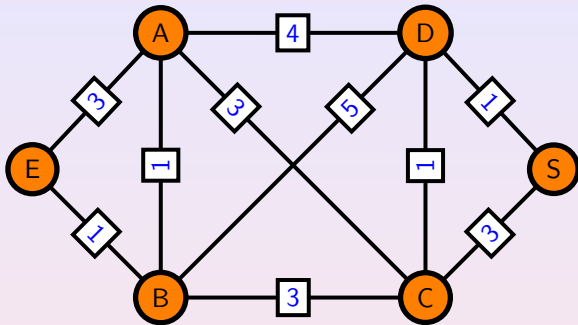
- ④ $S = \{ D, B, M, S \}, D_S(N) = \min\{ D_S(v) \};$
- ⑤ $S = \{ D, B, M, S, N \}, D_S(H) = \min\{ D_S(v) \};$
- ⑥ $S = \{ D, B, M, S, N, H \}, D_S(F) = \min\{ D_S(v) \};$
- ⑦ $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min\{ D_S(v) \}.$

Example



- ④ $S = \{ D, B, M, S \}, D_S(N) = \min\{ D_S(v) \};$
- ⑤ $S = \{ D, B, M, S, N \}, D_S(H) = \min\{ D_S(v) \};$
- ⑥ $S = \{ D, B, M, S, N, H \}, D_S(F) = \min\{ D_S(v) \};$
- ⑦ $S = \{ D, B, M, S, N, H, F \}, D_S(K) = \min\{ D_S(v) \}.$

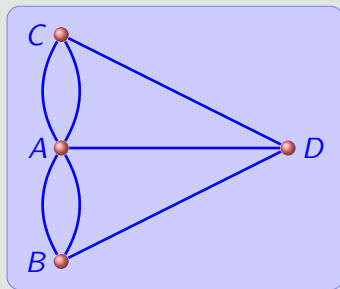
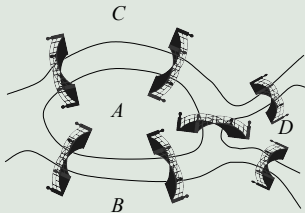
Dijkstra算法一个完整的例子



E	B	A	C	D	S	选择
0	∞	∞	∞	∞	∞	E
	1(E)	3(E)	∞	∞	∞	B
		2(B)	4(B)	6(B)	∞	A
			4(B)	6(B)	∞	C
				5(C)	7(C)	D
					6(D)	S

Euler路径与回路— Example

七桥问题(Königsberg Bridge)



Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径;
- 经过图中每个边一次并且仅一次的回路称为Euler回路.

Theorem

一个图存在Euler回路当且仅当该图是一个连通图且每个顶点的度数都是偶数.

一个图存在Euler路径当且仅当该图是一个连通图且至多有两个顶点的度数为奇数.

七桥问题

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径.

Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径;
- 经过图中每个边一次并且仅一次的回路称为Euler回路.

Theorem

1. 一个图有Euler回路, 当且仅当, 它是连通的, 且每个顶点的度数都是偶数.
2. 一个图有Euler路径, 当且仅当, 它是连通的, 至多有一个顶点的度数为奇数, 且有一个顶点度数为奇数, 其他顶点的度数都是偶数.
3. 一个图有哈密顿回路, 当且仅当, 它是连通的, 且每个顶点的度数都是偶数, 且每个顶点的度数都是偶数.
4. 一个图有哈密顿路径, 当且仅当, 它是连通的, 且每个顶点的度数都是偶数, 且每个顶点的度数都是偶数.

七桥问题

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径.

Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径;
- 经过图中每个边一次并且仅一次的回路称为Euler回路.

Theorem

- D 是Euler图 当且仅当 D 是强连通的, 且每个结点的出度与入度相等.

证明: 必要性: 若 D 是Euler图, 则 D 是强连通的. 因为 D 中每条边均被经过一次, 故 D 中任意两个结点 u, v 均存在一条从 u 到 v 的路径. 又因为 D 中每条边均被经过一次, 故 D 中任意两个结点 u, v 均存在一条从 v 到 u 的路径. 因此 D 是强连通的. 又因为 D 中每条边均被经过一次, 故 D 中每个结点的出度与入度相等.

充分性: 若 D 是强连通的, 且每个结点的出度与入度相等, 则 D 是Euler图. 因为 D 是强连通的, 故 D 中任意两个结点 u, v 均存在一条从 u 到 v 的路径. 又因为 D 中每个结点的出度与入度相等, 故 D 中每条边均被经过一次. 因此 D 是Euler图.

例题15

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径.

Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径;
- 经过图中每个边一次并且仅一次的回路称为Euler回路.

Theorem

- D 是Euler图 iff D 是强连通的, 且每个结点的出度与入度相等.
- D 有Euler路径 iff D 是弱连通的, 且最多有一个点的出度比入度大1, 同时也最多有一个点入度比出度大1, 其他的点出度和入度相等.
- G 是Euler图 iff G 中每个结点的度数均是偶数. G 中仅有0个度数为奇数的结点.
- G 有Euler路 iff G 中仅有0个或2个度数为奇数的结点.

图例4.5

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径.

Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径；
- 经过图中每个边一次并且仅一次的回路称为Euler回路。

Theorem

- D 是Euler图 iff D 是强连通的, 且每个结点的出度与入度相等。
- D 有Euler路径 iff D 是弱连通的, 且最多有一个点的出度比入度大1, 同时也最多有一个点入度比出度大1, 其他的点出度和入度相等。
- G 是Euler图 iff G 中每个结点的度数均是偶数. G 中仅有0个度数为奇数的结点。
- G 有Euler路 iff G 中仅有0个或2个度数为奇数的结点。

Example

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径。

Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径；
- 经过图中每个边一次并且仅一次的回路称为Euler回路。

Theorem

- D 是Euler图 iff D 是强连通的, 且每个结点的出度与入度相等。
- D 有Euler路径 iff D 是弱连通的, 且最多有一个点的出度比入度大1, 同时也最多有一个点入度比出度大1, 其他的点出度和入度相等。
- G 是Euler图 iff G 中每个结点的度数均是偶数. G 中仅有0个度数为奇数的结点。
- G 有Euler路 iff G 中仅有0个或2个度数为奇数的结点。

例题1

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径。

Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径；
- 经过图中每个边一次并且仅一次的回路称为Euler回路。

Theorem

- D 是Euler图 iff D 是强连通的, 且每个结点的出度与入度相等。
- D 有Euler路径 iff D 是弱连通的, 且最多有一个点的出度比入度大1, 同时也最多有一个点入度比出度大1, 其他的点出度和入度相等。
- G 是Euler图 iff G 中每个结点的度数均是偶数. G 中仅有0个度数为奇数的结点。
- G 有Euler路 iff G 中仅有0个或2个度数为奇数的结点。

七桥问题

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径。

Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径；
- 经过图中每个边一次并且仅一次的回路称为Euler回路。

Theorem

- D 是Euler图 iff D 是强连通的, 且每个结点的出度与入度相等。
- D 有Euler路径 iff D 是弱连通的, 且最多有一个点的出度比入度大1, 同时也最多有一个点入度比出度大1, 其他的点出度和入度相等。
- G 是Euler图 iff G 中每个结点的度数均是偶数. G 中仅有0个度数为奇数的结点。
- G 有Euler路 iff G 中仅有0个或2个度数为奇数的结点。

七桥问题

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径。

Euler路径与回路

Definition

- 经过图中每个边一次并且仅一次的路径称为Euler路径；
- 经过图中每个边一次并且仅一次的回路称为Euler回路。

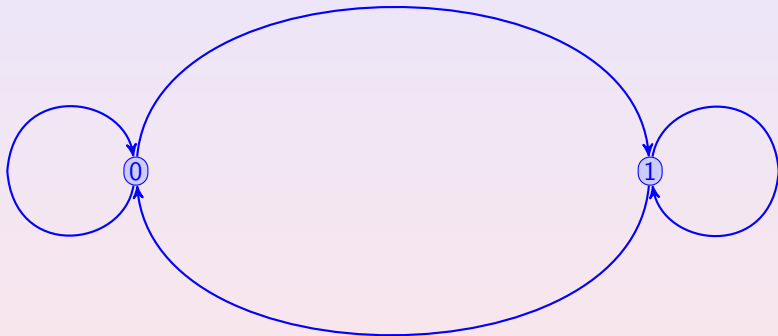
Theorem

- D 是Euler图 iff D 是强连通的, 且每个结点的出度与入度相等。
- D 有Euler路径 iff D 是弱连通的, 且最多有一个点的出度比入度大1, 同时也最多有一个点入度比出度大1, 其他的点出度和入度相等。
- G 是Euler图 iff G 中每个结点的度数均是偶数. G 中仅有0个度数为奇数的结点。
- G 有Euler路 iff G 中仅有0个或2个度数为奇数的结点。

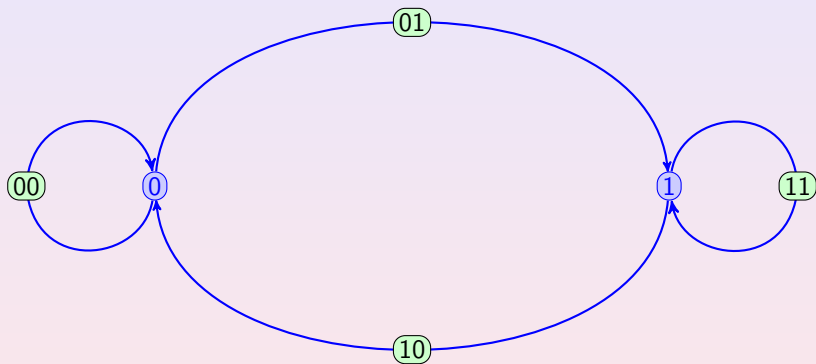
七桥问题

三个结点的度数为奇数, 没有Euler回路, 也没有Euler路径。

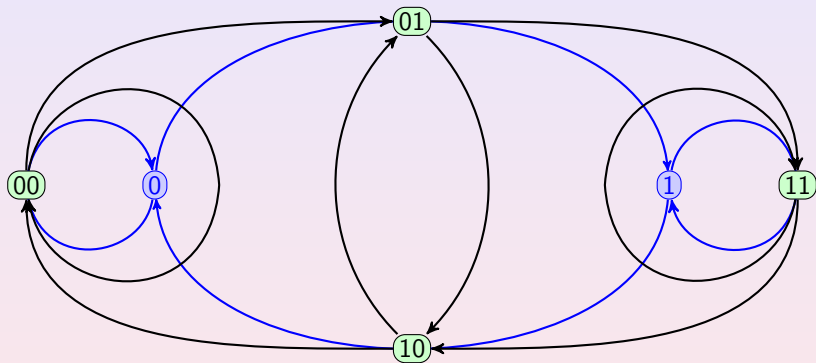
Example: De Bruijn图: 从 $B(2, 1)$ 到 $B(2, 3)$



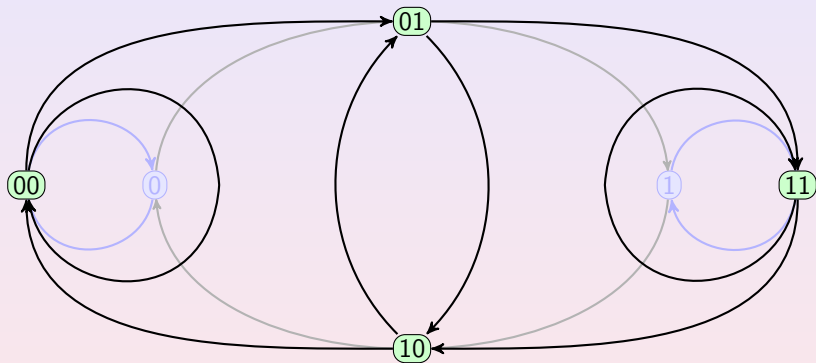
Example: De Bruijn图: 从 $B(2, 1)$ 到 $B(2, 3)$



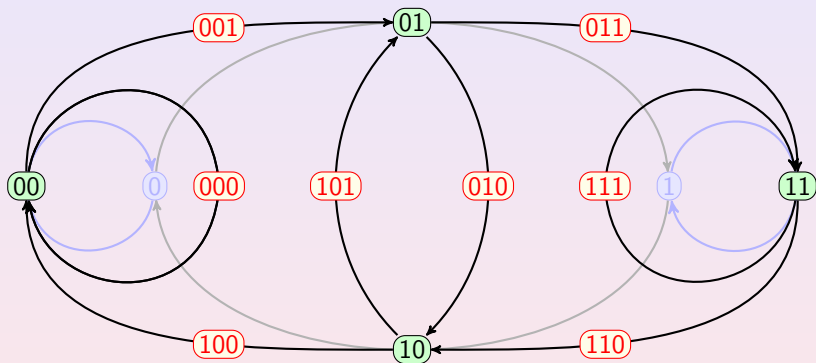
Example: De Bruijn图: 从 $B(2, 1)$ 到 $B(2, 3)$



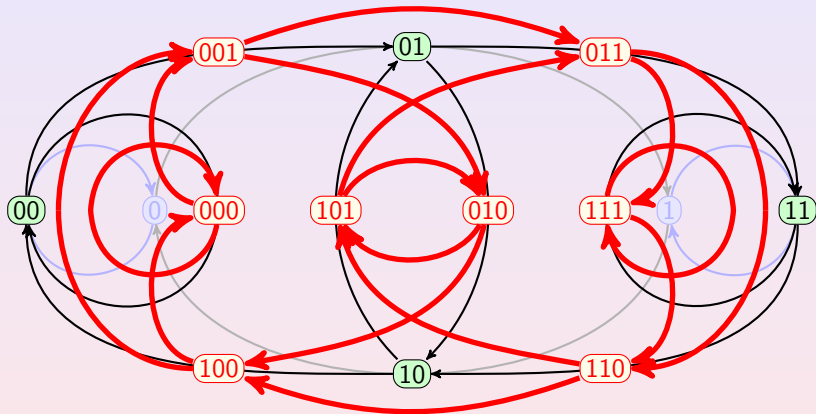
Example: De Bruijn图: 从 $B(2, 1)$ 到 $B(2, 3)$



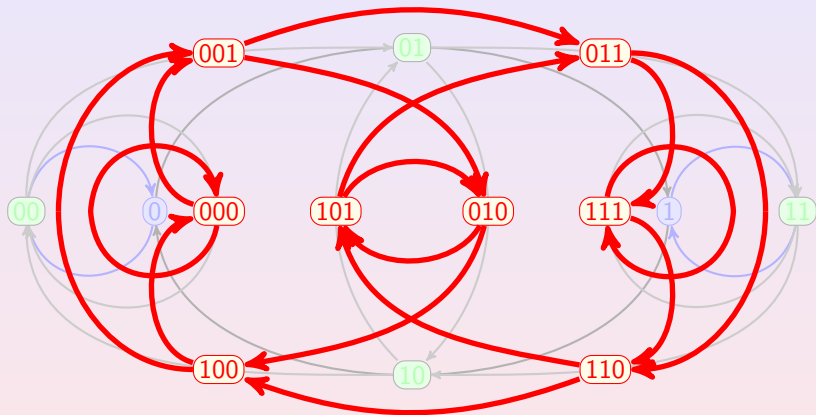
Example: De Bruijn图: 从 $B(2, 1)$ 到 $B(2, 3)$



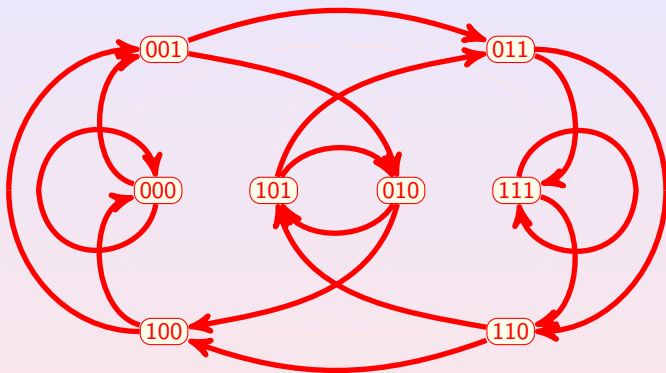
Example: De Bruijn图: 从 $B(2, 1)$ 到 $B(2, 3)$



Example: De Bruijn图: 从 $B(2, 1)$ 到 $B(2, 3)$

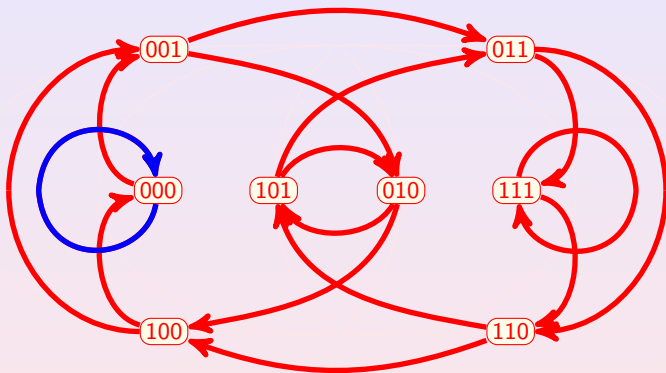


Example: De Bruijn图 的任意Euler回所移出的数字串



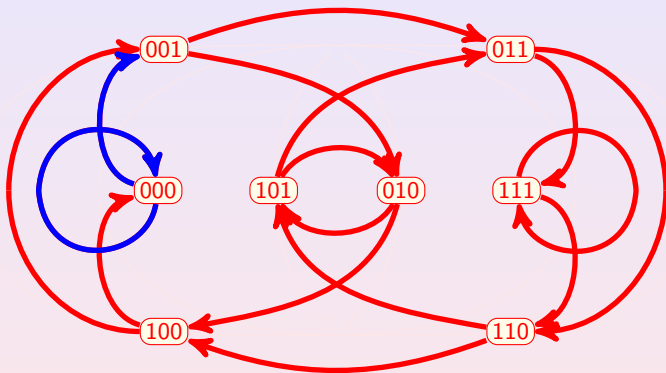
Euler回: 000,
移出数字:

Example: De Bruijn图 的任意Euler回所移出的数字串



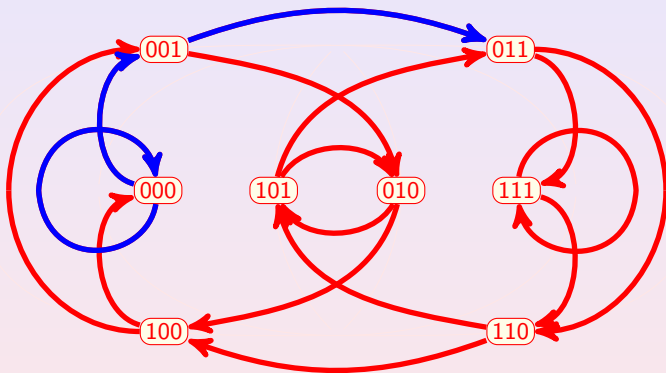
Euler回: 000, 000,
移出数字: 0,

Example: De Bruijn图 的任意Euler回所移出的数字串



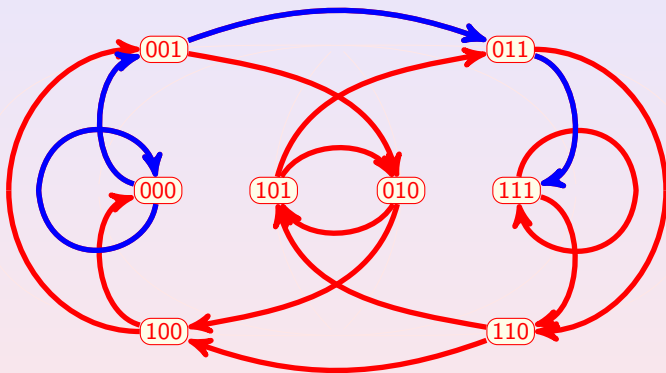
Euler回: 000, 000, 001,
移出数字: 0, 0,

Example: De Bruijn图 的任意Euler回所移出的数字串



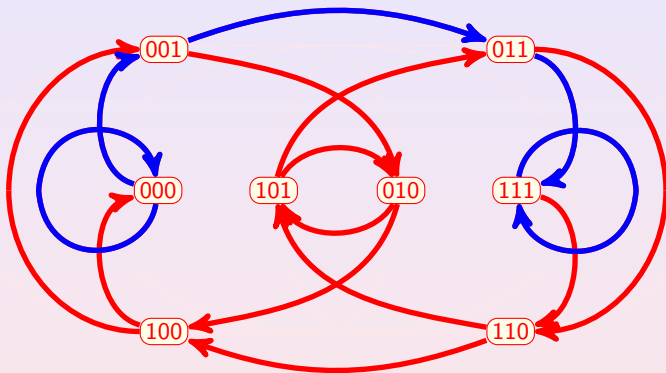
Euler回: 000, 000, 001, 011,
移出数字: 0, 0, 0,

Example: De Bruijn图 的任意Euler回所移出的数字串



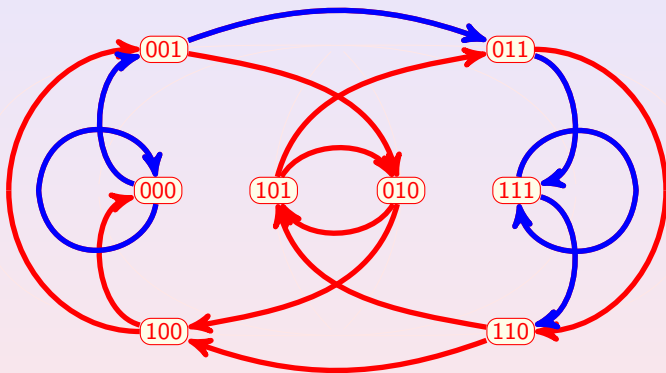
Euler回: 000, 000, 001, 011, 111,
移出数字: 0, 0, 0, 0,

Example: De Bruijn图 的任意Euler回所移出的数字串



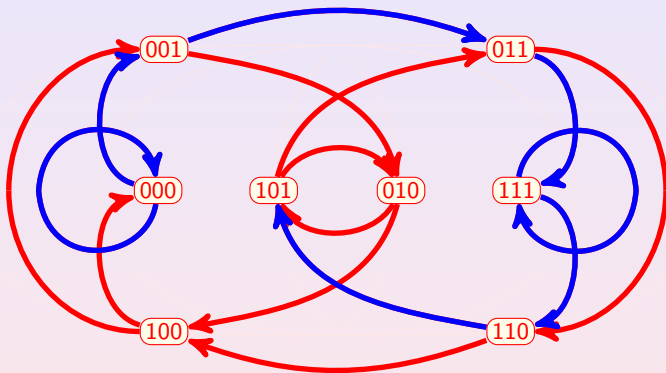
Euler回: 000, 000, 001, 011, 111, 111,
移出数字: 0, 0, 0, 0, 1,

Example: De Bruijn图 的任意Euler回所移出的数字串



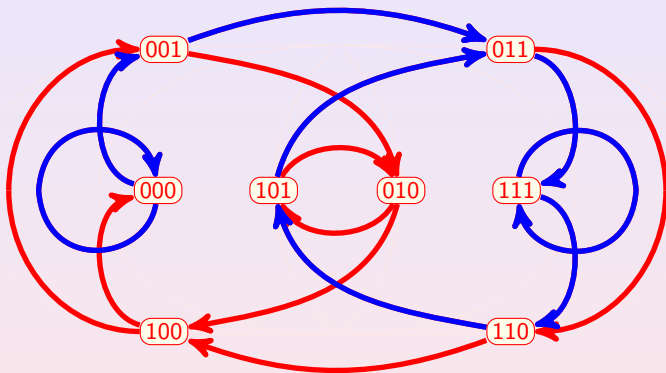
Euler回: 000, 000, 001, 011, 111, 111, 110,
 移出数字: 0, 0, 0, 0, 1, 1,

Example: De Bruijn 图的任意 Euler 回所移出的数字串



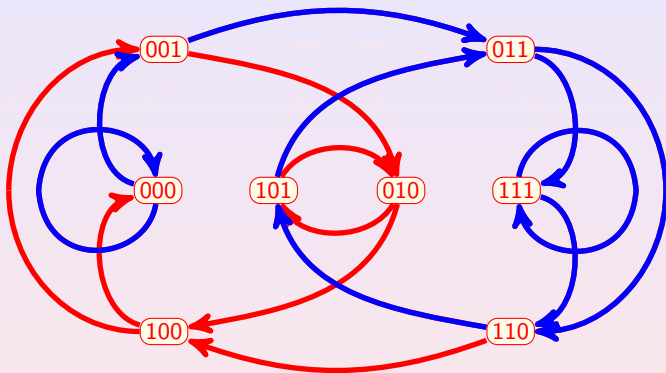
Euler回: 000, 000, 001, 011, 111, 111, 110, 101,
 移出数字: 0, 0, 0, 0, 1, 1, 1,

Example: De Bruijn图 的任意Euler回所移出的数字串



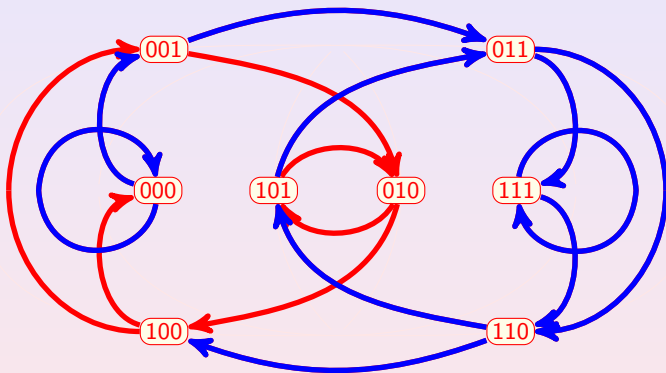
Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011,
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1,

Example: De Bruijn图 的任意Euler回所移出的数字串



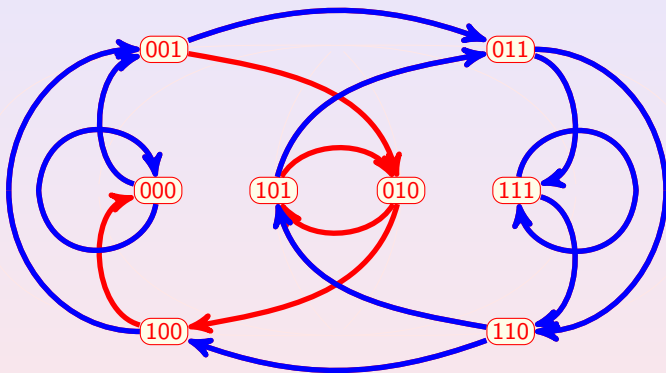
Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011, 110,
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1, 0,

Example: De Bruijn图 的任意Euler回所移出的数字串



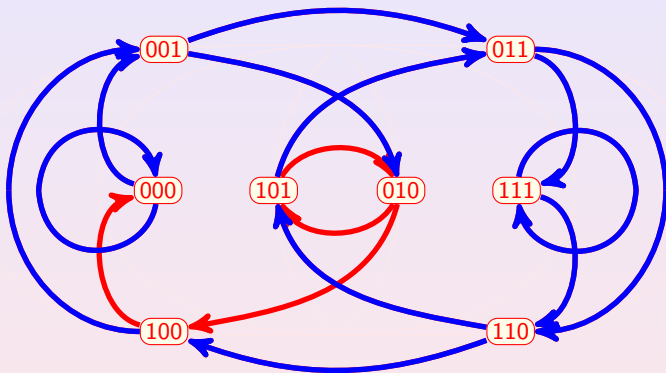
Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011, 110, 100,
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1, 0, 1,

Example: De Bruijn图 的任意Euler回所移出的数字串



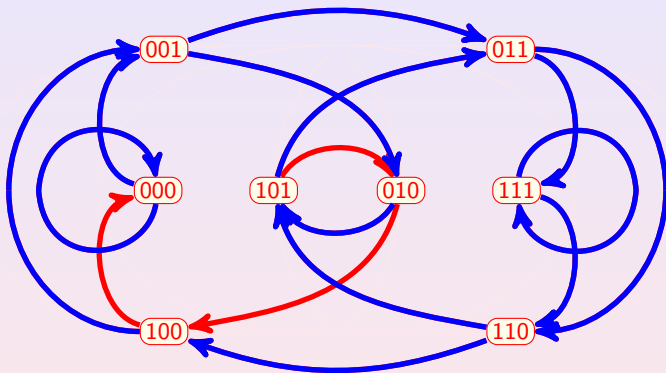
Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011, 110, 100, 001,
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1,

Example: De Bruijn图 的任意Euler回所移出的数字串



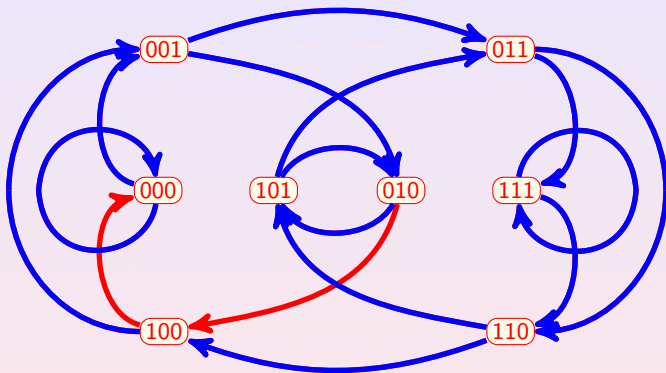
Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011, 110, 100, 001, 010,
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0,

Example: De Bruijn图 的任意Euler回所移出的数字串



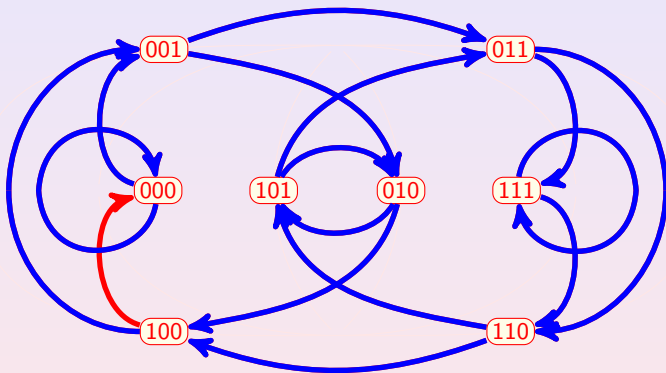
Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011, 110, 100, 001, 010, 101,
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0,

Example: De Bruijn图 的任意Euler回所移出的数字串



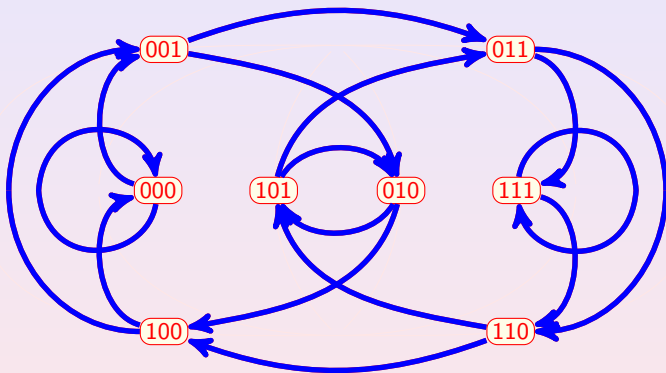
Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011, 110, 100, 001, 010, 101, 010,
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1,

Example: De Bruijn图 的任意Euler回所移出的数字串



Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011, 110, 100, 001, 010, 101, 010, 100,
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0,

Example: De Bruijn图 的任意Euler回所移出的数字串



Euler回: 000, 000, 001, 011, 111, 111, 110, 101, 011, 110, 100, 001, 010, 101, 010, 100, 000
 移出数字: 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1

De Bruijn 序列

{0	0	0	0}	1	1	1	1	0	1	1	0	0	1	0	1		
0	{0	0	0	1}	1	1	1	0	1	1	0	0	1	0	1		
0	0	{0	0	1	1}	1	1	0	1	1	0	0	1	0	1		
0	0	0	{0	1	1	1}	1	0	1	1	0	0	1	0	1		
0	0	0	0	{1	1	1	1}	0	1	1	0	0	1	0	1		
0	0	0	0	1	{1	1	1	0}	1	1	0	0	1	0	1		
0	0	0	0	1	1	{1	1	0	1}	1	0	0	1	0	1		
0	0	0	0	1	1	1	{1	0	1	1}	0	0	1	0	1		
0	0	0	0	1	1	1	1	{0	1	1	0}	0	1	0	1		
0	0	0	0	1	1	1	1	0	{1	1	0	0}	1	0	1		
0	0	0	0	1	1	1	1	0	1	{1	0	0	1}	0	1		
0	0	0	0	1	1	1	1	0	1	1	{0	0	1	0}	1		
0	0	0	0	1	1	1	1	0	1	1	0	{0	1	0	1}		
0}	0	0	0	1	1	1	1	0	1	1	0	0	{1	0	1		
...	0	0}	0	0	1	1	1	1	0	1	1	0	0	1	{0	1	...
...	0	0	0}	0	1	1	1	1	0	1	1	0	0	1	0	{1	...

割边

Definition

设 G 是无向连通图， $e \in E$ 称为**割边(桥)**，iff， G 删除边 e 之后将增加连通分支数.

Example

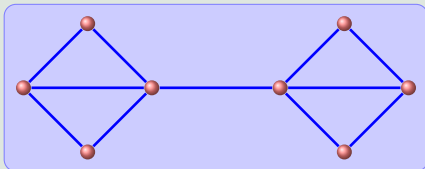


割边

Definition

设 G 是无向连通图, $e \in E$ 称为**割边(桥)**, iff, G 删除边 e 之后将增加连通分支数.

Example

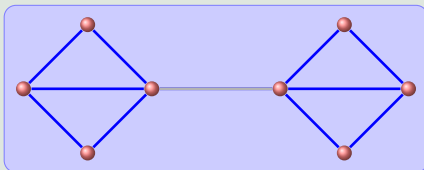


割边

Definition

设 G 是无向连通图, $e \in E$ 称为**割边(桥)**, iff, G 删除边 e 之后将增加连通分支数.

Example



割边与圈的关系

Theorem

边 e 是图 G 的割边当且仅当边 e 不在任何基本回路上。

Proof(反证法)

● 设 e 是包含边 e (端点为 x 和 y)的连通分支, 则仅需证明结论对 e 成立。

● 假设 e 不是割边, 那么 e 在 G 的某个基本回路上, 需证明删除 e 后, 图 G 仍然连通, 即删除 e 后图 G 仍是连通。

● 假设 e 不是割边, 那么 e 在 G 的某个基本回路上, 需证明删除 e 后, 图 G 仍是连通。

● 假设 e 不是割边, 那么 e 在 G 的某个基本回路上, 需证明删除 e 后, 图 G 仍是连通。
 假设 e 不是割边, 那么 e 在 G 的某个基本回路上, 需证明删除 e 后, 图 G 仍是连通。
 假设 e 不是割边, 那么 e 在 G 的某个基本回路上, 需证明删除 e 后, 图 G 仍是连通。
 假设 e 不是割边, 那么 e 在 G 的某个基本回路上, 需证明删除 e 后, 图 G 仍是连通。
 假设 e 不是割边, 那么 e 在 G 的某个基本回路上, 需证明删除 e 后, 图 G 仍是连通。



割边与圈的关系

Theorem

边 e 是图 G 的割边当且仅当边 e 不在任何基本回路上.

Proof(反证法)

- ① 设 H 是包含边 e (端点为 x 和 y)的连通分支, 则仅需证明结论对 H 成立.
- ② 必要性: 设 e 是割边, 但是 e 在 H 的某个基本回路 C 上, 需证明删除 e 后还保持连通, 从而与割边的定义矛盾.
- ③ 设 $u, v \in V(H)$. 若 (u, v) 路径不经过 e , 则 u 和 v 在 $H - e$ 还是可达.
- ④ 若 (u, v) 路径经过 e , 可设 (u, v) 路径为 $(ue_1u_1 \cdots xeye'_1v_1 \cdots v)$, 则 u 和 x , y 和 v 在图 $H - e$ 中可达, 而 e 在回路 C 上, $\therefore$ 删除 e 后, x 和 y 还是可达, 根据可达关系的传递性 u 和 v 在 $H - e$ 中可达, 即 $H - e$ 是连通图. 矛盾.
- ⑤ 充分性: 设 e 不在任何回路上, 但 $H - e$ 后还保持连通, 即不经过 e 的 (x, y) 路径 P 存在, 这样 P 加 e 即构成回路, 从而导致矛盾.



割边与圈的关系

Theorem

边 e 是图 G 的割边当且仅当边 e 不在任何基本回路上。

Proof(反证法)

- ① 设 H 是包含边 e (端点为 x 和 y)的连通分支, 则仅需证明结论对 H 成立.
- ② 必要性: 设 e 是割边, 但是 e 在 H 的某个基本回路 C 上, 需证明删除 e 后还保持连通, 从而与割边的定义矛盾.
- ③ 设 $u, v \in V(H)$. 若 (u, v) 路径不经过 e , 则 u 和 v 在 $H - e$ 还是可达.
- ④ 若 (u, v) 路径经过 e , 可设 (u, v) 路径为 $(ue_1u_1 \cdots xeye'_1v_1 \cdots v)$, 则 u 和 x , y 和 v 在图 $H - e$ 中可达, 而 e 在回路 C 上, $\therefore$ 删除 e 后, x 和 y 还是可达, 根据可达关系的传递性 u 和 v 在 $H - e$ 中可达, 即 $H - e$ 是连通图. 矛盾.
- ⑤ 充分性: 设 e 不在任何回路上, 但 $H - e$ 后还保持连通, 即不经过 e 的 (x, y) 路径 P 存在, 这样 P 加 e 即构成回路, 从而导致矛盾.



割边与圈的关系

Theorem

边 e 是图 G 的割边当且仅当边 e 不在任何基本回路上。

Proof(反证法)

- ① 设 H 是包含边 e (端点为 x 和 y)的连通分支, 则仅需证明结论对 H 成立.
- ② 必要性: 设 e 是割边, 但是 e 在 H 的某个基本回路 C 上, 需证明删除 e 后还保持连通, 从而与割边的定义矛盾.
- ③ 设 $u, v \in V(H)$. 若 (u, v) 路径不经过 e , 则 u 和 v 在 $H - e$ 还是可达.
- ④ 若 (u, v) 路径经过 e , 可设 (u, v) 路径为 $(ue_1u_1 \cdots xeye'_1v_1 \cdots v)$, 则 u 和 x , y 和 v 在图 $H - e$ 中可达, 而 e 在回路 C 上, $\therefore$ 删除 e 后, x 和 y 还是可达, 根据可达关系的传递性 u 和 v 在 $H - e$ 中可达, 即 $H - e$ 是连通图. 矛盾.
- ⑤ 充分性: 设 e 不在任何回路上, 但 $H - e$ 后还保持连通, 即不经过 e 的 (x, y) 路径 P 存在, 这样 P 加 e 即构成回路, 从而导致矛盾.



割边与圈的关系

Theorem

边 e 是图 G 的割边当且仅当边 e 不在任何基本回路上.

Proof(反证法)

- ① 设 H 是包含边 e (端点为 x 和 y)的连通分支, 则仅需证明结论对 H 成立.
- ② 必要性: 设 e 是割边, 但是 e 在 H 的某个基本回路 C 上, 需证明删除 e 后还保持连通, 从而与割边的定义矛盾.
- ③ 设 $u, v \in V(H)$. 若 (u, v) 路径不经过 e , 则 u 和 v 在 $H - e$ 还是可达.
- ④ 若 (u, v) 路径经过 e , 可设 (u, v) 路径为 $(ue_1u_1 \cdots xeye'_1v_1 \cdots v)$, 则 u 和 x , y 和 v 在图 $H - e$ 中可达, 而 e 在回路 C 上, $\therefore$ 删除 e 后, x 和 y 还是可达, 根据可达关系的传递性 u 和 v 在 $H - e$ 中可达, 即 $H - e$ 是连通图. 矛盾.
- ⑤ 充分性: 设 e 不在任何回路上, 但 $H - e$ 后还保持连通, 即不经过 e 的 (x, y) 路径 P 存在, 这样 P 加 e 即构成回路, 从而导致矛盾.



割边与圈的关系

Theorem

边 e 是图 G 的割边当且仅当边 e 不在任何基本回路上。

Proof(反证法)

- ① 设 H 是包含边 e (端点为 x 和 y)的连通分支, 则仅需证明结论对 H 成立.
- ② 必要性: 设 e 是割边, 但是 e 在 H 的某个基本回路 C 上, 需证明删除 e 后还保持连通, 从而与割边的定义矛盾.
- ③ 设 $u, v \in V(H)$. 若 (u, v) 路径不经过 e , 则 u 和 v 在 $H - e$ 还是可达.
- ④ 若 (u, v) 路径经过 e , 可设 (u, v) 路径为 $(ue_1u_1 \cdots xeye'_1v_1 \cdots v)$, 则 u 和 x , y 和 v 在图 $H - e$ 中可达, 而 e 在回路 C 上, $\therefore$ 删除 e 后, x 和 y 还是可达, 根据可达关系的传递性 u 和 v 在 $H - e$ 中可达, 即 $H - e$ 是连通图. 矛盾.
- ⑤ 充分性: 设 e 不在任何回路上, 但 $H - e$ 后还保持连通, 即不经过 e 的 (x, y) 路径 P 存在, 这样 P 加 e 即构成回路, 从而导致矛盾.



割边与圈的关系

Theorem

边 e 是图 G 的割边当且仅当边 e 不在任何基本回路上.

Proof(反证法)

- ① 设 H 是包含边 e (端点为 x 和 y)的连通分支, 则仅需证明结论对 H 成立.
- ② 必要性: 设 e 是割边, 但是 e 在 H 的某个基本回路 C 上, 需证明删除 e 后还保持连通, 从而与割边的定义矛盾.
- ③ 设 $u, v \in V(H)$. 若 (u, v) 路径不经过 e , 则 u 和 v 在 $H - e$ 还是可达.
- ④ 若 (u, v) 路径经过 e , 可设 (u, v) 路径为 $(ue_1u_1 \cdots xeye'_1v_1 \cdots v)$, 则 u 和 x , y 和 v 在图 $H - e$ 中可达, 而 e 在回路 C 上, $\therefore$ 删除 e 后, x 和 y 还是可达, 根据可达关系的传递性 u 和 v 在 $H - e$ 中可达, 即 $H - e$ 是连通图. 矛盾.
- ⑤ 充分性: 设 e 不在任何回路上, 但 $H - e$ 后还保持连通, 即不经过 e 的 (x, y) 路径 P 存在, 这样 P 加 e 即构成回路, 从而导致矛盾.



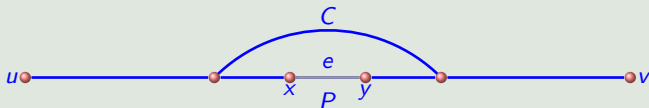
割边与圈的关系

Theorem

边 e 是图 G 的割边当且仅当边 e 不在任何基本回路上。

Proof(反证法)

- ① 设 H 是包含边 e (端点为 x 和 y)的连通分支, 则仅需证明结论对 H 成立.
- ② 必要性: 设 e 是割边, 但是 e 在 H 的某个基本回路 C 上, 需证明删除 e 后还保持连通, 从而与割边的定义矛盾.
- ③ 设 $u, v \in V(H)$. 若 (u, v) 路径不经过 e , 则 u 和 v 在 $H - e$ 还是可达.
- ④ 若 (u, v) 路径经过 e , 可设 (u, v) 路径为 $(ue_1u_1 \cdots xeye'_1v_1 \cdots v)$, 则 u 和 x , y 和 v 在图 $H - e$ 中可达, 而 e 在回路 C 上, $\therefore$ 删除 e 后, x 和 y 还是可达, 根据可达关系的传递性 u 和 v 在 $H - e$ 中可达, 即 $H - e$ 是连通图. 矛盾.
- ⑤ 充分性: 设 e 不在任何回路上, 但 $H - e$ 后还保持连通, 即不经过 e 的 (x, y) 路径 P 存在, 这样 P 加 e 即构成回路, 从而导致矛盾.



Fleury's Algorithm

Fleury's Algorithm

设 $G(n, m)$ 是无向连通图, 则: 按下述算法构造出的边的序列一定是Euler回路;

- ① 初始化: $C := ()$, $v :=$ 任意一结点;
- ② if $length(C) = m$, then Stop;
- ③ 设 $C = (e_1, e_2, \dots, e_k)$, if 图 $G - \{e_1, e_2, \dots, e_k\}$ 有连接 v 一个非割边, 则选择该非割边, 否则选择连接 v 的一个割边, 设所选的边为 e_{k+1} ; 令 $C := \text{concate}(C, e_{k+1})$; $v := e_{k+1}$ 的另一个端点;
- ④ goto ②.

Fleury's Algorithm

Fleury's Algorithm

设 $G(n, m)$ 是无向连通图，则：按下述算法构造出的边的序列一定是 Euler 回路；

- ① 初始化: $C := ()$, $v :=$ 任意一结点;
- ② if $length(C) = m$, then Stop;
- ③ 设 $C = (e_1, e_2, \dots, e_k)$, if 图 $G - \{e_1, e_2, \dots, e_k\}$ 有连接 v 一个非割边, 则选择该非割边, 否则选择连接 v 的一个割边, 设所选的边为 e_{k+1} ; 令 $C := \text{concate}(C, e_{k+1})$; $v := e_{k+1}$ 的另一个端点;
- ④ goto ②.

Fleury's Algorithm

Fleury's Algorithm

设 $G(n, m)$ 是无向连通图, 则: 按下述算法构造出的边的序列一定是 Euler 回路;

- ① 初始化: $C := ()$, $v :=$ 任意一结点;
- ② if $length(C) = m$, then Stop;
- ③ 设 $C = (e_1, e_2, \dots, e_k)$, if 图 $G - \{e_1, e_2, \dots, e_k\}$ 有连接 v 一个非割边, 则选择该非割边, 否则选择连接 v 的一个割边, 设所选的边为 e_{k+1} ; 令 $C := \text{concate}(C, e_{k+1})$; $v := e_{k+1}$ 的另一个端点;
- ④ goto ②.

Fleury's Algorithm

Fleury's Algorithm

设 $G(n, m)$ 是无向连通图, 则: 按下述算法构造出的边的序列一定是 Euler 回路;

- ① 初始化: $C := ()$, $v :=$ 任意一结点;
- ② if $length(C) = m$, then Stop;
- ③ 设 $C = (e_1, e_2, \dots, e_k)$, if 图 $G - \{e_1, e_2, \dots, e_k\}$ 有连接 v 一个非割边, 则选择该非割边, 否则选择连接 v 的一个割边, 设所选的边为 e_{k+1} ; 令 $C := \text{concat}(C, e_{k+1})$; $v := e_{k+1}$ 的另一个端点;
- goto ②.

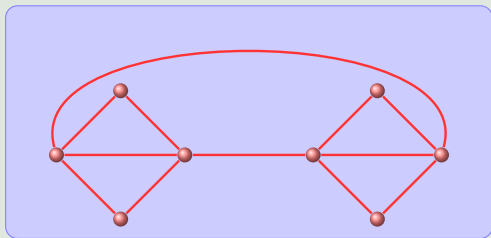
Fleury's Algorithm

Fleury's Algorithm

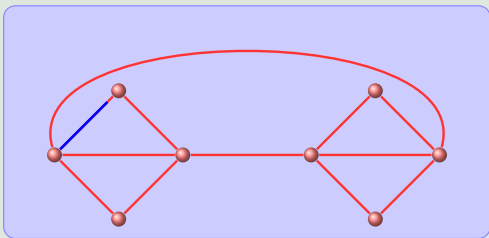
设 $G(n, m)$ 是无向连通图, 则: 按下述算法构造出的边的序列一定是 Euler 回路;

- ① 初始化: $C := ()$, $v :=$ 任意一结点;
- ② if $length(C) = m$, then Stop;
- ③ 设 $C = (e_1, e_2, \dots, e_k)$, if 图 $G - \{e_1, e_2, \dots, e_k\}$ 有连接 v 一个非割边, 则选择该非割边, 否则选择连接 v 的一个割边, 设所选的边为 e_{k+1} ; 令 $C := \text{concate}(C, e_{k+1})$; $v := e_{k+1}$ 的另一个端点;
- ④ goto ②.

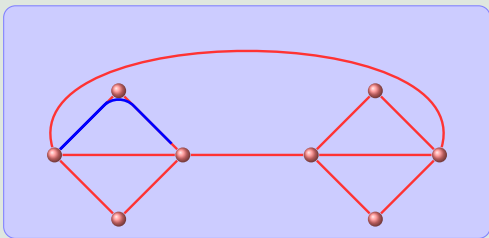
Fleury's Algorithm 动态演示



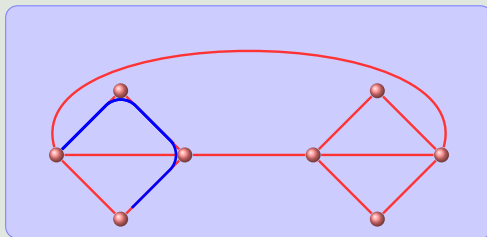
Fleury's Algorithm 动态演示



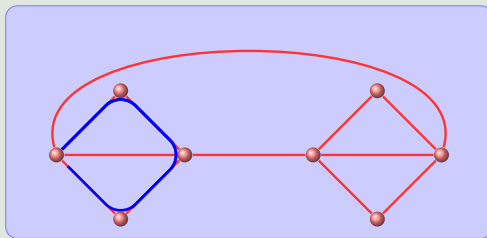
Fleury's Algorithm 动态演示



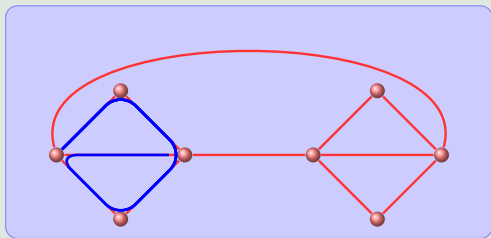
Fleury's Algorithm 动态演示



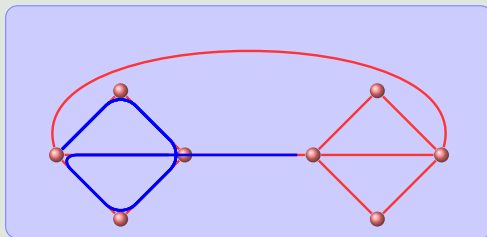
Fleury's Algorithm 动态演示



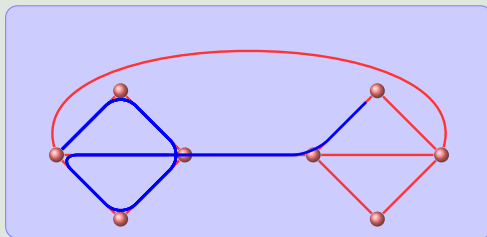
Fleury's Algorithm 动态演示



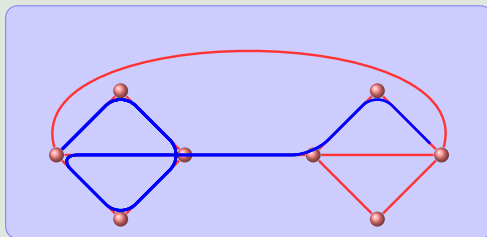
Fleury's Algorithm 动态演示



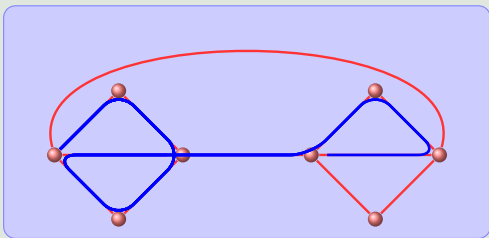
Fleury's Algorithm 动态演示



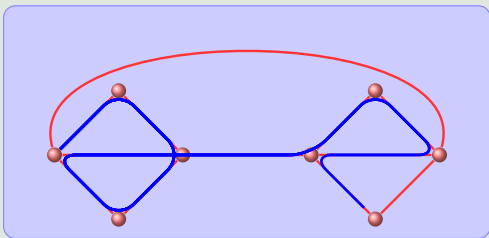
Fleury's Algorithm 动态演示



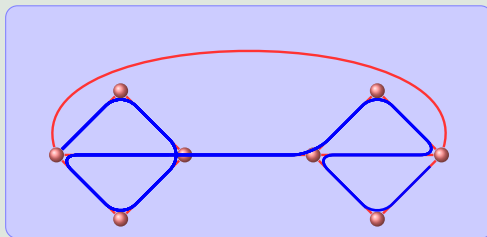
Fleury's Algorithm 动态演示



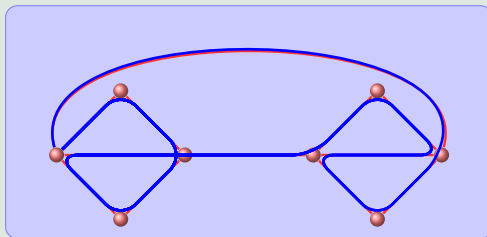
Fleury's Algorithm 动态演示



Fleury's Algorithm 动态演示

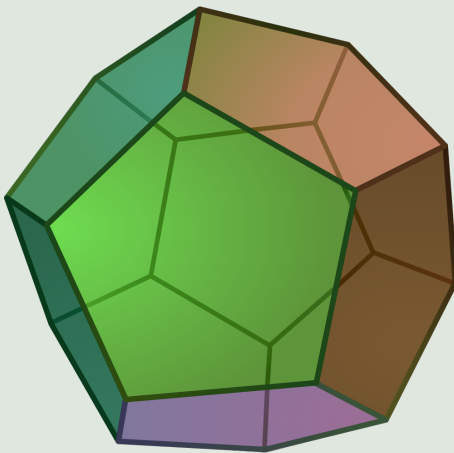


Fleury's Algorithm 动态演示



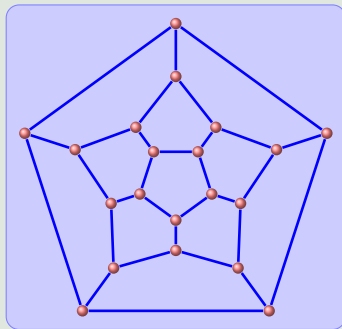
Hamilton路径与回路— Example (1/2)

Hamilton's "Round the World" Puzzle, 正12面体(Dodecahedron)



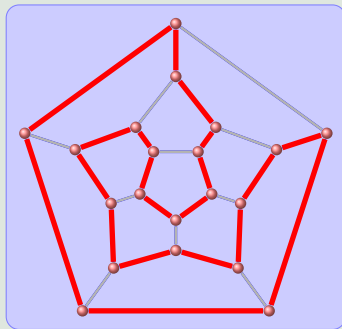
Hamilton路径与回路— Example(2/2)

Hamilton's "Round the World" Puzzle



Hamilton路径与回路— Example(2/2)

Hamilton's "Round the World" Puzzle



Hamilton路径与回路

Definition

- 经过图中每个结点一次并且仅一次的路径称为Hamilton路径;
- 经过图中每个结点一次并且仅一次的回路称为Hamilton回路.

Theorem

设 $G(n, m)$ 是简单无向图, 则:

Proof(连通性).

反证法: 设 G 不是连通图, 则 $G = G_1 \cup G_2$, 并且 $G_1 \cap G_2 = \emptyset$,
 $|V_1| = n_1$, $|V_2| = n_2$, $n_1 + n_2 = n$, 设 $u \in V_1 \wedge v \in V_2$,
 则 $\deg(u) \leq n_1 - 1 \wedge \deg(v) \leq n_2 - 1$, $\therefore \deg(u) + \deg(v) \leq n - 2$, 矛盾. □

Hamilton路径与回路

Definition

- 经过图中每个结点一次并且仅一次的路径称为Hamilton路径;
- 经过图中每个结点一次并且仅一次的回路称为Hamilton回路。

Theorem

设 $G(n, m)$ 是简单无向图, 则:

- 若 G 是连通图, 则 G 是哈密顿图, 当且仅当 G 中存在一条Hamilton回路;
- 若 G 是连通图, 则 G 是哈密顿图, 当且仅当 G 中存在一条Hamilton路径。

Proof(连通性).

反证法: 设 G 不是连通图, 则 $G = G_1 \cup G_2$, 并且 $G_1 \cap G_2 = \emptyset$,

$|V_1| = n_1$, $|V_2| = n_2$, $n_1 + n_2 = n$, 设 $u \in V_1 \wedge v \in V_2$,

则 $\deg(u) \leq n_1 - 1 \wedge \deg(v) \leq n_2 - 1$, $\therefore \deg(u) + \deg(v) \leq n - 2$, 矛盾. □

Hamilton路径与回路

Definition

- 经过图中每个结点一次并且仅一次的路径称为Hamilton路径;
- 经过图中每个结点一次并且仅一次的回路称为Hamilton回路.

Theorem

设 $G(n, m)$ 是简单无向图, 则:

- 如果每对结点的度数之和大于 $n-1$, 则该国一定存在Hamilton路;
- 如果每对结点度数之和大于 n , 则该国一定存在Hamilton回路.

Proof(连通性).

反证法: 设 G 不是连通图, 则 $G = G_1 \cup G_2$, 并且 $G_1 \cap G_2 = \emptyset$,

$|V_1| = n_1, |V_2| = n_2, n_1 + n_2 = n$, 设 $u \in V_1 \wedge v \in V_2$,

则 $\deg(u) \leq n_1 - 1 \wedge \deg(v) \leq n_2 - 1, \therefore \deg(u) + \deg(v) \leq n - 2$, 矛盾. □

Hamilton路径与回路

Definition

- 经过图中每个结点一次并且仅一次的路径称为Hamilton路径；
- 经过图中每个结点一次并且仅一次的回路称为Hamilton回路。

Theorem

设 $G(n, m)$ 是简单无向图，则：

- 如果每对结点的度数之和大于 $n-1$ ，则该图一定有条Hamilton路；
- 如果每对结点的度数之和大于 n ，则该图一定有条Hamilton回路。

Proof(连通性)

反证法：设 G 不是连通图，则 $G = G_1 \cup G_2$ ，并且 $G_1 \cap G_2 = \emptyset$ ，

$|V_1| = n_1$ ， $|V_2| = n_2$ ， $n_1 + n_2 = n$ ，设 $u \in V_1 \wedge v \in V_2$ ，

则 $\deg(u) \leq n_1 - 1 \wedge \deg(v) \leq n_2 - 1$ ， $\therefore \deg(u) + \deg(v) \leq n - 2$ ，矛盾。



Hamilton路径与回路

Definition

- 经过图中每个结点一次并且仅一次的路径称为Hamilton路径;
- 经过图中每个结点一次并且仅一次的回路称为Hamilton回路.

Theorem

设 $G(n, m)$ 是简单无向图, 则:

- 如果每对结点的度数之和大于 $n-1$, 则该图一定有条Hamilton路;
- 如果每对结点的度数之和大于 n , 则该图一定有条Hamilton回路.

Proof(连通性).

反证法: 设 G 不是连通图, 则 $G = G_1 \cup G_2$, 并且 $G_1 \cap G_2 = \emptyset$,

$|V_1| = n_1, |V_2| = n_2, n_1 + n_2 = n$, 设 $u \in V_1 \wedge v \in V_2$,

则 $\deg(u) \leq n_1 - 1 \wedge \deg(v) \leq n_2 - 1, \therefore \deg(u) + \deg(v) \leq n - 2$, 矛盾.



Hamilton路径与回路

Definition

- 经过图中每个结点一次并且仅一次的路径称为Hamilton路径；
- 经过图中每个结点一次并且仅一次的回路称为Hamilton回路。

Theorem

设 $G(n, m)$ 是简单无向图，则：

- 如果每对结点的度数之和大于 $n-1$ ，则该图一定有条Hamilton路；
- 如果每对结点的度数之和大于 n ，则该图一定有条Hamilton回路。

Proof(连通性).

反证法：设 G 不是连通图，则 $G = G_1 \cup G_2$ ，并且 $G_1 \cap G_2 = \emptyset$ ，

$|V_1| = n_1$ ， $|V_2| = n_2$ ， $n_1 + n_2 = n$ ，设 $u \in V_1 \wedge v \in V_2$ ，

则 $\deg(u) \leq n_1 - 1 \wedge \deg(v) \leq n_2 - 1$ ， $\therefore \deg(u) + \deg(v) \leq n - 2$ ，矛盾。



Hamilton路径与回路

Definition

- 经过图中每个结点一次并且仅一次的路径称为Hamilton路径;
- 经过图中每个结点一次并且仅一次的回路称为Hamilton回路.

Theorem

设 $G(n, m)$ 是简单无向图, 则:

- 如果每对结点的度数之和大于 $n-1$, 则该图一定有条Hamilton路;
- 如果每对结点的度数之和大于 n , 则该图一定有条Hamilton回路.

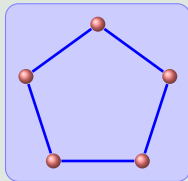
Proof(连通性).

反证法: 设 G 不是连通图, 则 $G = G_1 \cup G_2$, 并且 $G_1 \cap G_2 = \emptyset$,
 $|V_1| = n_1, |V_2| = n_2, n_1 + n_2 = n$, 设 $u \in V_1 \wedge v \in V_2$,
 则 $\deg(u) \leq n_1 - 1 \wedge \deg(v) \leq n_2 - 1, \therefore \deg(u) + \deg(v) \leq n - 2$, 矛盾.



Counterexample

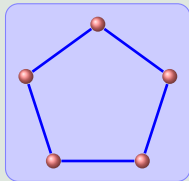
定理的条件不是必要条件的反例



- 有Hamilton回路;
- 但是每对结点的度数之和为 $4 < 5 = n$.

Counterexample

定理的条件不是必要条件的反例



- 有Hamilton回路;
- 但是每对结点的度数之和为 $4 < 5 = n$.

Hamilton回路的一个必要条件

Theorem

设 $G = \langle V, E \rangle$ 是Hamilton图(有Hamilton回路), 对任意的非空结点子集合 $S \subseteq V$ 有:

$$\omega(G - S) \leq |S|$$

其中: $\omega(G - S)$ 是图 $G - S$ 的连通分支数.

Proof.

设 C 是图 G 的一条Hamilton回路, 从 G 中删除 S 中的结点后, 最多将 C 分成 $|S|$ 段路径, 而 $C - S$ 是 $G - S$ 的子图, 因此 $G - S$ 的连通分支数不会大于 $C - S$ 的连通分支数, 故 $\omega(G - S) \leq |S|$. □

Hamilton回路的一个必要条件

Theorem

设 $G = \langle V, E \rangle$ 是Hamilton图(有Hamilton回路), 对任意的非空结点子集合 $S \subseteq V$ 有:

$$\omega(G - S) \leq |S|$$

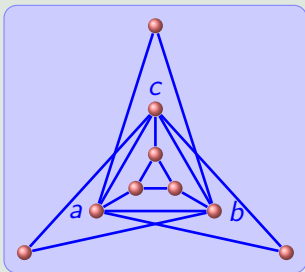
其中: $\omega(G - S)$ 是图 $G - S$ 的连通分支数.

Proof.

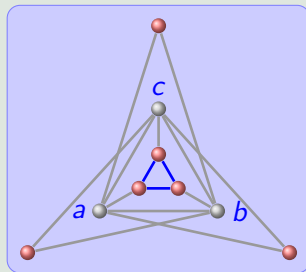
设 C 是图 G 的一条Hamilton回路, 从 G 中删除 S 中的结点后, 最多将 C 分成 $|S|$ 段路径, 而 $C - S$ 是 $G - S$ 的子图, 因此 $G - S$ 的连通分支数不会大于 $C - S$ 的连通分支数, 故 $\omega(G - S) \leq |S|$. □

Example

没有Hamilton回路

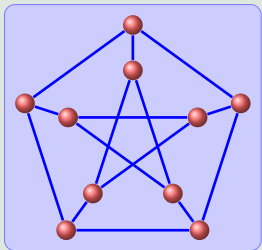


没有Hamilton回路



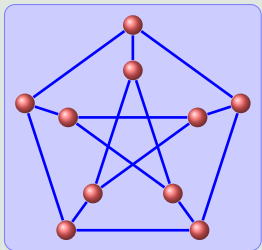
删除 a, b, c 后连通分支数为 $4 > 3$

反例：Petersen图不是Hamilton图(反证法)



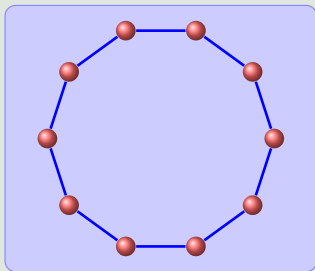
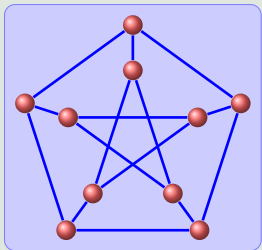
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



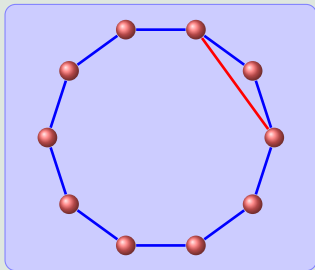
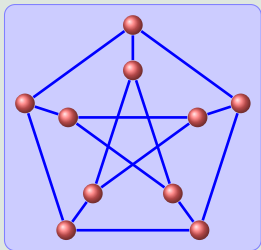
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



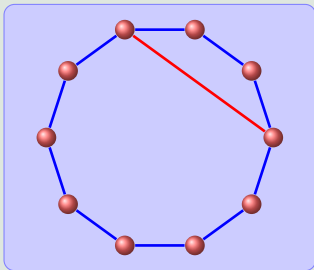
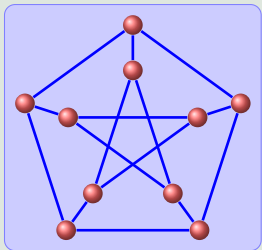
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



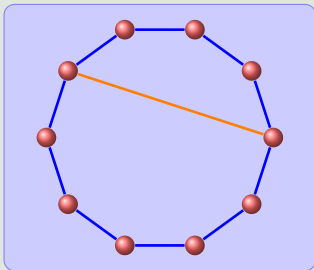
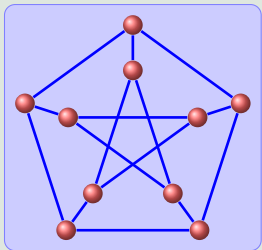
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



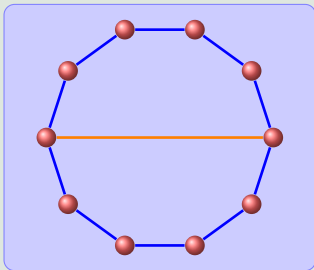
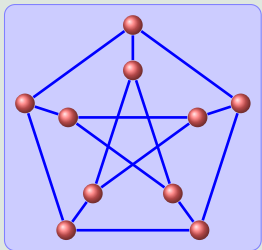
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



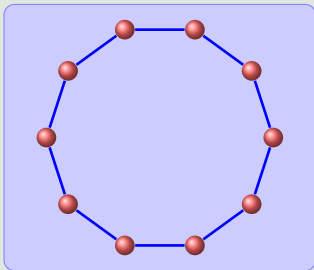
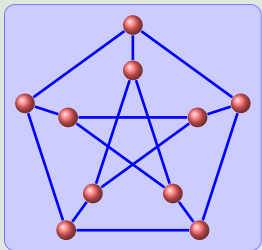
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



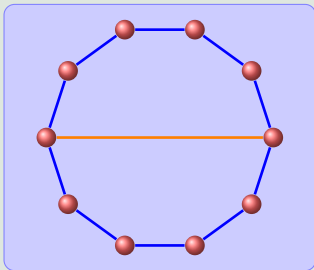
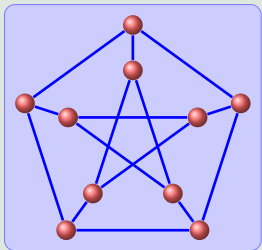
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



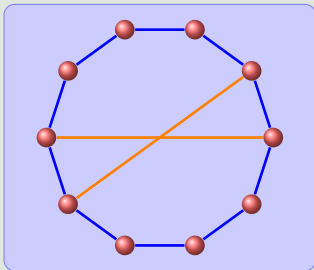
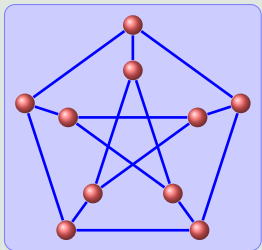
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



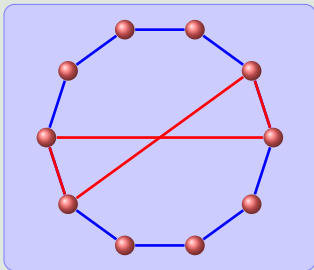
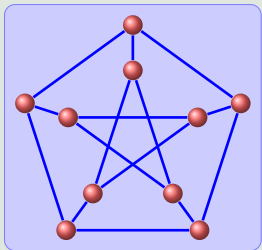
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



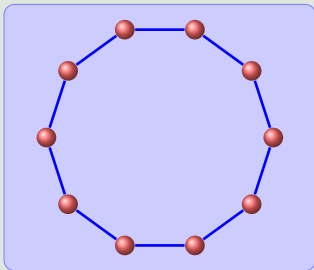
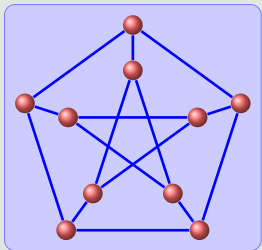
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



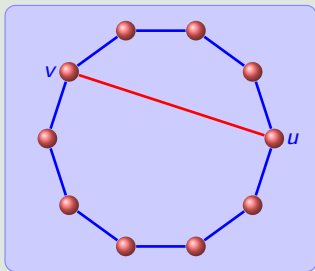
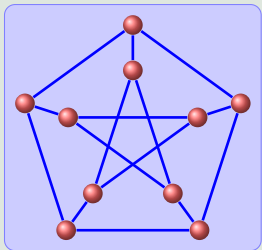
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)



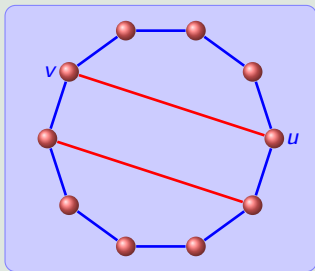
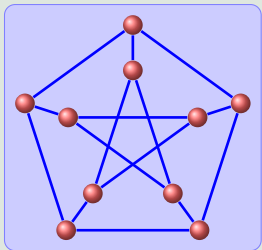
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)

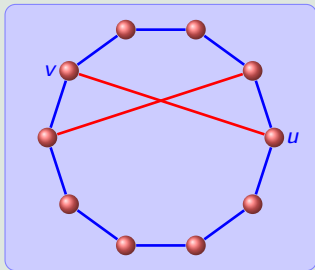


- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

反例：Petersen图不是Hamilton图(反证法)

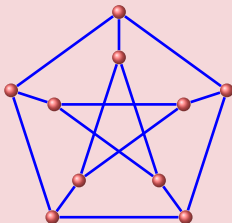


- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.



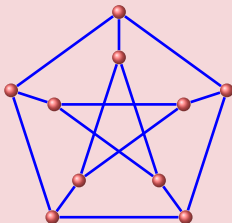
- Petersen图的围长为5. 每个点的度数为3.
- 设有一个Hamilton回路, 如右图所示.
- 则回路中点还要引出一边到回路上与该点不相邻的其他点. 且是在回路上与该结点的距离不小于4.
- 设每个结点引出的第三边与该结点的距离都是5, 则有一个长度为4的基本回路, 矛盾.
- 设某一点 u 的第三边连接在回路上与 u 距离为4的点 v , 则与 u 距离为5的点无论如何引边都产生长度为4的回路, 矛盾.

注释



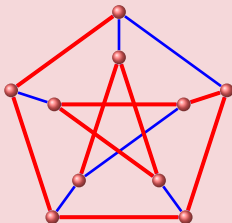
- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

注释



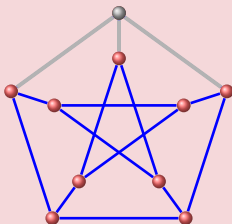
- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

注释



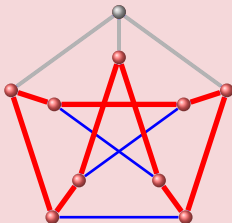
- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

注释



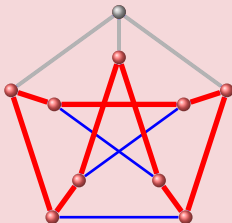
- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

注释



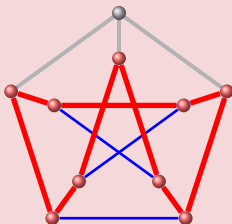
- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

注释



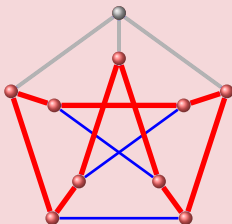
- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
 - 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
 - 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
 - 判断一个图是否Hamilton图是NP-complete problem.
 见<http://en.wikipedia.org/wiki/NP-complete>.
 - 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

注释



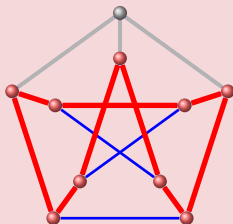
- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

注释



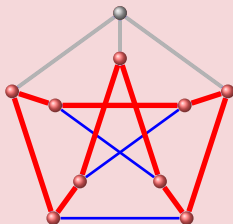
- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

注释



- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

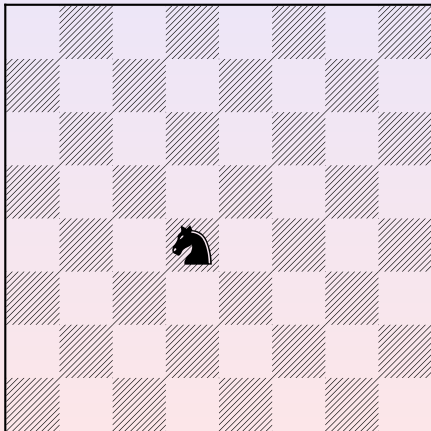
注释



- Petersen图有条Hamilton路径;
- Petersen图删除一个点后是Hamilton图;
- Petersen图的更多性质参见
http://en.wikipedia.org/wiki/Petersen_graph.
- 无向完全图 K_n 共有 $\frac{(n-1)!}{2}$ 个不同的Hamilton回路.
- 有向完全图 K_n^* 共有 $(n-1)!$ 个不同的Hamilton回路.
- 判断一个图是否Hamilton图是NP-complete problem.
见<http://en.wikipedia.org/wiki/NP-complete>.
- 只有暴力法(brute-force search)(也称穷举法(exhaustive search)).

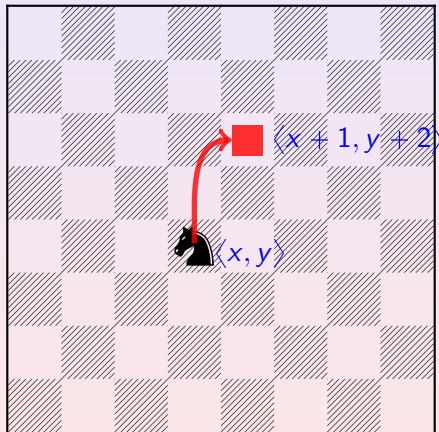
马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour).



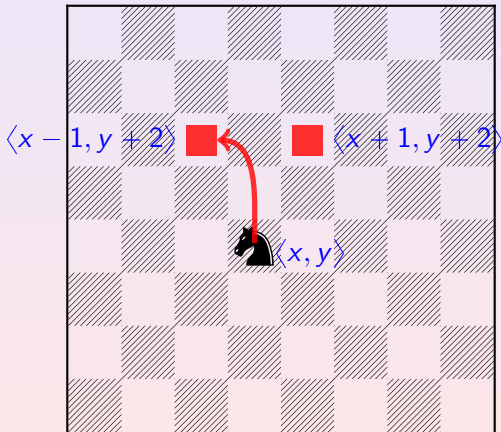
马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour)。



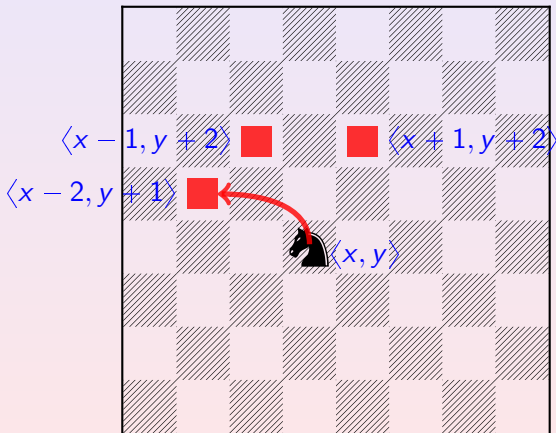
马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour)。



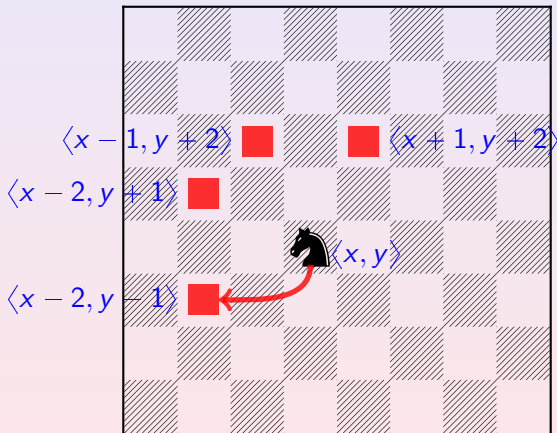
马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour)。



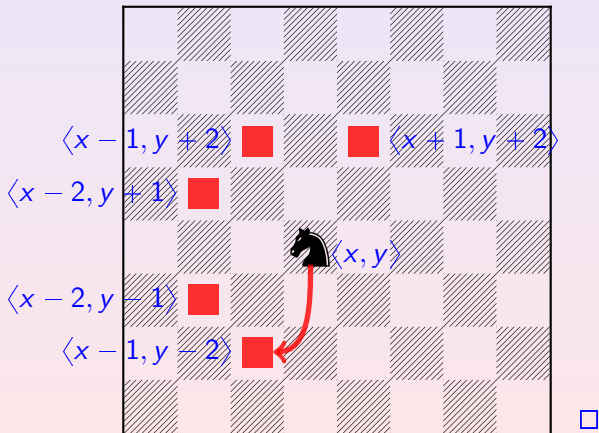
马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour)。



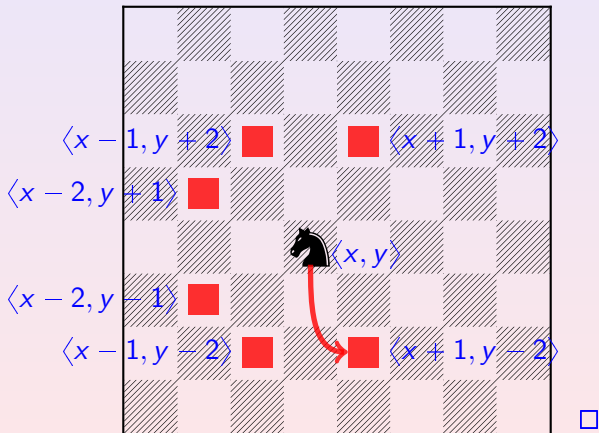
马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour)。



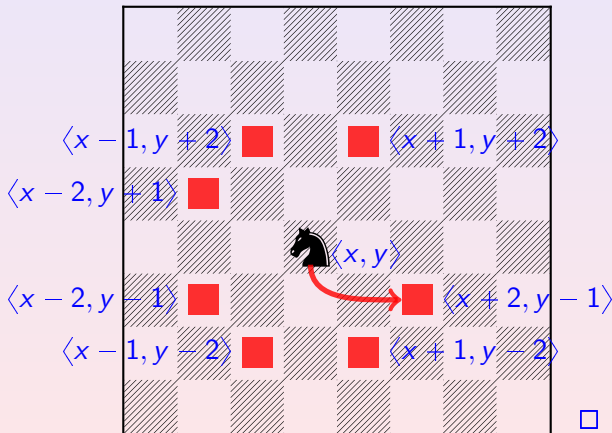
马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour)。



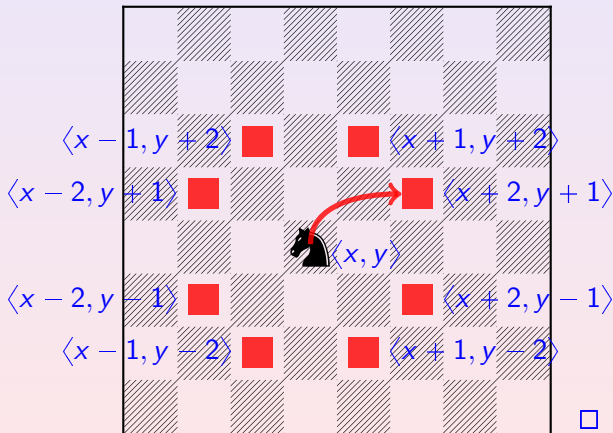
马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour)。



马跳棋盘问题(Knight tour)

国际象棋中马如何从棋盘中的某一格子出发没有重复地跳完棋盘的每一个格子(回到起点称为closed Knight tour)。



A close Knight tour

搜寻马跳棋盘线路的暴力算法

● 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/
int step = 1; /* 全局, 记录当前的步数*/
int x = 0, y = 0; /* 当前马所在的位置,
                  可初始化任意位置*/
```

●

① 如果step不在棋盘范围内, 返回;

② 如果b[x][y]不为零, 说明该位置已被访问, 返回;

③ 将位置(x,y)作为可能跳下的点(共8个), 令step++并记录到b[x][y];

④ 如果step == 65, 说明不剩路, 返回并输出; 否则, 递归调用函数, 计算从(x,y)出发的跳马线路;

● 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位子,  
                  可初始化任意位置*/
```

- 如果x和y不在棋盘范围内, 返回;

- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位子,  
                  可初始化任意位置*/
```

- - ① 如果x和y不在棋盘范围内, 返回;
 - ② 如果b[x][y]非零, 表示该点已被访问, 返回;
 - ③ b[x][y] = step++;
 - ④ 分别设置x和y为马可跳的下个点(共8个), 如x+2和y+1等, goto ①.
 - ⑤ 如果step <= 64, 表示不成功, 进行回溯, 即将b[x][y]重新置零且step--.
- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位子,  
                  可初始化任意位置*/
```

- - ① 如果x和y不在棋盘范围内, 返回;
 - ② 如果b[x][y]非零, 表示该点已被访问, 返回;
 - ③ b[x][y] = step++;
 - ④ 分别设置x和y为马可跳的下个点(共8个), 如x+2和y+1等, goto ①.
 - ⑤ 如果step <= 64, 表示不成功, 进行回溯, 即将b[x][y]重新置零且step--.
- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位子,  
                  可初始化任意位置*/
```

- ① 如果x和y不在棋盘范围内, 返回;
 - ② 如果b[x][y]非零, 表示该点已被访问, 返回;
 - ③ b[x][y] = step++;
 - ④ 分别设置x和y为马可跳的下个点(共8个), 如x+2和y+1等, goto ①.
 - ⑤ 如果step <= 64, 表示不成功, 进行回溯, 即将b[x][y]重新置零且step--.
- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位子,  
                  可初始化任意位置*/
```

- - ① 如果x和y不在棋盘范围内, 返回;
 - ② 如果b[x][y]非零, 表示该点已被访问, 返回;
 - ③ b[x][y] = step++;
 - ④ 分别设置x和y为马可跳的下个点(共8个), 如x+2和y+1等, goto ①.
 - ⑤ 如果step <= 64, 表示不成功, 进行回溯, 即将b[x][y]重新置零且step--.

- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位子,  
                  可初始化任意位置*/
```

- - ① 如果x和y不在棋盘范围内, 返回;
 - ② 如果b[x][y]非零, 表示该点已被访问, 返回;
 - ③ b[x][y] = step++;
 - ④ 分别设置x和y为马可跳的下个点(共8个), 如x+2和y+1等, goto ①.
 - ⑤ 如果step <= 64, 表示不成功, 进行回溯, 即将b[x][y]重新置零且step--.

- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位子,  
                  可初始化任意位置*/
```

- - ① 如果x和y不在棋盘范围内, 返回;
 - ② 如果b[x][y]非零, 表示该点已被访问, 返回;
 - ③ b[x][y] = step++;
 - ④ 分别设置x和y为马可跳的下个点(共8个), 如x+2和y+1等, goto ①.
 - ⑤ 如果step <= 64, 表示不成功, 进行回溯, 即将b[x][y]重新置零且step--.

- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位子,  
                  可初始化任意位置*/
```

- ① 如果x和y不在棋盘范围内, 返回;
 - ② 如果b[x][y]非零, 表示该点已被访问, 返回;
 - ③ b[x][y] = step++;
 - ④ 分别设置x和y为马可跳的下个点(共8个), 如x+2和y+1等, goto ①.
 - ⑤ 如果step <= 64, 表示不成功, 进行回溯, 即将b[x][y]重新置零且step--.
- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

搜寻马跳棋盘线路的暴力算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘, 记录跳马的步数*/  
int step = 1; /* 全局, 记录当前的步数*/  
int x = 0, y = 0; /* 当前马所在的位置,  
                  可初始化任意位置*/
```

- - ① 如果x和y不在棋盘范围内, 返回;
 - ② 如果b[x][y]非零, 表示该点已被访问, 返回;
 - ③ b[x][y] = step++;
 - ④ 分别设置x和y为马可跳的下个点(共8个), 如x+2和y+1等, goto ①.
 - ⑤ 如果step <= 64, 表示不成功, 进行回溯, 即将b[x][y]重新置零且step--.
- 如果step == 65, 打印b, 输出跳马线路.

一个 8×8 的棋盘, 如果要枚举出所有可能的路径, 需要跳马 4×10^{51} 次!

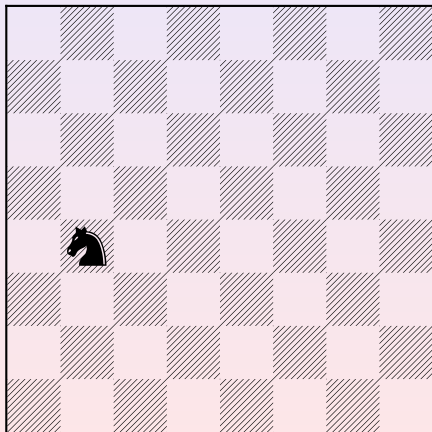
源代码

```
int b[8][8] = {0}; /* see source file ktbrute.c */
int step = 1;
void traceKnight(int y, int x)
{
    /* 测试是否超出棋盘范围 */
    if (y < 0 || y > 8 - 1 || x < 0 || x > 8 - 1) return;
    /* 测试该坐标已遍历 */
    if (board[y][x] != 0) return;
    board[y][x] = step ++; /* 选择该坐标 */
    /* 递归 */
    traceKnight(y - 2, x - 1);
    traceKnight(y - 2, x + 1);
    traceKnight(y - 1, x + 2);
    traceKnight(y + 1, x + 2);
    traceKnight(y + 2, x + 1);
    traceKnight(y + 2, x - 1);
    traceKnight(y + 1, x - 2);
    traceKnight(y - 1, x - 2);
    if (step <= 8 * 8) { /* 回溯 */
        board[y][x] = 0;
        step --;
    }
}
```

}

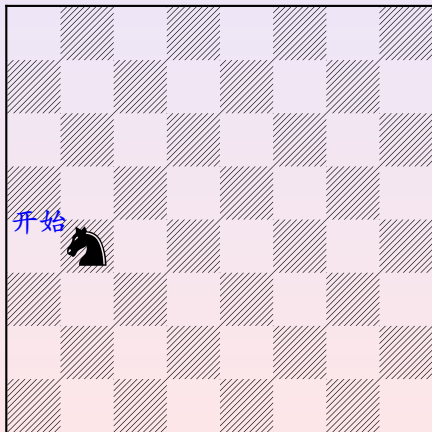
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



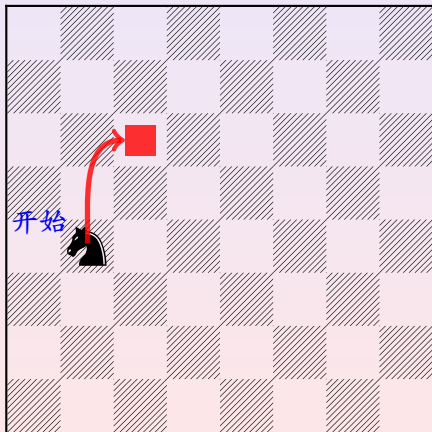
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



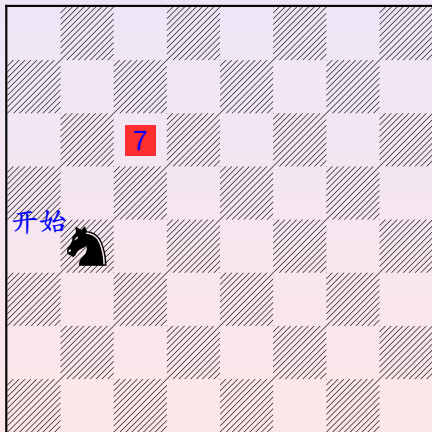
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



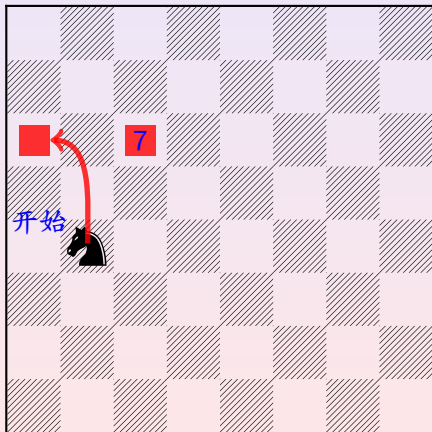
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



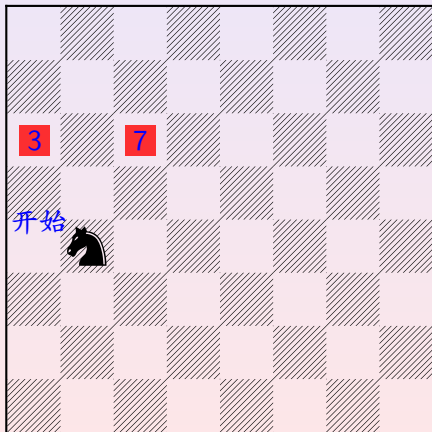
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



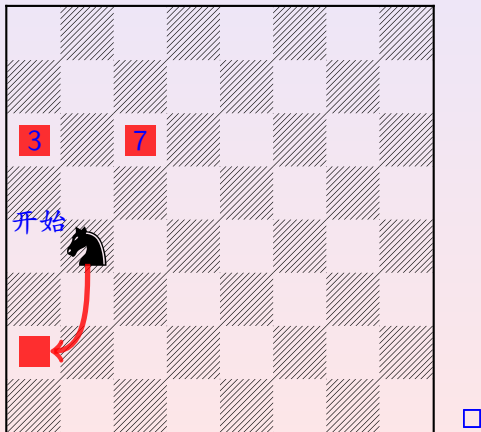
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



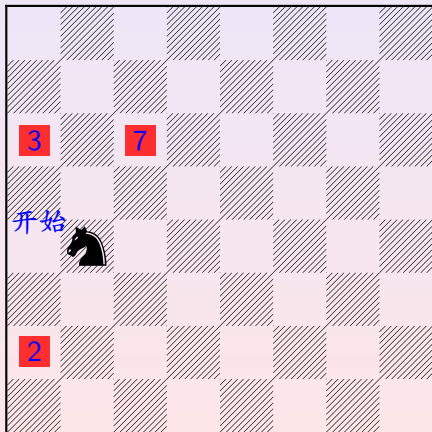
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



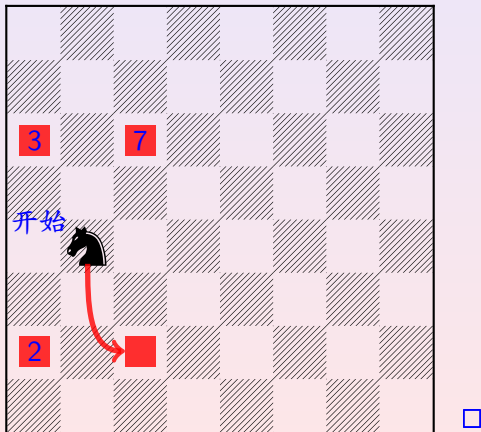
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



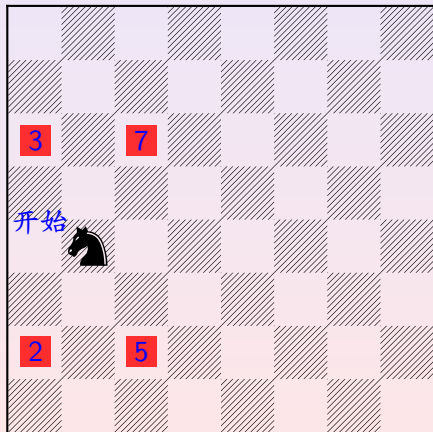
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



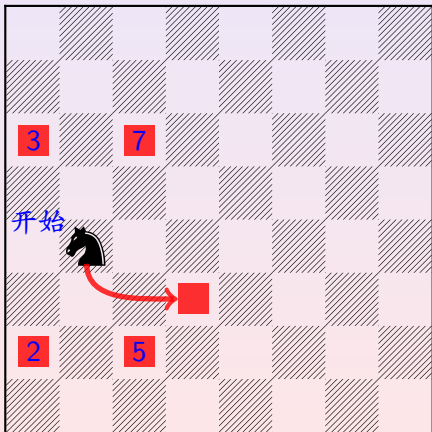
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



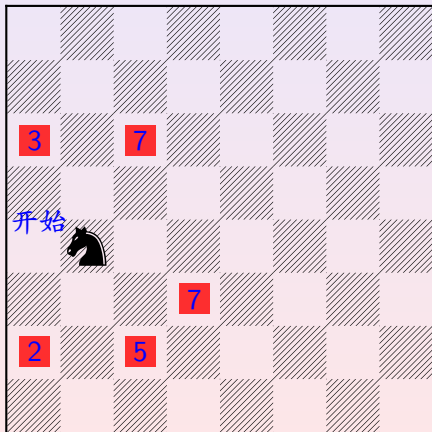
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



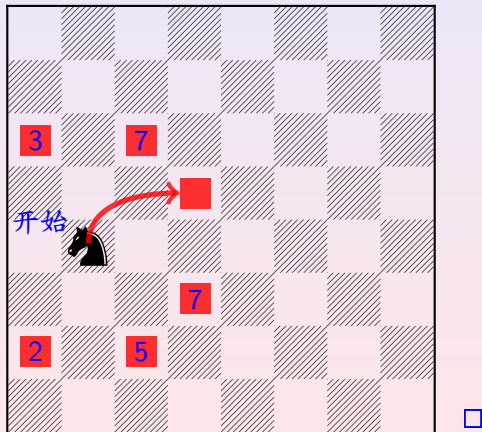
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



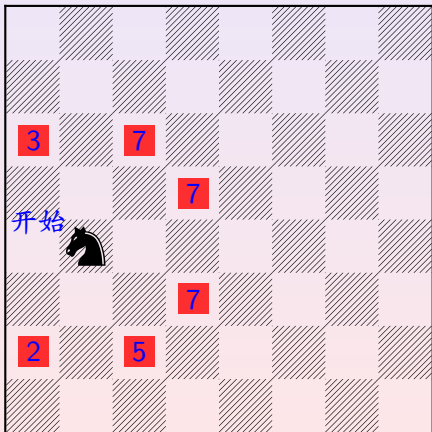
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



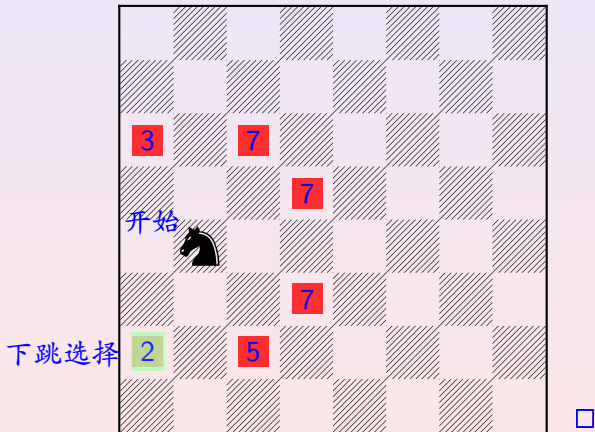
Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



Warnsdorff 启发式算法(1823)

为了避免暴力算法中的非线性递归，算法在可能成为跳马下一跳的8格子中仅选一个，被选格是还有往下继续跳下去可能性最少的一个，即把还有更多可跳可能性的格子留给后续的跳马动作。



Warnsdorff 算法

Warnsdorff 算法

- 初始化:

```
int b[8][8] = { 0 }; /* 棋盘，记录跳马的步数*/
int x = 0, y = 0; /* 当前马所在的位子，
                    可初始化任意位置*/
```

Warnsdorff 算法

Warnsdorff 算法

Warnsdorff 算法

Warnsdorff 算法

Warnsdorff 算法

Warnsdorff 算法

- 打印b, 输出跳马线路.

Warnsdorff 算法

- 打印b, 输出跳马线路.

该算法不能保证对任意的起点都有解存在(算法收敛), 对如 8×8 的棋盘如果起点为(5, 4)将搜索失败. 详见文件warndorff.c.

本章小节

1 路径

- 路径与基本路径
- 连通性
- 回路与基本回路
- 距离与直径

2 路径的应用

- 赋权图与最短路径
- Euler路径与回路
- Hamilton路径与回路